```
// ===== File 1/28: app/__init__.py =====
// (empty)



// ===== File 2/28: app/bootstrap.py =====
from __future__ import annotations

import json
import uuid
from typing import Dict, Iterable, Sequence

from app.db import execute, fetch_all, fetch_one
from app.schema_compat import clear_schema_cache, table_has_columns
from app.task_compat import (
    TASK_PUBLIC_STATUSES,
    announcement_status_to_task_fields,
    builder_category_slug,
    derive_budget_bounds,
    derive_quick_offer_price,
    derive_reward_amount,
    ensure_task_payload,
    is_uuid_like,
    legacy_offer_status_to_canonical,
    primary_map_point,
    primary_source_address,
    route_points_from_payload,
)


CORE_TABLES: Dict[str, str] = {
    "users": """
        CREATE TABLE IF NOT EXISTS users (
          id TEXT PRIMARY KEY,
          email TEXT NOT NULL UNIQUE,
          phone TEXT NULL,
          password_hash TEXT NOT NULL,
          role TEXT NOT NULL DEFAULT 'user',
          created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
          deleted_at TIMESTAMPTZ NULL
        );
    """,
    "announcements": """
        CREATE TABLE IF NOT EXISTS announcements (
          id TEXT PRIMARY KEY,
          user_id TEXT NOT NULL,
          category TEXT NOT NULL,
          title TEXT NOT NULL,
          status TEXT NOT NULL DEFAULT 'active',
          data JSONB NOT NULL DEFAULT '{}'::jsonb,
          created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
          updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
          deleted_at TIMESTAMPTZ
        );
    """,
}


AUX_TABLES: Dict[str, str] = {
    "user_profiles": """
        CREATE TABLE IF NOT EXISTS user_profiles (
          user_id TEXT PRIMARY KEY,
          display_name TEXT NOT NULL DEFAULT 'Пользователь',
          bio TEXT NULL,
          city TEXT NULL,
          home_location JSONB NULL,
          extra JSONB NULL DEFAULT '{}'::jsonb,
          created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
```

```
      updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
    );
""",
"user_stats": """
    CREATE TABLE IF NOT EXISTS user_stats (
      user_id TEXT PRIMARY KEY,
      rating_avg DOUBLE PRECISION NOT NULL DEFAULT 0,
      rating_count INTEGER NOT NULL DEFAULT 0,
      completed_count INTEGER NOT NULL DEFAULT 0,
      cancelled_count INTEGER NOT NULL DEFAULT 0,
      created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
      updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
    );
""",
"reviews": """
    CREATE TABLE IF NOT EXISTS reviews (
      id TEXT PRIMARY KEY,
      task_id TEXT NULL,
      from_user_id TEXT NOT NULL,
      to_user_id TEXT NOT NULL,
      stars INTEGER NOT NULL,
      text TEXT NULL,
      created_at TIMESTAMPTZ NOT NULL DEFAULT now()
    );
""",
"user_devices": """
    CREATE TABLE IF NOT EXISTS user_devices (
      id TEXT PRIMARY KEY,
      user_id TEXT NOT NULL,
      platform TEXT NOT NULL,
      device_id TEXT NOT NULL,
      push_token TEXT NULL,
      locale TEXT NULL,
      timezone TEXT NULL,
      device_name TEXT NULL,
      created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
      last_seen_at TIMESTAMPTZ NOT NULL DEFAULT now(),
      deleted_at TIMESTAMPTZ NULL
    );
""",
"announcement_offers": """
    CREATE TABLE IF NOT EXISTS announcement_offers (
      id TEXT PRIMARY KEY,
      announcement_id TEXT NOT NULL,
      performer_id TEXT NOT NULL,
      message TEXT NULL,
      proposed_price INTEGER NULL,
      status TEXT NOT NULL DEFAULT 'pending',
      chat_thread_id TEXT NULL,
      created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
      deleted_at TIMESTAMPTZ NULL
    );
""",
"chat_threads": """
    CREATE TABLE IF NOT EXISTS chat_threads (
      id TEXT PRIMARY KEY,
      kind TEXT NOT NULL,
      task_id TEXT NULL,
      offer_id TEXT NULL,
      created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
      last_message_at TIMESTAMPTZ NULL
    );
""",
"chat_participants": """
    CREATE TABLE IF NOT EXISTS chat_participants (
      thread_id TEXT NOT NULL,
      user_id TEXT NOT NULL,
      role TEXT NOT NULL,
```

```
      joined_at TIMESTAMPTZ NOT NULL DEFAULT now(),
      left_at TIMESTAMPTZ NULL,
      last_read_message_id TEXT NULL,
      PRIMARY KEY (thread_id, user_id)
    );
""",
"chat_messages": """
    CREATE TABLE IF NOT EXISTS chat_messages (
      id TEXT PRIMARY KEY,
      thread_id TEXT NOT NULL,
      sender_id TEXT NOT NULL,
      type TEXT NOT NULL,
      text TEXT NOT NULL,
      is_blocked BOOLEAN NOT NULL DEFAULT FALSE,
      blocked_reason TEXT NULL,
      created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
      edited_at TIMESTAMPTZ NULL,
      deleted_at TIMESTAMPTZ NULL
    );
""",
"message_reads": """
    CREATE TABLE IF NOT EXISTS message_reads (
      message_id TEXT NOT NULL,
      user_id TEXT NOT NULL,
      read_at TIMESTAMPTZ NOT NULL DEFAULT now(),
      PRIMARY KEY (message_id, user_id)
    );
""",
"reports": """
    CREATE TABLE IF NOT EXISTS reports (
      id TEXT PRIMARY KEY,
      reporter_id TEXT NOT NULL,
      target_type TEXT NOT NULL,
      target_id TEXT NOT NULL,
      reason_code TEXT NOT NULL,
      reason_text TEXT NULL,
      status TEXT NOT NULL,
      resolution TEXT NULL,
      resolved_by TEXT NULL,
      moderator_comment TEXT NULL,
      created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
      resolved_at TIMESTAMPTZ NULL
    );
""",
"moderation_actions": """
    CREATE TABLE IF NOT EXISTS moderation_actions (
      id TEXT PRIMARY KEY,
      moderator_id TEXT NOT NULL,
      action_type TEXT NOT NULL,
      target_type TEXT NOT NULL,
      target_id TEXT NOT NULL,
      reason TEXT NULL,
      payload JSONB NOT NULL DEFAULT '{}'::jsonb,
      created_at TIMESTAMPTZ NOT NULL DEFAULT now()
    );
""",
"user_restrictions": """
    CREATE TABLE IF NOT EXISTS user_restrictions (
      id TEXT PRIMARY KEY,
      user_id TEXT NOT NULL,
      type TEXT NOT NULL,
      status TEXT NOT NULL,
      issued_by TEXT NOT NULL,
      starts_at TIMESTAMPTZ NOT NULL DEFAULT now(),
      ends_at TIMESTAMPTZ NULL,
      revoked_at TIMESTAMPTZ NULL
    );
""",
```

```
    "notifications": """

        CREATE TABLE IF NOT EXISTS notifications (
          id TEXT PRIMARY KEY,
          user_id TEXT NOT NULL,
          type TEXT NOT NULL,
          body TEXT NOT NULL,
          payload JSONB NOT NULL DEFAULT '{}'::jsonb,
          is_read BOOLEAN NOT NULL DEFAULT FALSE,
          created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
          read_at TIMESTAMPTZ NULL
        );
    """,
}


TASK_DOMAIN_TABLES: Dict[str, str] = {
    "task_assignments": """
        CREATE TABLE IF NOT EXISTS task_assignments (
          id UUID PRIMARY KEY,
          task_id UUID NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,
          offer_id UUID NOT NULL REFERENCES task_offers(id) ON DELETE CASCADE,
          customer_id UUID NOT NULL REFERENCES users(id),
          performer_id UUID NOT NULL REFERENCES users(id),
          assignment_status TEXT NOT NULL DEFAULT 'assigned'
            CHECK (assignment_status IN ('assigned', 'in_progress', 'completed', 'cancelled')),
          execution_stage TEXT NOT NULL DEFAULT 'accepted'
                   CHECK  (execution_stage  IN  ('accepted',  'en_route',  'on_site',  'in_progress',  'handoff',
'completed', 'cancelled')),
          route_visibility TEXT NOT NULL DEFAULT 'performer_only'
            CHECK (route_visibility IN ('hidden', 'performer_only', 'customer_visible')),
          chat_thread_id UUID NULL,
          created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
          updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
          started_at TIMESTAMPTZ NULL,
          completed_at TIMESTAMPTZ NULL,
          cancelled_at TIMESTAMPTZ NULL,
          cancellation_reason TEXT NULL
        );
    """,
    "task_assignment_events": """
        CREATE TABLE IF NOT EXISTS task_assignment_events (
          id UUID PRIMARY KEY,
          assignment_id UUID NOT NULL REFERENCES task_assignments(id) ON DELETE CASCADE,
          task_id UUID NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,
          event_type TEXT NOT NULL
            CHECK (event_type IN ('assignment_status', 'execution_stage', 'route_visibility', 'chat_bound')),
          from_value TEXT NULL,
          to_value TEXT NOT NULL,
          changed_by UUID NULL REFERENCES users(id),
          payload JSONB NOT NULL DEFAULT '{}'::jsonb,
          created_at TIMESTAMPTZ NOT NULL DEFAULT now()
        );
    """,
}


COMPAT_DDLS: Iterable[str] = (
    "CREATE EXTENSION IF NOT EXISTS postgis;",
    "ALTER TYPE offer_status ADD VALUE IF NOT EXISTS 'withdrawn_by_sender';",
    "ALTER TABLE users ADD COLUMN IF NOT EXISTS phone TEXT NULL;",
    "ALTER TABLE users ADD COLUMN IF NOT EXISTS deleted_at TIMESTAMPTZ NULL;",
    "ALTER TABLE announcements ADD COLUMN IF NOT EXISTS location_point geography(Point,4326);",
    "ALTER TABLE user_profiles ADD COLUMN IF NOT EXISTS display_name TEXT NULL;",
    "ALTER TABLE user_profiles ADD COLUMN IF NOT EXISTS bio TEXT NULL;",
    "ALTER TABLE user_profiles ADD COLUMN IF NOT EXISTS city TEXT NULL;",
    "ALTER TABLE user_profiles ADD COLUMN IF NOT EXISTS home_location JSONB NULL;",
    "ALTER TABLE user_profiles ADD COLUMN IF NOT EXISTS extra JSONB NULL DEFAULT '{}'::jsonb;",
```

```
"ALTER TABLE user_profiles ADD COLUMN IF NOT EXISTS created_at TIMESTAMPTZ NOT NULL DEFAULT now();",
"ALTER TABLE user_profiles ADD COLUMN IF NOT EXISTS updated_at TIMESTAMPTZ NOT NULL DEFAULT now();",
"ALTER TABLE user_stats ADD COLUMN IF NOT EXISTS rating_avg DOUBLE PRECISION NOT NULL DEFAULT 0;",
"ALTER TABLE user_stats ADD COLUMN IF NOT EXISTS rating_count INTEGER NOT NULL DEFAULT 0;",
"ALTER TABLE user_stats ADD COLUMN IF NOT EXISTS completed_count INTEGER NOT NULL DEFAULT 0;",
"ALTER TABLE user_stats ADD COLUMN IF NOT EXISTS cancelled_count INTEGER NOT NULL DEFAULT 0;",
"ALTER TABLE user_stats ADD COLUMN IF NOT EXISTS created_at TIMESTAMPTZ NOT NULL DEFAULT now();",
"ALTER TABLE user_stats ADD COLUMN IF NOT EXISTS updated_at TIMESTAMPTZ NOT NULL DEFAULT now();",
"ALTER TABLE reviews ADD COLUMN IF NOT EXISTS id TEXT NULL;",
"ALTER TABLE reviews ADD COLUMN IF NOT EXISTS task_id TEXT NULL;",
"ALTER TABLE reviews ADD COLUMN IF NOT EXISTS from_user_id TEXT NULL;",
"ALTER TABLE reviews ADD COLUMN IF NOT EXISTS to_user_id TEXT NULL;",
"ALTER TABLE reviews ADD COLUMN IF NOT EXISTS stars INTEGER NULL;",
"ALTER TABLE reviews ADD COLUMN IF NOT EXISTS text TEXT NULL;",
"ALTER TABLE reviews ADD COLUMN IF NOT EXISTS author_role TEXT NULL;",
"ALTER TABLE reviews ADD COLUMN IF NOT EXISTS target_role TEXT NULL;",
"ALTER TABLE reviews ADD COLUMN IF NOT EXISTS created_at TIMESTAMPTZ NOT NULL DEFAULT now();",
"ALTER TABLE user_devices ADD COLUMN IF NOT EXISTS id TEXT NULL;",
"ALTER TABLE user_devices ADD COLUMN IF NOT EXISTS user_id TEXT NULL;",
"ALTER TABLE user_devices ADD COLUMN IF NOT EXISTS platform TEXT NULL;",
"ALTER TABLE user_devices ADD COLUMN IF NOT EXISTS device_id TEXT NULL;",
"ALTER TABLE user_devices ADD COLUMN IF NOT EXISTS push_token TEXT NULL;",
"ALTER TABLE user_devices ADD COLUMN IF NOT EXISTS locale TEXT NULL;",
"ALTER TABLE user_devices ADD COLUMN IF NOT EXISTS timezone TEXT NULL;",
"ALTER TABLE user_devices ADD COLUMN IF NOT EXISTS device_name TEXT NULL;",
"ALTER TABLE user_devices ADD COLUMN IF NOT EXISTS created_at TIMESTAMPTZ NOT NULL DEFAULT now();",
"ALTER TABLE user_devices ADD COLUMN IF NOT EXISTS last_seen_at TIMESTAMPTZ NOT NULL DEFAULT now();",
"ALTER TABLE user_devices ADD COLUMN IF NOT EXISTS deleted_at TIMESTAMPTZ NULL;",
"ALTER TABLE announcement_offers ADD COLUMN IF NOT EXISTS id TEXT NULL;",
"ALTER TABLE announcement_offers ADD COLUMN IF NOT EXISTS announcement_id TEXT NULL;",
"ALTER TABLE announcement_offers ADD COLUMN IF NOT EXISTS performer_id TEXT NULL;",
"ALTER TABLE announcement_offers ADD COLUMN IF NOT EXISTS message TEXT NULL;",
"ALTER TABLE announcement_offers ADD COLUMN IF NOT EXISTS proposed_price INTEGER NULL;",
"ALTER TABLE announcement_offers ADD COLUMN IF NOT EXISTS status TEXT NOT NULL DEFAULT 'pending';",
"ALTER TABLE announcement_offers ADD COLUMN IF NOT EXISTS chat_thread_id TEXT NULL;",
"ALTER TABLE announcement_offers ADD COLUMN IF NOT EXISTS created_at TIMESTAMPTZ NOT NULL DEFAULT now();",
"ALTER TABLE announcement_offers ADD COLUMN IF NOT EXISTS deleted_at TIMESTAMPTZ NULL;",
"ALTER TABLE tasks ADD COLUMN IF NOT EXISTS deleted_at TIMESTAMPTZ NULL;",
"ALTER TABLE tasks ADD COLUMN IF NOT EXISTS budget_min NUMERIC NULL;",
"ALTER TABLE tasks ADD COLUMN IF NOT EXISTS budget_max NUMERIC NULL;",
"ALTER TABLE tasks ADD COLUMN IF NOT EXISTS quick_offer_price NUMERIC NULL;",
"ALTER TABLE tasks ADD COLUMN IF NOT EXISTS reoffer_policy TEXT NOT NULL DEFAULT 'blocked_after_reject';",
"ALTER TABLE task_offers ADD COLUMN IF NOT EXISTS pricing_mode TEXT NOT NULL DEFAULT 'counter_price';",
"ALTER TABLE task_offers ADD COLUMN IF NOT EXISTS agreed_price NUMERIC NULL;",
"ALTER TABLE task_offers ADD COLUMN IF NOT EXISTS minimum_price_accepted BOOLEAN NOT NULL DEFAULT FALSE;",
"ALTER TABLE task_offers ADD COLUMN IF NOT EXISTS can_reoffer BOOLEAN NOT NULL DEFAULT TRUE;",
"ALTER TABLE task_offers ADD COLUMN IF NOT EXISTS reoffer_block_reason TEXT NULL;",
"ALTER TABLE task_offers ADD COLUMN IF NOT EXISTS chat_thread_id UUID NULL;",
"ALTER TABLE task_offers ADD COLUMN IF NOT EXISTS accepted_at TIMESTAMPTZ NULL;",
"ALTER TABLE task_offers ADD COLUMN IF NOT EXISTS rejected_at TIMESTAMPTZ NULL;",
"ALTER TABLE task_offers ADD COLUMN IF NOT EXISTS withdrawn_at TIMESTAMPTZ NULL;",
"ALTER TABLE chat_threads ADD COLUMN IF NOT EXISTS assignment_id UUID NULL;",
"ALTER TABLE chat_threads ADD COLUMN IF NOT EXISTS archived_at TIMESTAMPTZ NULL;",
"""
UPDATE announcements
SET location_point = ST_SetSRID(
    ST_MakePoint(
        COALESCE(
            NULLIF(data -> 'point' ->> 'lon', ''),
            NULLIF(data -> 'pickup_point' ->> 'lon', ''),
            NULLIF(data -> 'help_point' ->> 'lon', '')
        )::double precision,
        COALESCE(
            NULLIF(data -> 'point' ->> 'lat', ''),
            NULLIF(data -> 'pickup_point' ->> 'lat', ''),
            NULLIF(data -> 'help_point' ->> 'lat', '')
        )::double precision
    ),
```

```
        4326
    )::geography
    WHERE location_point IS NULL
      AND (
            jsonb_typeof(data -> 'point') = 'object'
            OR jsonb_typeof(data -> 'pickup_point') = 'object'
            OR jsonb_typeof(data -> 'help_point') = 'object'
      )
      AND COALESCE(
            NULLIF(data -> 'point' ->> 'lat', ''),
            NULLIF(data -> 'pickup_point' ->> 'lat', ''),
            NULLIF(data -> 'help_point' ->> 'lat', '')
          ) ~ '^-?[0-9]+(\\.[0-9]+)?$'
      AND COALESCE(
            NULLIF(data -> 'point' ->> 'lon', ''),
            NULLIF(data -> 'pickup_point' ->> 'lon', ''),
            NULLIF(data -> 'help_point' ->> 'lon', '')
          ) ~ '^-?[0-9]+(\\.[0-9]+)?$';
    """,
    "CREATE UNIQUE INDEX IF NOT EXISTS idx_users_phone_unique ON users(phone) WHERE phone IS NOT NULL;",
    "CREATE UNIQUE INDEX IF NOT EXISTS idx_reviews_id_unique ON reviews(id) WHERE id IS NOT NULL;",
      "CREATE UNIQUE INDEX IF NOT EXISTS ux_reviews_task_from_user ON reviews(task_id, from_user_id) WHERE
task_id IS NOT NULL AND from_user_id IS NOT NULL;",
                "CREATE UNIQUE INDEX IF NOT EXISTS idx_announcement_offers_unique_pending ON
announcement_offers(announcement_id, performer_id) WHERE deleted_at IS NULL AND status = 'pending';",
)


INDEX_DDLS: Iterable[tuple[str, str, Sequence[str]]] = (
      ("announcements", "CREATE INDEX IF NOT EXISTS idx_announcements_user_id ON announcements(user_id);",
("user_id",)),
    (
        "announcements",
        "CREATE INDEX IF NOT EXISTS idx_announcements_created_at ON announcements(created_at DESC);",
        ("created_at",),
    ),
      ("announcements", "CREATE INDEX IF NOT EXISTS idx_announcements_status ON announcements(status);",
("status",)),
    (
        "announcements",
            "CREATE INDEX IF NOT EXISTS idx_announcements_location_point_gist ON announcements USING GIST
(location_point);",
        ("location_point",),
    ),
    (
        "reviews",
        "CREATE INDEX IF NOT EXISTS idx_reviews_to_user_created_at ON reviews(to_user_id, created_at DESC);",
        ("to_user_id", "created_at"),
    ),
    (
        "reviews",
           "CREATE INDEX IF NOT EXISTS idx_reviews_from_user_created_at ON reviews(from_user_id, created_at
DESC);",
        ("from_user_id", "created_at"),
    ),
    (
        "reviews",
             "CREATE INDEX IF NOT EXISTS idx_reviews_to_user_target_role_created_at ON reviews(to_user_id,
target_role, created_at DESC);",
        ("to_user_id", "target_role", "created_at"),
    ),
    (
        "user_devices",
        "CREATE INDEX IF NOT EXISTS idx_user_devices_device_id ON user_devices(device_id);",
        ("device_id",),
    ),
    (
        "user_devices",
```

```
                "CREATE  INDEX  IF  NOT  EXISTS  idx_user_devices_user_id_deleted_at  ON  user_devices(user_id,
deleted_at);",
        ("user_id", "deleted_at"),
    ),
    (
        "user_devices",
        "CREATE INDEX IF NOT EXISTS idx_user_devices_push_token ON user_devices(push_token);",

        ("push_token",),
    ),
    (
        "announcement_offers",
                            "CREATE   INDEX   IF   NOT   EXISTS   idx_announcement_offers_announcement_id   ON
announcement_offers(announcement_id);",
        ("announcement_id",),
    ),
    (
        "announcement_offers",
                             "CREATE   INDEX   IF   NOT   EXISTS   idx_announcement_offers_performer_id   ON
announcement_offers(performer_id);",
        ("performer_id",),
    ),
    (
        "announcement_offers",
                        "CREATE   UNIQUE   INDEX   IF   NOT   EXISTS   idx_announcement_offers_chat_thread_id   ON
announcement_offers(chat_thread_id) WHERE chat_thread_id IS NOT NULL;",
        ("chat_thread_id",),
    ),
    (
        "announcement_offers",
         "CREATE INDEX IF NOT EXISTS idx_announcement_offers_status_deleted_at ON announcement_offers(status,
deleted_at);",
        ("status", "deleted_at"),
    ),
    (
        "chat_threads",
           "CREATE  INDEX  IF  NOT  EXISTS  idx_chat_threads_last_message_at  ON  chat_threads(last_message_at  DESC
NULLS LAST);",
        ("last_message_at",),
    ),
    (
        "chat_threads",
        "CREATE INDEX IF NOT EXISTS idx_chat_threads_offer_id ON chat_threads(offer_id);",
        ("offer_id",),
    ),
    (
        "chat_threads",
        "CREATE INDEX IF NOT EXISTS idx_chat_threads_assignment_id ON chat_threads(assignment_id);",
        ("assignment_id",),
    ),
    (
        "chat_participants",
        "CREATE INDEX IF NOT EXISTS idx_chat_participants_user_id ON chat_participants(user_id);",
        ("user_id",),
    ),
    (
        "chat_messages",
        "CREATE INDEX IF NOT EXISTS idx_chat_messages_thread_created_at ON chat_messages(thread_id, created_at
DESC);",
        ("thread_id", "created_at"),
    ),
       ("reports",  "CREATE  INDEX  IF  NOT  EXISTS  idx_reports_target  ON  reports(target_type,  target_id);",
("target_type", "target_id")),
    (
        "reports",
        "CREATE INDEX IF NOT EXISTS idx_reports_status_created_at ON reports(status, created_at DESC);",
        ("status", "created_at"),
    ),
```

```
    (
        "reports",
            "CREATE INDEX IF NOT EXISTS idx_reports_resolved_created_at ON reports(resolved_at, created_at
DESC);",
        ("resolved_at", "created_at"),
    ),
    (
        "moderation_actions",
             "CREATE INDEX IF NOT EXISTS idx_moderation_actions_target ON moderation_actions(target_type,
target_id);",
        ("target_type", "target_id"),
    ),
    (
        "moderation_actions",
            "CREATE INDEX IF NOT EXISTS idx_moderation_actions_created_at ON moderation_actions(created_at
DESC);",
        ("created_at",),
    ),
    (
        "user_restrictions",
        "CREATE INDEX IF NOT EXISTS idx_user_restrictions_user_status ON user_restrictions(user_id, status);",
        ("user_id", "status"),
    ),
    (
        "user_restrictions",
            "CREATE INDEX IF NOT EXISTS idx_user_restrictions_user_revoked_at ON user_restrictions(user_id,
revoked_at);",
        ("user_id", "revoked_at"),
    ),
    (
        "notifications",
          "CREATE INDEX IF NOT EXISTS idx_notifications_user_created_at ON notifications(user_id, created_at
DESC);",
        ("user_id", "created_at"),
    ),
    (
        "tasks",
                    "CREATE INDEX IF NOT EXISTS idx_tasks_deleted_status ON tasks(deleted_at, status,
moderation_status);",
        ("deleted_at", "status", "moderation_status"),
    ),
    (
        "tasks",
                  "CREATE INDEX IF NOT EXISTS idx_tasks_budget_range ON tasks(budget_min, budget_max,
quick_offer_price);",
        ("budget_min", "budget_max", "quick_offer_price"),
    ),
    (
        "task_offers",
         "CREATE UNIQUE INDEX IF NOT EXISTS ux_task_offers_chat_thread_id ON task_offers(chat_thread_id) WHERE
chat_thread_id IS NOT NULL;",
        ("chat_thread_id",),
    ),
    (
        "task_assignments",
        "CREATE INDEX IF NOT EXISTS idx_task_assignments_task_id ON task_assignments(task_id);",
        ("task_id",),
    ),
    (
        "task_assignments",
                        "CREATE INDEX IF NOT EXISTS idx_task_assignments_performer_status_updated ON
task_assignments(performer_id, assignment_status, updated_at DESC);",
        ("performer_id", "assignment_status", "updated_at"),
    ),
    (
        "task_assignments",
                        "CREATE INDEX IF NOT EXISTS idx_task_assignments_customer_status_updated ON
task_assignments(customer_id, assignment_status, updated_at DESC);",
```

```
        ("customer_id", "assignment_status", "updated_at"),
    ),
    (
        "task_assignments",
        "CREATE UNIQUE INDEX IF NOT EXISTS ux_task_assignments_offer_id ON task_assignments(offer_id);",
        ("offer_id",),
    ),
    (
        "task_assignments",
                        "CREATE   UNIQUE   INDEX   IF   NOT   EXISTS   ux_task_assignments_chat_thread_id   ON
task_assignments(chat_thread_id) WHERE chat_thread_id IS NOT NULL;",
        ("chat_thread_id",),
    ),
    (
        "task_assignments",
         "CREATE UNIQUE INDEX IF NOT EXISTS ux_task_assignments_active_task ON task_assignments(task_id) WHERE
assignment_status IN ('assigned', 'in_progress');",
        ("task_id", "assignment_status"),
    ),
    (
        "task_assignment_events",
                       "CREATE   INDEX   IF   NOT   EXISTS   idx_task_assignment_events_assignment_created   ON
task_assignment_events(assignment_id, created_at DESC);",
        ("assignment_id", "created_at"),
    ),
)


def table_exists(table_name: str) -> bool:
    row = fetch_one(f"SELECT to_regclass('public.{table_name}') IS NOT NULL")
    return bool(row and row[0])


def ensure_core_tables() -> None:
    for table_name, ddl in CORE_TABLES.items():
        if not table_exists(table_name):
            execute(ddl)


def ensure_auxiliary_tables() -> None:
    for table_name, ddl in AUX_TABLES.items():
        if not table_exists(table_name):
            execute(ddl)


def ensure_task_domain_tables() -> None:
    for table_name, ddl in TASK_DOMAIN_TABLES.items():
        if not table_exists(table_name):
            execute(ddl)


def ensure_compat_columns() -> None:
    for ddl in COMPAT_DDLS:
        execute(ddl)


def ensure_indexes() -> None:
    clear_schema_cache()
    for table_name, ddl, required_columns in INDEX_DDLS:
        if table_has_columns(table_name, required_columns):
            execute(ddl)


def ensure_chat_thread_kind_compat() -> None:
    row = fetch_one(
        """
        SELECT data_type, udt_name
        FROM information_schema.columns
```

```
        WHERE table_schema = 'public'
          AND table_name = 'chat_threads'
          AND column_name = 'kind'
        """,
    )
    if not row:
        return

    data_type = str(row[0] or "").lower()
    udt_name = str(row[1] or "").lower()
    if data_type == "user-defined" and udt_name == "chat_thread_kind":
        execute("ALTER TYPE chat_thread_kind ADD VALUE IF NOT EXISTS 'offer';")


def _category_id_for_slug(slug: str) -> str | None:
    row = fetch_one(
        """
        SELECT id::text
        FROM categories
        WHERE slug = %s
        LIMIT 1
        """,
        (slug,),
    )

    if row and row[0]:
        return str(row[0])
    return None


def _task_exists(task_id: str) -> bool:
    row = fetch_one("SELECT 1 FROM tasks WHERE id::text = %s", (task_id,))
    return bool(row)


def _user_exists(user_id: str) -> bool:
    row = fetch_one("SELECT 1 FROM users WHERE id::text = %s", (user_id,))
    return bool(row)


def _ensure_task_route_points(task_id: str, data: dict[str, object]) -> None:
    if fetch_one("SELECT 1 FROM task_route_points WHERE task_id::text = %s LIMIT 1", (task_id,)):
        return

    for point in route_points_from_payload(task_id, data):
        raw_point = point.get("point")
        if not isinstance(raw_point, tuple):
            continue

        execute(
            """
            INSERT INTO task_route_points (
                id, task_id, point_order, title, address_text, point, point_kind, created_at
            )
            VALUES (
                %s, %s, %s, %s, %s,
                ST_SetSRID(ST_MakePoint(%s, %s), 4326)::geography,
                %s, now()
            )
            ON CONFLICT DO NOTHING
            """,
            (
                str(uuid.uuid4()),
                task_id,
                int(point["point_order"]),
                point.get("title"),
                point.get("address_text"),
                raw_point[1],
```

```
                raw_point[0],
                point.get("point_kind"),
            ),
        )


def backfill_legacy_tasks() -> None:
    rows = fetch_all(
        """
        SELECT
            id::text,
            user_id,
            category,
            title,
            status,
            data,
            created_at,
            updated_at,
            deleted_at,
                   CASE  WHEN  location_point  IS  NULL  THEN  NULL  ELSE  ST_Y(location_point::geometry)  END  AS
location_lat,
            CASE WHEN location_point IS NULL THEN NULL ELSE ST_X(location_point::geometry) END AS location_lon
        FROM announcements
        ORDER BY created_at ASC
        """
    )

    for row in rows:
        ann_id = str(row[0])
        user_id = str(row[1] or "")
        category = str(row[2] or "")
        title = str(row[3] or "")
        status = str(row[4] or "")
        raw_data = row[5] if isinstance(row[5], dict) else {}
        created_at = row[6]
        updated_at = row[7] or row[6]
        deleted_at = row[8]
        location_lat = row[9]
        location_lon = row[10]

        if not is_uuid_like(ann_id) or not is_uuid_like(user_id):
            continue
        if not _user_exists(user_id):
            continue
        if _task_exists(ann_id):
                                _ensure_task_route_points(ann_id,   ensure_task_payload(raw_data,   title=title,
announcement_status=status, deleted_at=deleted_at))
            continue

        has_accepted_offer = bool(
            fetch_one(
                """
                SELECT 1
                FROM announcement_offers
                WHERE announcement_id = %s
                  AND status = 'accepted'
                  AND deleted_at IS NULL
                LIMIT 1
                """,
                (ann_id,),
            )
        )
        task_status, moderation_status = announcement_status_to_task_fields(
            status,
            deleted=deleted_at is not None,
            has_accepted_offer=has_accepted_offer,
        )
        data = ensure_task_payload(raw_data, title=title, announcement_status=status, deleted_at=deleted_at)
```

```
        budget_min, budget_max = derive_budget_bounds(data)
        quick_offer_price = derive_quick_offer_price(data)
        reward_amount = derive_reward_amount(data)
        address_text = primary_source_address(data)
        point = None
        if location_lat is not None and location_lon is not None:
            point = (float(location_lat), float(location_lon))
        else:
            point = primary_map_point(data)

        category_id = _category_id_for_slug(builder_category_slug(category))
        if not category_id:
            continue

         published_at = created_at if moderation_status == "published" and task_status in TASK_PUBLIC_STATUSES
| {"agreed", "in_progress", "completed", "cancelled"} else None
          closed_at = created_at if task_status in {"closed", "completed", "cancelled"} or deleted_at is not
None else None
        description = (
            str(data.get("generated_description") or "")
            or str(data.get("notes") or "")
            or title
        )

        execute(
            """
            INSERT INTO tasks (
                id,
                customer_id,
                title,
                description,
                category_id,
                reward_amount,
                currency,
                price_type,
                deadline_at,
                location_point,
                address_text,
                customer_comment,
                performer_preferences,
                status,
                moderation_status,
                views_count,
                favorites_count,
                responses_count,
                accepted_offer_id,
                extra,
                created_at,
                updated_at,
                published_at,
                closed_at,
                deleted_at,
                budget_min,
                budget_max,
                quick_offer_price,
                reoffer_policy
            )
            VALUES (
                %s,
                %s,
                %s,
                %s,
                %s,
                %s,
                'RUB',
                %s,
                NULL,
                CASE
```

```
                    WHEN %s::double precision IS NULL OR %s::double precision IS NULL THEN NULL
                    ELSE ST_SetSRID(
                        ST_MakePoint(%s::double precision, %s::double precision),
                        4326
                    )::geography
                END,
                %s,
                %s,
                NULL,
                %s,
                %s,
                0,
                0,
                COALESCE((%s)::integer, 0),
                NULL,
                %s::jsonb,
                %s,
                %s,
                %s,
                %s,
                %s,
                %s,
                %s,
                %s,
                %s

            )
            ON CONFLICT DO NOTHING
            """,
            (
                ann_id,
                user_id,
                title,
                description,
                category_id,
                reward_amount,
                "negotiable" if budget_min is not None or budget_max is not None else ("free" if reward_amount
== 0 else "fixed"),
                point[1] if point else None,
                point[0] if point else None,
                point[1] if point else None,
                point[0] if point else None,
                address_text,
                data.get("notes"),
                task_status,
                moderation_status,
                int(data.get("offers_count") or 0),
                json.dumps(data, ensure_ascii=False),
                created_at,
                updated_at,
                published_at,
                closed_at,
                deleted_at,
                budget_min,
                budget_max,
                quick_offer_price,
                str(data.get("offer_policy", {}).get("reoffer_policy") or "blocked_after_reject"),
            ),
        )
        _ensure_task_route_points(ann_id, data)


def backfill_legacy_task_offers() -> None:
    rows = fetch_all(
        """
        SELECT
            ao.id::text,
            ao.announcement_id,
```

```
            ao.performer_id,
            ao.message,
            ao.proposed_price,
            ao.status,
            ao.created_at,
            ao.chat_thread_id
        FROM announcement_offers ao
        WHERE ao.deleted_at IS NULL
        ORDER BY ao.created_at ASC
        """
    )

    for row in rows:
        offer_id = str(row[0])
        task_id = str(row[1] or "")
        performer_id = str(row[2] or "")
        message = row[3]
        proposed_price = row[4]
        legacy_status = str(row[5] or "pending")
        created_at = row[6]
        chat_thread_id = str(row[7]) if row[7] else None

        if not (is_uuid_like(offer_id) and is_uuid_like(task_id) and is_uuid_like(performer_id)):
            continue
        if not _user_exists(performer_id):
            continue
        if not _task_exists(task_id):
            continue
        if fetch_one("SELECT 1 FROM task_offers WHERE id::text = %s", (offer_id,)):
            continue

        task_row = fetch_one("SELECT extra FROM tasks WHERE id::text = %s", (task_id,))
          task_data = ensure_task_payload(task_row[0] if task_row and isinstance(task_row[0], dict) else {},
title="", announcement_status="active")
        quick_offer_price = derive_quick_offer_price(task_data)

        status = legacy_offer_status_to_canonical(legacy_status)
        pricing_mode = "counter_price" if proposed_price is not None else "quick_min_price"
        minimum_price_accepted = proposed_price is None
         agreed_price = proposed_price if status == "accepted_by_customer" and proposed_price is not None else
(quick_offer_price if status == "accepted_by_customer" else None)
        can_reoffer = status == "sent"

        execute(
            """
            INSERT INTO task_offers (
                id,
                task_id,
                performer_id,
                message,
                proposed_price,
                currency,
                status,
                created_at,
                updated_at,
                cancelled_at,
                pricing_mode,
                agreed_price,
                minimum_price_accepted,
                can_reoffer,
                reoffer_block_reason,
                chat_thread_id,
                accepted_at,
                rejected_at,
                withdrawn_at
            )
            VALUES (
                %s, %s, %s, %s, %s, 'RUB', %s, %s, %s, NULL,
```

```
            %s, %s, %s, %s, %s, %s,
            %s, %s, %s
        )
        ON CONFLICT DO NOTHING
        """,
        (
            offer_id,
            task_id,
            performer_id,
            message,
            proposed_price,
            status,
            created_at,
            created_at,
            pricing_mode,
            agreed_price,
            minimum_price_accepted,
            can_reoffer,
            None if can_reoffer else "legacy_status_terminal",
            chat_thread_id if chat_thread_id and is_uuid_like(chat_thread_id) else None,
            created_at if status == "accepted_by_customer" else None,
            created_at if status == "rejected_by_customer" else None,
            created_at if status == "withdrawn_by_sender" else None,
        ),
    )

task_rows = fetch_all("SELECT id::text FROM tasks")
for task_row in task_rows:
    task_id = str(task_row[0])
    accepted = fetch_one(
        """
        SELECT id::text
        FROM task_offers
        WHERE task_id::text = %s
          AND status = 'accepted_by_customer'
        ORDER BY created_at DESC
        LIMIT 1
        """,
        (task_id,),
    )
    responses = fetch_one(
        """
        SELECT COUNT(*)
        FROM task_offers
        WHERE task_id::text = %s
          AND status IN ('sent', 'accepted_by_customer')
        """,
        (task_id,),
    )
    execute(
        """
        UPDATE tasks
        SET accepted_offer_id = %s,
            responses_count = %s,
            status = CASE
                WHEN %s::uuid IS NOT NULL AND status IN ('published', 'in_responses') THEN 'agreed'
                WHEN %s::integer > 0 AND status = 'published' THEN 'in_responses'
                ELSE status
            END,
            updated_at = now()
        WHERE id::text = %s
        """,
        (
            accepted[0] if accepted else None,
            int(responses[0] or 0) if responses else 0,
            accepted[0] if accepted else None,
            int(responses[0] or 0) if responses else 0,
            task_id,
```

```
            ),
        )


def backfill_legacy_task_assignments() -> None:
    rows = fetch_all(
        """
        SELECT
            t.id::text,
            t.customer_id::text,
            o.id::text,
            o.performer_id::text,
            o.chat_thread_id::text
        FROM tasks t
        JOIN task_offers o
          ON o.id = t.accepted_offer_id
        WHERE t.accepted_offer_id IS NOT NULL
        """
    )

    for row in rows:
        task_id = str(row[0])
        customer_id = str(row[1])
        offer_id = str(row[2])
        performer_id = str(row[3])

        chat_thread_id = str(row[4]) if row[4] else None
        chat_thread_uuid = chat_thread_id if chat_thread_id and is_uuid_like(chat_thread_id) else None

        existing = fetch_one(
            """
            SELECT id::text
            FROM task_assignments
            WHERE offer_id::text = %s
            LIMIT 1
            """,
            (offer_id,),
        )
        if not existing and chat_thread_uuid:
            existing = fetch_one(
                """
                SELECT id::text
                FROM task_assignments
                WHERE chat_thread_id = %s::uuid
                LIMIT 1
                """,
                (chat_thread_uuid,),
            )
        if not existing:
            existing = fetch_one(
                """
                SELECT id::text
                FROM task_assignments
                WHERE task_id::text = %s
                  AND assignment_status IN ('assigned', 'in_progress')
                ORDER BY created_at DESC
                LIMIT 1
                """,
                (task_id,),
            )

        if existing:
            assignment_id = str(existing[0])
            execute(
                """
                UPDATE task_assignments
                SET task_id = %s,
                    offer_id = %s,
```

```
            customer_id = %s,
            performer_id = %s,
            assignment_status = 'assigned',
            execution_stage = 'accepted',
            route_visibility = 'performer_only',
            chat_thread_id = COALESCE(%s::uuid, chat_thread_id),
            updated_at = now()
        WHERE id::text = %s
        """,
        (
            task_id,
            offer_id,
            customer_id,
            performer_id,
            chat_thread_uuid,
            assignment_id,
        ),
    )
else:
    assignment_id = str(uuid.uuid4())
    execute(
        """
        INSERT INTO task_assignments (
            id, task_id, offer_id, customer_id, performer_id,
            assignment_status, execution_stage, route_visibility,
            chat_thread_id, created_at, updated_at
        )
        VALUES (
            %s, %s, %s, %s, %s,
            'assigned', 'accepted', 'performer_only',
            %s, now(), now()
        )
        """,
        (
            assignment_id,
            task_id,
            offer_id,
            customer_id,
            performer_id,
            chat_thread_uuid,
        ),
    )

execute(
    """
    INSERT INTO task_assignment_events (
        id, assignment_id, task_id, event_type, from_value, to_value, changed_by, payload
    )
    SELECT %s, %s, %s, 'assignment_status', NULL, 'assigned', %s, '{}'::jsonb
    WHERE NOT EXISTS (
        SELECT 1
        FROM task_assignment_events
        WHERE assignment_id = %s::uuid
          AND event_type = 'assignment_status'
          AND to_value = 'assigned'
    )
    """,
    (str(uuid.uuid4()), assignment_id, task_id, customer_id, assignment_id),
)
execute(
    """
    INSERT INTO task_assignment_events (
        id, assignment_id, task_id, event_type, from_value, to_value, changed_by, payload
    )
    SELECT %s, %s, %s, 'execution_stage', NULL, 'accepted', %s, '{}'::jsonb
    WHERE NOT EXISTS (
        SELECT 1
        FROM task_assignment_events
```

```
                WHERE assignment_id = %s::uuid
                  AND event_type = 'execution_stage'
                  AND to_value = 'accepted'
            )
            """,
            (str(uuid.uuid4()), assignment_id, task_id, customer_id, assignment_id),
        )

    assignment_rows = fetch_all(
        """
        SELECT id::text, task_id::text, chat_thread_id::text, offer_id::text
        FROM task_assignments
        WHERE chat_thread_id IS NOT NULL
        """
    )
    for row in assignment_rows:
        assignment_id = str(row[0])
        task_id = str(row[1])
        chat_thread_id = str(row[2])
        offer_id = str(row[3])
        execute(
            """
            UPDATE chat_threads
            SET task_id = %s,
                offer_id = %s,
                assignment_id = %s
            WHERE id::text = %s
            """,
            (task_id, offer_id, assignment_id, chat_thread_id),
        )
        execute(
            """
            UPDATE task_offers
            SET chat_thread_id = %s
            WHERE id::text = %s
            """,
            (chat_thread_id, offer_id),
        )


def backfill_task_status_events() -> None:
    task_rows = fetch_all(
        """
        SELECT id::text, customer_id::text, status::text
        FROM tasks
        """
    )
    for row in task_rows:
        task_id = str(row[0])
        changed_by = str(row[1])
        status = str(row[2])
        exists = fetch_one("SELECT 1 FROM task_status_events WHERE task_id::text = %s LIMIT 1", (task_id,))
        if exists:
            continue
        execute(
            """
            INSERT INTO task_status_events (
                id, task_id, from_status, to_status, changed_by, reason, created_at
            )
            VALUES (%s, %s, NULL, %s, %s, %s, now())
            ON CONFLICT DO NOTHING
            """,
            (str(uuid.uuid4()), task_id, status, changed_by, "legacy_backfill"),
        )


def ensure_all_tables() -> None:
    ensure_core_tables()
```

```
    ensure_auxiliary_tables()
    ensure_task_domain_tables()
    ensure_compat_columns()
    ensure_chat_thread_kind_compat()
    clear_schema_cache()
    ensure_indexes()
    backfill_legacy_tasks()
    backfill_legacy_task_offers()
    backfill_legacy_task_assignments()
    backfill_task_status_events()



// ===== File 3/28: app/chat.py =====
from __future__ import annotations

import anyio
import asyncio
import uuid
from datetime import datetime
from functools import lru_cache
from typing import Any, Dict, List, Optional

from fastapi import HTTPException, WebSocket
from fastapi.encoders import jsonable_encoder

from app.db import execute, fetch_all, fetch_one
from app.ops import create_notification
from app.schema_compat import table_has_column
from app.task_compat import normalize_optional_text


class ChatWebSocketHub:
    def __init__(self) -> None:
        self._connections: Dict[str, List[WebSocket]] = {}
        self._lock = asyncio.Lock()

    async def connect(self, thread_id: str, websocket: WebSocket) -> None:
        await websocket.accept()
        async with self._lock:
            thread_sockets = self._connections.setdefault(thread_id, [])
            thread_sockets.append(websocket)

    async def disconnect(self, thread_id: str, websocket: WebSocket) -> None:
        async with self._lock:
            thread_sockets = self._connections.get(thread_id)
            if not thread_sockets:
                return

            self._connections[thread_id] = [item for item in thread_sockets if item is not websocket]
            if not self._connections[thread_id]:
                self._connections.pop(thread_id, None)

    async def broadcast(self, thread_id: str, payload: Dict[str, Any]) -> None:
        async with self._lock:
            thread_sockets = list(self._connections.get(thread_id, []))

        if not thread_sockets:
            return

        try:
            serialized_payload = jsonable_encoder(payload)
        except Exception:
            return

        stale: List[WebSocket] = []
        for websocket in thread_sockets:
            try:
```

```
                await websocket.send_json(serialized_payload)
            except Exception:
                stale.append(websocket)

        if stale:
            async with self._lock:
                existing = self._connections.get(thread_id, [])
                self._connections[thread_id] = [item for item in existing if item not in stale]
                if not self._connections[thread_id]:
                    self._connections.pop(thread_id, None)


_chat_ws_hub = ChatWebSocketHub()
_chat_user_ws_hub = ChatWebSocketHub()
SYSTEM_CHAT_SENDER_ID = "system"


async def connect_chat_socket(thread_id: str, websocket: WebSocket) -> None:
    await _chat_ws_hub.connect(thread_id, websocket)


async def disconnect_chat_socket(thread_id: str, websocket: WebSocket) -> None:
    await _chat_ws_hub.disconnect(thread_id, websocket)


async def broadcast_chat_event(thread_id: str, payload: Dict[str, Any]) -> None:
    await _chat_ws_hub.broadcast(thread_id, payload)


async def broadcast_chat_message(thread_id: str, message: Dict[str, Any]) -> None:
    await broadcast_chat_event(
        thread_id=thread_id,
        payload={"type": "message", "payload": message},
    )


async def connect_user_chat_socket(user_id: str, websocket: WebSocket) -> None:
    await _chat_user_ws_hub.connect(user_id, websocket)


async def disconnect_user_chat_socket(user_id: str, websocket: WebSocket) -> None:
    await _chat_user_ws_hub.disconnect(user_id, websocket)


async def broadcast_user_chat_event(user_id: str, payload: Dict[str, Any]) -> None:
    await _chat_user_ws_hub.broadcast(user_id, payload)


def publish_chat_message_sync(thread_id: str, message: Dict[str, Any]) -> None:
    try:
        anyio.from_thread.run(broadcast_chat_message, thread_id, message)
    except Exception:
        # websocket-уведомление не должно валить REST-отправку
        return


def publish_thread_preview_sync(thread_id: str, user_id: str | None = None) -> None:
    try:
        if user_id:
            anyio.from_thread.run(broadcast_thread_preview_to_user, thread_id, user_id)
        else:
            anyio.from_thread.run(broadcast_thread_preview_update, thread_id)
    except Exception:
        return


def _normalize_text(text: str) -> str:
    return " ".join((text or "").strip().split())
```

```
@lru_cache(maxsize=1)
def _chat_message_sender_storage() -> tuple[str, bool]:
    row = fetch_one(
        """
        SELECT data_type, is_nullable
        FROM information_schema.columns
        WHERE table_schema = 'public'
          AND table_name = 'chat_messages'
          AND column_name = 'sender_id'
        """
    )
    if not row:
        return ("text", False)
    return (str(row[0] or "").lower(), str(row[1] or "").upper() == "YES")


def _system_sender_db_value(thread_id: str) -> str | None:
    data_type, is_nullable = _chat_message_sender_storage()
    if data_type in {"text", "character varying", "character"}:
        return SYSTEM_CHAT_SENDER_ID
    if is_nullable:
        return None

    participant = fetch_one(
        """
        SELECT user_id::text
        FROM chat_participants
        WHERE thread_id = %s
          AND left_at IS NULL
        ORDER BY user_id
        LIMIT 1
        """,
        (thread_id,),
    )
    if participant and participant[0]:
        return str(participant[0])

    raise HTTPException(status_code=500, detail="System sender could not be resolved")


def _message_row_to_dict(row) -> Dict[str, Any]:
    message_type = str(row[5]) if len(row) > 5 and row[5] is not None else "text"
    sender_value = SYSTEM_CHAT_SENDER_ID if message_type == "system" or row[2] is None else str(row[2])
    return {
        "id": str(row[0]),
        "thread_id": str(row[1]),
        "sender_id": sender_value,
        "text": row[3],
        "created_at": row[4],
        "type": message_type,
    }


def _avatar_select_sql(alias: str) -> str:
    prefix = f"{alias}." if alias else ""
    if table_has_column("user_profiles", "extra"):
        return f"{prefix}extra->>'avatar_url' AS avatar_url"
    return "NULL AS avatar_url"


@lru_cache(maxsize=1)
def _chat_threads_offer_fk_target() -> Optional[str]:
    rows = fetch_all(
        """
        SELECT ccu.table_name
        FROM information_schema.table_constraints tc
```

```
        JOIN information_schema.key_column_usage kcu
          ON tc.constraint_name = kcu.constraint_name
         AND tc.table_schema = kcu.table_schema
        JOIN information_schema.constraint_column_usage ccu
          ON ccu.constraint_name = tc.constraint_name
         AND ccu.table_schema = tc.table_schema
        WHERE tc.table_schema = 'public'
          AND tc.table_name = 'chat_threads'
          AND tc.constraint_type = 'FOREIGN KEY'

          AND kcu.column_name = 'offer_id'
        """
    )
    targets = [str(row[0]) for row in rows if row and row[0]]
    return targets[0] if targets else None


def _can_store_announcement_offer_id_in_chat_thread() -> bool:
    target = _chat_threads_offer_fk_target()
    return target in (None, "announcement_offers")


def _announcement_offer_chat_thread_id_sql() -> str:
    if table_has_column("announcement_offers", "chat_thread_id"):
        return "ao.chat_thread_id::text = ct.id::text OR "
    return ""


@lru_cache(maxsize=1)
def _offer_thread_kind_value() -> str:
    row = fetch_one(
        """
        SELECT data_type, udt_name
        FROM information_schema.columns
        WHERE table_schema = 'public'
          AND table_name = 'chat_threads'
          AND column_name = 'kind'
        """,
    )
    if not row:
        return "offer"

    data_type = str(row[0] or "").lower()
    udt_name = str(row[1] or "").lower()
    if data_type != "user-defined" or udt_name != "chat_thread_kind":
        return "offer"

    labels = {
        str(item[0]).lower()
        for item in fetch_all(
            """
            SELECT enumlabel
            FROM pg_type t
            JOIN pg_enum e
              ON t.oid = e.enumtypid
            WHERE t.typname = 'chat_thread_kind'
            ORDER BY e.enumsortorder
            """
        )
    }

    if "offer" in labels:
        return "offer"
    if "task" in labels:
        return "task"
    if "system" in labels:
        return "system"
    return "offer"
```

```
def ensure_chat_participant(thread_id: str, user_id: str, role: str) -> None:
    if table_has_column("chat_participants", "role"):
        execute(
            """
            INSERT INTO chat_participants (thread_id, user_id, role)
            VALUES (%s, %s, %s)
            ON CONFLICT (thread_id, user_id)
            DO UPDATE SET role = EXCLUDED.role, left_at = NULL
            """,
            (thread_id, user_id, role),
        )
        return

    execute(
        """
        INSERT INTO chat_participants (thread_id, user_id)
        VALUES (%s, %s)
        ON CONFLICT (thread_id, user_id)
        DO UPDATE SET left_at = NULL
        """,
        (thread_id, user_id),
    )


def assert_thread_access(thread_id: str, user_id: str) -> None:
    row = fetch_one(
        """
        SELECT 1
        FROM chat_participants
        WHERE thread_id = %s
          AND user_id = %s
          AND left_at IS NULL
        """,
        (thread_id, user_id),
    )
    if not row:
        raise HTTPException(status_code=403, detail="Thread access denied")


def get_or_create_offer_thread(
    *,
    task_id: str,
    offer_id: str,
    assignment_id: str | None,
    owner_id: str,
    performer_id: str,
) -> str:
    existing = fetch_one(
        """
        SELECT id::text
        FROM chat_threads
        WHERE (%s::uuid IS NOT NULL AND assignment_id = %s::uuid)
           OR offer_id::text = %s
           OR (task_id::text = %s AND kind = %s)
        ORDER BY created_at DESC
        LIMIT 1
        """,
        (assignment_id, assignment_id, offer_id, task_id, _offer_thread_kind_value()),
    )
    if existing:
        thread_id = str(existing[0])
        ensure_chat_participant(thread_id, owner_id, "owner")
        ensure_chat_participant(thread_id, performer_id, "performer")
        execute(
            """
            UPDATE chat_threads
```

```
            SET task_id = %s,
                offer_id = %s,
                assignment_id = %s
            WHERE id::text = %s
            """,
            (task_id, offer_id, assignment_id, thread_id),
        )
        if table_has_column("task_offers", "chat_thread_id"):
            execute("UPDATE task_offers SET chat_thread_id = %s WHERE id::text = %s", (thread_id, offer_id))
        if assignment_id:
                 execute("UPDATE task_assignments SET chat_thread_id = %s WHERE id::text = %s", (thread_id,
assignment_id))
        publish_thread_preview_sync(thread_id)
        return thread_id

    thread_id = str(uuid.uuid4())
    execute(
        """
        INSERT INTO chat_threads (id, kind, task_id, offer_id, last_message_at, assignment_id)
        VALUES (%s, %s, %s, %s, NULL, %s)
        """,
        (thread_id, _offer_thread_kind_value(), task_id, offer_id, assignment_id),
    )
    ensure_chat_participant(thread_id, owner_id, "owner")
    ensure_chat_participant(thread_id, performer_id, "performer")
    if table_has_column("task_offers", "chat_thread_id"):
        execute("UPDATE task_offers SET chat_thread_id = %s WHERE id::text = %s", (thread_id, offer_id))
    if assignment_id:
        execute(
            """
            UPDATE task_assignments
            SET chat_thread_id = %s,
                updated_at = now()
            WHERE id::text = %s
            """,
            (thread_id, assignment_id),
        )
    publish_thread_preview_sync(thread_id)
    return thread_id


def _thread_preview_rows(user_id: str, thread_id: str | None = None) -> List[Dict[str, Any]]:
    params: List[Any] = [user_id, user_id, user_id]
    extra_filter = ""
    if thread_id is not None:
        extra_filter = " AND ct.id::text = %s"
        params.append(thread_id)

    rows = fetch_all(
        f"""
        SELECT
            ct.id::text,
            ct.kind::text,
            other.user_id::text,
            COALESCE(
                NULLIF(BTRIM(up.display_name), ''),
                NULLIF(BTRIM(u.phone), ''),
                NULLIF(BTRIM(u.email), ''),
                'Собеседник'
            ) AS partner_display_name,
            {_avatar_select_sql("up")},
            COALESCE(lm.text, 'Чат открыт') AS last_message_text,
            COALESCE(ct.last_message_at, lm.created_at, ct.created_at) AS last_message_at,
            COALESCE(unread.unread_count, 0) AS unread_count,
            t.id::text AS task_id,
            t.title AS task_title
        FROM chat_threads ct
        JOIN chat_participants me
```

```
      ON me.thread_id = ct.id
     AND me.user_id = %s
     AND me.left_at IS NULL
    LEFT JOIN chat_messages read_msg
      ON read_msg.id = me.last_read_message_id
    LEFT JOIN chat_participants other
      ON other.thread_id = ct.id

     AND other.user_id <> %s
     AND other.left_at IS NULL
    LEFT JOIN users u
      ON u.id = other.user_id
    LEFT JOIN user_profiles up
      ON up.user_id = other.user_id
    LEFT JOIN task_assignments ta
      ON ta.id = ct.assignment_id
    LEFT JOIN task_offers tf
      ON tf.id = COALESCE(ct.offer_id, ta.offer_id)
    LEFT JOIN tasks t
      ON t.id = COALESCE(ct.task_id, ta.task_id, tf.task_id)
    LEFT JOIN LATERAL (
        SELECT text, created_at
        FROM chat_messages
        WHERE thread_id = ct.id
          AND deleted_at IS NULL
        ORDER BY created_at DESC
        LIMIT 1
    ) lm ON TRUE
    LEFT JOIN LATERAL (
        SELECT COUNT(*) AS unread_count
        FROM chat_messages m
        WHERE m.thread_id = ct.id
          AND m.deleted_at IS NULL
          AND (
                COALESCE(m.type, 'text') = 'system'
                OR m.sender_id::text <> %s
          )
          AND (
                me.last_read_message_id IS NULL
                OR m.created_at > COALESCE(read_msg.created_at, 'epoch'::timestamptz)
                OR (
                    m.created_at = COALESCE(read_msg.created_at, 'epoch'::timestamptz)
                    AND m.id::text > COALESCE(me.last_read_message_id::text, '')
                )
              )
    ) unread ON TRUE
    WHERE ct.kind <> 'support'
      AND ct.archived_at IS NULL
      AND (t.id IS NULL OR t.deleted_at IS NULL)
      {extra_filter}
    ORDER BY COALESCE(ct.last_message_at, lm.created_at, ct.created_at) DESC, ct.created_at DESC
    """,
    tuple(params),
)

return [
    {
        "thread_id": str(row[0]),
        "kind": str(row[1]),
        "partner_id": row[2],
        "partner_display_name": row[3],
        "partner_avatar_url": row[4],
        "last_message_text": row[5],
        "last_message_at": row[6],
        "unread_count": int(row[7] or 0),
        "announcement_id": row[8],
        "announcement_title": row[9],
    }
```

```
        for row in rows
    ]


async def broadcast_thread_preview_update(thread_id: str) -> None:
    participants = fetch_all(
        """
        SELECT user_id::text
        FROM chat_participants
        WHERE thread_id::text = %s
          AND left_at IS NULL
        """,
        (thread_id,),
    )
    for row in participants:
        user_id = str(row[0])
        preview = _thread_preview_rows(user_id=user_id, thread_id=thread_id)
        if not preview:
            continue
              await  broadcast_user_chat_event(user_id=user_id,  payload={"type":  "thread_upsert",  "payload":
preview[0]})


async def broadcast_thread_preview_to_user(thread_id: str, user_id: str) -> None:
    preview = _thread_preview_rows(user_id=user_id, thread_id=thread_id)
    if not preview:
        return
    await broadcast_user_chat_event(user_id=user_id, payload={"type": "thread_upsert", "payload": preview[0]})


def list_user_threads(user_id: str) -> List[Dict[str, Any]]:
    return _thread_preview_rows(user_id=user_id)


def list_thread_messages(
    thread_id: str,
    user_id: str,
    limit: int = 50,
    before: Optional[datetime] = None,
) -> List[Dict[str, Any]]:
    assert_thread_access(thread_id, user_id)

    lim = max(1, min(int(limit), 100))
    params: List[Any] = [thread_id]
    sql = """
        SELECT id, thread_id, sender_id, text, created_at, type
        FROM chat_messages
        WHERE thread_id = %s
          AND deleted_at IS NULL
    """
    if before is not None:
        sql += " AND created_at < %s"
        params.append(before)

    sql += " ORDER BY created_at DESC LIMIT %s"
    params.append(lim)

    rows = fetch_all(sql, tuple(params))
    ordered = list(reversed(rows))

    if ordered:
        execute(
            """
            UPDATE chat_participants
            SET last_read_message_id = %s
            WHERE thread_id = %s
              AND user_id = %s
            """,
```

```
            (ordered[-1][0], thread_id, user_id),
        )
        publish_thread_preview_sync(thread_id, user_id=user_id)

    return [_message_row_to_dict(row) for row in ordered]


def post_thread_message(thread_id: str, sender_id: str, text: str) -> Dict[str, Any]:
    assert_thread_access(thread_id, sender_id)

    clean_text = _normalize_text(text)
    if not clean_text:
        raise HTTPException(status_code=400, detail="Message text is required")

    message_id = str(uuid.uuid4())
    execute(
        """
        INSERT INTO chat_messages (id, thread_id, sender_id, type, text)
        VALUES (%s, %s, %s, 'text', %s)
        """,
        (message_id, thread_id, sender_id, clean_text),
    )
    execute(
        """
        UPDATE chat_threads
        SET last_message_at = now()
        WHERE id = %s
        """,
        (thread_id,),
    )
    execute(
        """
        UPDATE chat_participants
        SET last_read_message_id = %s
        WHERE thread_id = %s
          AND user_id = %s
        """,
        (message_id, thread_id, sender_id),
    )

    recipients = fetch_all(
        """
        SELECT user_id
        FROM chat_participants
        WHERE thread_id = %s
          AND user_id <> %s
          AND left_at IS NULL
        """,
        (thread_id, sender_id),
    )
    for row in recipients:
        create_notification(
            user_id=row[0],
            notif_type="chat",
            body=clean_text,
            payload={"thread_id": thread_id, "message_id": message_id},
        )

    row = fetch_one(
        """
        SELECT id, thread_id, sender_id, text, created_at, type
        FROM chat_messages
        WHERE id = %s
        """,
        (message_id,),
    )
    if not row:
        raise HTTPException(status_code=500, detail="Message was not saved")
```

```
    message = _message_row_to_dict(row)
    publish_thread_preview_sync(thread_id)
    return message



def post_system_thread_message(thread_id: str, text: str) -> Dict[str, Any]:
    clean_text = _normalize_text(text)
    if not clean_text:
        raise HTTPException(status_code=400, detail="Message text is required")

    message_id = str(uuid.uuid4())
    sender_db_value = _system_sender_db_value(thread_id)
    execute(
        """
        INSERT INTO chat_messages (id, thread_id, sender_id, type, text)
        VALUES (%s, %s, %s, 'system', %s)
        """,
        (message_id, thread_id, sender_db_value, clean_text),
    )
    execute(
        """
        UPDATE chat_threads
        SET last_message_at = now()
        WHERE id = %s
        """,
        (thread_id,),
    )

    recipients = fetch_all(
        """
        SELECT user_id
        FROM chat_participants
        WHERE thread_id = %s
          AND left_at IS NULL
        """,
        (thread_id,),
    )
    for row in recipients:
        create_notification(
            user_id=row[0],
            notif_type="chat_system",
            body=clean_text,
            payload={"thread_id": thread_id, "message_id": message_id, "sender_id": SYSTEM_CHAT_SENDER_ID},
        )

    row = fetch_one(
        """
        SELECT id, thread_id, sender_id, text, created_at, type
        FROM chat_messages
        WHERE id = %s
        """,
        (message_id,),
    )
    if not row:
        raise HTTPException(status_code=500, detail="System message was not saved")

    message = _message_row_to_dict(row)
    publish_chat_message_sync(thread_id, message)
    publish_thread_preview_sync(thread_id)
    return message



// ===== File 4/28: app/db.py =====
# app/db.py
# Productionized minimal DB helper (psycopg3)
```

```
import os
from dotenv import load_dotenv
import psycopg
from psycopg.rows import tuple_row

load_dotenv()

DATABASE_URL = os.getenv("DATABASE_URL")
if not DATABASE_URL:
    raise RuntimeError("DATABASE_URL is not set")

# global connection (MVP)
conn = psycopg.connect(DATABASE_URL, row_factory=tuple_row)
conn.autocommit = True


def fetch_one(query: str, params: tuple = ()):
    with conn.cursor() as cur:
        cur.execute(query, params)
        return cur.fetchone()


def fetch_all(query: str, params: tuple = ()):
    with conn.cursor() as cur:
        cur.execute(query, params)
        return cur.fetchall()


def execute(query: str, params: tuple = ()):
    with conn.cursor() as cur:
        cur.execute(query, params)
        return True



// ===== File 5/28: app/geocoding.py =====
# app/geocoding.py
from __future__ import annotations

import json
import os
import urllib.parse
import urllib.request
from typing import Optional, Tuple


_NOMINATIM_URL = os.getenv("NOMINATIM_URL", "https://nominatim.openstreetmap.org/search")
_USER_AGENT = os.getenv(
    "GEOCODER_USER_AGENT",
    # По правилам Nominatim желательно указывать понятный User-Agent
    "slayma-mvp/1.0 (contact: you@example.com)"
)


def geocode_address(address: str, timeout_seconds: float = 6.0) -> Optional[Tuple[float, float]]:
    """
    Возвращает (lat, lon) или None.
    MVP-геокодинг через Nominatim (OSM). Для продакшена лучше вынести в отдельный сервис/кэш.
    """
    q = (address or "").strip()
    if not q:
        return None

    params = urllib.parse.urlencode(
        {
            "q": q,
            "format": "json",
```

```
            "limit": "1",
        }
    )
    url = f"{_NOMINATIM_URL}?{params}"

    req = urllib.request.Request(
        url,
        headers={
            "User-Agent": _USER_AGENT,
            "Accept": "application/json",
        },
        method="GET",
    )

    try:
        with urllib.request.urlopen(req, timeout=timeout_seconds) as resp:
            raw = resp.read()
        data = json.loads(raw.decode("utf-8"))
        if not isinstance(data, list) or not data:
            return None

        first = data[0]
        lat = float(first["lat"])
        lon = float(first["lon"])
        return lat, lon
    except Exception:
        return None


// ===== File 6/28: app/main.py =====
from __future__ import annotations

import json
import os
import uuid
import itsdangerous
import anyio
from datetime import datetime, timezone
from functools import lru_cache
from pathlib import Path
from typing import Any, Dict, List, Optional, Tuple

from fastapi import Depends, FastAPI, File, HTTPException, UploadFile, WebSocket, WebSocketDisconnect
from fastapi.staticfiles import StaticFiles
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
from pydantic import BaseModel, Field
from starlette.concurrency import run_in_threadpool

from app.bootstrap import ensure_all_tables
from app.chat import (
    assert_thread_access,
    broadcast_chat_event,
    broadcast_chat_message,
    broadcast_thread_preview_to_user,
    connect_user_chat_socket,
    connect_chat_socket,
    disconnect_user_chat_socket,
    disconnect_chat_socket,
    get_or_create_offer_thread,
    list_thread_messages,
    list_user_threads,
    post_system_thread_message,
    post_thread_message,
)
from app.db import execute, fetch_all, fetch_one
from app.geocoding import geocode_address
from app.moderation_image import get_nsfw_detector
from app.moderation_text import classify_text
```

```
from app.ops import create_notification, create_report, ensure_appeal_report, report_status_select_sql
from app.routes_module.router import router as routes_router
from app.schema_compat import table_has_column
from app.schemas import (
    AcceptOfferOut,
    AnnouncementOut,
    AppealIn,
    ChatMessageIn,
    ChatMessageOut,
    ChatThreadOut,
    CreateAnnouncementIn,
    CreateOfferIn,
    DeviceRegisterIn,
    DeviceUnregisterIn,
    ExecutionStageUpdateIn,
    GeoPointOut,
    LoginIn,
    MeProfileOut,
    OKOut,
    RegisterIn,
    ReportCreateIn,
    ReportOut,
    SupportMessageIn,
    SupportMessageOut,
    SupportThreadOut,
    TokenOut,
    UpdateMyProfileIn,
    OfferOut,
    OfferOutExpanded,
    UserProfileOut,
    UserReviewListOut,
    UserReviewOut,
    UserStatsOut,
    UserOut,
)
from app.security import create_access_token, decode_token, hash_password, verify_password
from app.support import get_or_create_support_thread, list_support_messages, post_support_message
from app.task_compat import (
    announcement_status_to_task_fields,
    builder_category_slug,
    canonical_execution_status,
    canonical_offer_status_to_legacy,
    derive_budget_bounds,
    derive_quick_offer_price,
    derive_reward_amount,
    ensure_task_payload,
    is_uuid_like,
    legacy_offer_status_to_canonical,
    primary_destination_address,
    primary_map_point,
    primary_source_address,
    route_points_from_payload,
    route_visibility_for_execution,
    task_offer_row_to_legacy_dict,
    task_row_to_announcement_dict,
    task_to_announcement_status,
)

app = FastAPI(title="Slayma Backend (MVP)")
bearer = HTTPBearer(auto_error=True)
app.include_router(routes_router)

# ----------------------------
# Statuses (string enum in DB)
# ----------------------------
STATUS_PENDING = "pending_review"
STATUS_NEEDS_FIX = "needs_fix"
STATUS_REJECTED = "rejected"
```

```
STATUS_ACTIVE = "active"
STATUS_ARCHIVED = "archived"

REVIEW_ROLE_ALL = "all"
REVIEW_ROLE_CUSTOMER = "customer"
REVIEW_ROLE_PERFORMER = "performer"
VALID_REVIEW_ROLES = {REVIEW_ROLE_ALL, REVIEW_ROLE_CUSTOMER, REVIEW_ROLE_PERFORMER}


class ReviewFeedItemOut(BaseModel):
    id: str
    from_user_display_name: str
    stars: int = Field(..., ge=1, le=5)
    text: Optional[str] = None
    created_at: datetime
    target_role: Optional[str] = None


class ReviewFeedSummaryOut(BaseModel):
    average: float = 0.0
    count: int = 0


class ReviewFeedOut(BaseModel):
    items: list[ReviewFeedItemOut] = Field(default_factory=list)
    selected_role: str = REVIEW_ROLE_ALL
    summary: ReviewFeedSummaryOut = Field(default_factory=ReviewFeedSummaryOut)


class ReviewEligibilityOut(BaseModel):
    can_submit: bool = False
    already_submitted: bool = False
    announcement_id: str
    announcement_title: Optional[str] = None
    thread_id: Optional[str] = None
    counterpart_user_id: Optional[str] = None
    counterpart_display_name: Optional[str] = None
    counterpart_role: Optional[str] = None
    message: Optional[str] = None


class SubmitReviewIn(BaseModel):
    stars: int = Field(..., ge=1, le=5)
    text: Optional[str] = Field(default=None, max_length=2000)

# ----------------------------
# Upload config
# ----------------------------
UPLOADS_DIR = Path(os.getenv("UPLOADS_DIR", "uploads"))
NSFW_REVIEW = float(os.getenv("NSFW_REVIEW", "0.30"))
NSFW_HARD_BLOCK = float(os.getenv("NSFW_HARD_BLOCK", "0.85"))

app.mount("/uploads", StaticFiles(directory=str(UPLOADS_DIR), check_dir=False), name="uploads")


@app.on_event("startup")
def ensure_tables() -> None:
    ensure_all_tables()


# ----------------------------
# Auth
# ----------------------------
def _user_from_token(token: str) -> UserOut:
    if token == "DEV_TOKEN":
        return UserOut(id="dev", email="dev@localdomain.com", role="user")

    try:
```

```
        payload = decode_token(token)
    except Exception:
        raise HTTPException(status_code=401, detail="Invalid token")

    user_id = payload.get("sub")
    if not user_id:
        raise HTTPException(status_code=401, detail="Invalid token payload")

    row = fetch_one("SELECT id::text, email, role FROM users WHERE id = %s", (user_id,))
    if not row:
        raise HTTPException(status_code=401, detail="User not found")

    return UserOut(id=row[0], email=row[1], role=row[2])


def get_current_user(creds: HTTPAuthorizationCredentials = Depends(bearer)) -> UserOut:
    return _user_from_token(creds.credentials)


def _extract_ws_token(websocket: WebSocket) -> Optional[str]:
    auth_header = websocket.headers.get("authorization")
    if auth_header:
        prefix = "bearer "
        if auth_header.lower().startswith(prefix):
            token = auth_header[len(prefix) :].strip()

            if token:
                return token

    query_token = websocket.query_params.get("token")
    if query_token:
        return query_token.strip() or None

    return None


def _ws_current_user(websocket: WebSocket) -> UserOut:
    token = _extract_ws_token(websocket)
    if not token:
        raise HTTPException(status_code=401, detail="Missing token")
    return _user_from_token(token)


@app.post("/auth/register", response_model=TokenOut)
def register(data: RegisterIn) -> TokenOut:
    existing = fetch_one("SELECT 1 FROM users WHERE email=%s", (data.email,))
    if existing:
        raise HTTPException(status_code=400, detail="Email already registered")

    user_id = str(uuid.uuid4())
    pwd_hash = hash_password(data.password)

    execute(
        "INSERT INTO users (id, email, password_hash, role) VALUES (%s,%s,%s,%s)",
        (user_id, data.email, pwd_hash, "user"),
    )
    _ensure_profile_and_stats(user_id)

    token = create_access_token({"sub": user_id, "role": "user"})
    return TokenOut(access_token=token)


@app.post("/auth/login", response_model=TokenOut)
def login(data: LoginIn) -> TokenOut:
    row = fetch_one(
        "SELECT id::text, password_hash, role FROM users WHERE email=%s",
        (data.email,),
    )
```

```
    if not row:
        raise HTTPException(status_code=401, detail="Invalid credentials")

    user_id, pwd_hash, role = row
    if not verify_password(data.password, pwd_hash):
        raise HTTPException(status_code=401, detail="Invalid credentials")

    token = create_access_token({"sub": user_id, "role": role})
    return TokenOut(access_token=token)


@app.get("/me", response_model=UserOut)
def me(user: UserOut = Depends(get_current_user)) -> UserOut:
    return user


# ----------------------------
# Profile / devices
# ----------------------------
def _model_dump(model: Any) -> Dict[str, Any]:
    if hasattr(model, "model_dump"):
        return model.model_dump()
    if hasattr(model, "dict"):
        return model.dict()
    return {}


def _normalize_optional_text(value: Optional[str], collapse_spaces: bool = False) -> Optional[str]:
    if value is None:
        return None

    normalized = value.strip()
    if collapse_spaces:
        normalized = " ".join(normalized.split())

    return normalized or None


def _normalize_home_location(value: Any) -> Optional[Dict[str, float]]:
    if value is None:
        return None

    raw_value = value
    if isinstance(raw_value, str):
        try:
            raw_value = json.loads(raw_value)
        except Exception:
            return None

    if not isinstance(raw_value, dict):
        return None

    try:
        lat = float(raw_value.get("lat"))
        lon = float(raw_value.get("lon"))
    except (TypeError, ValueError):
        return None

    if not (-90 <= lat <= 90 and -180 <= lon <= 180):
        return None

    return {"lat": lat, "lon": lon}


def _point_out(value: Any) -> Optional[GeoPointOut]:
    point = _normalize_home_location(value)
    if not point:
        return None
```

```
    return GeoPointOut(**point)


def _normalize_json_object(value: Any) -> Dict[str, Any]:
    if value is None:
        return {}

    raw_value = value
    if isinstance(raw_value, str):
        try:
            raw_value = json.loads(raw_value)
        except Exception:
            return {}

    if isinstance(raw_value, dict):
        return dict(raw_value)

    return {}


def _preferred_address_from_extra(value: Any) -> Optional[str]:
    extra = _normalize_json_object(value)
    return _normalize_optional_text(extra.get("preferred_address"), collapse_spaces=True)


@lru_cache(maxsize=1)
def _profile_has_extra_column() -> bool:
    return table_has_column("user_profiles", "extra")


@lru_cache(maxsize=1)
def _profile_home_location_udt_name() -> Optional[str]:
    row = fetch_one(
        """
        SELECT udt_name
        FROM information_schema.columns
        WHERE table_schema = 'public'
          AND table_name = 'user_profiles'
          AND column_name = 'home_location'
        """,
    )
    if not row or not row[0]:
        return None
    return str(row[0]).lower()


def _profile_extra_select_sql(alias: str) -> str:
    if not _profile_has_extra_column():
        return "NULL::jsonb AS profile_extra"

    prefix = f"{alias}." if alias else ""
    return f"{prefix}extra AS profile_extra"


def _profile_home_location_select_sql(alias: str) -> str:
    prefix = f"{alias}." if alias else ""
    column_type = _profile_home_location_udt_name()

    if not column_type:
        return "NULL AS home_lat, NULL AS home_lon"

    if column_type in {"geography", "geometry"}:
        return (
            f"CASE WHEN {prefix}home_location IS NULL THEN NULL ELSE ST_Y({prefix}home_location::geometry) END
AS home_lat, "
            f"CASE WHEN {prefix}home_location IS NULL THEN NULL ELSE ST_X({prefix}home_location::geometry) END
AS home_lon"
        )
```

```
    if column_type in {"jsonb", "json"}:
        return (
               f"CASE WHEN {prefix}home_location IS NULL THEN NULL ELSE NULLIF({prefix}home_location->>'lat',
''):: double precision END AS home_lat, "
               f"CASE WHEN {prefix}home_location IS NULL THEN NULL ELSE NULLIF({prefix}home_location->>'lon',
''):: double precision END AS home_lon"
        )

    return "NULL AS home_lat, NULL AS home_lon"


def _fetch_profile_extra(user_id: str) -> Dict[str, Any]:
    if not _profile_has_extra_column():
        return {}

    row = fetch_one(
        """
        SELECT extra
        FROM user_profiles
        WHERE user_id = %s
        """,
        (user_id,),
    )
    if not row:
        return {}


    return _normalize_json_object(row[0])


def _ensure_profile_and_stats(user_id: str) -> None:
    if user_id == "dev":
        return

    execute(
        """
        INSERT INTO user_profiles (user_id, display_name, created_at, updated_at)
        SELECT
            u.id,
            COALESCE(NULLIF(BTRIM(u.phone), ''), NULLIF(BTRIM(u.email), ''), 'Пользователь'),
            now(),
            now()
        FROM users u
        WHERE u.id = %s
        ON CONFLICT (user_id) DO NOTHING
        """,
        (user_id,),
    )
    execute(
        """
        INSERT INTO user_stats (user_id, created_at, updated_at)
        VALUES (%s, now(), now())
        ON CONFLICT (user_id) DO NOTHING
        """,
        (user_id,),
    )


def _dev_me_profile() -> MeProfileOut:
    return MeProfileOut(
        user={
            "id": "dev",
            "email": "dev@localdomain.com",
            "phone": None,
            "created_at": datetime.now(timezone.utc),
        },
        profile={
```

```
            "display_name": "Dev User",
            "bio": None,
            "city": None,
            "preferred_address": None,
            "home_location": None,
        },
        stats={
            "rating_avg": 0.0,
            "rating_count": 0,
            "completed_count": 0,
            "cancelled_count": 0,
        },
    )


def _fetch_me_profile(user_id: str) -> MeProfileOut:
    if user_id == "dev":
        return _dev_me_profile()

    _ensure_profile_and_stats(user_id)

    row = fetch_one(
        f"""
        SELECT
            u.id::text,
            u.email,
            u.phone,
            u.created_at,
            up.display_name,
            up.bio,
            up.city,
            {_profile_extra_select_sql("up")},
            {_profile_home_location_select_sql("up")},
            COALESCE(us.rating_avg, 0),
            COALESCE(us.rating_count, 0),
            COALESCE(us.completed_count, 0),
            COALESCE(us.cancelled_count, 0)
        FROM users u
        LEFT JOIN user_profiles up ON up.user_id = u.id
        LEFT JOIN user_stats us ON us.user_id = u.id
        WHERE u.id = %s
        """,
        (user_id,),
    )
    if not row:
        raise HTTPException(status_code=404, detail="User not found")

    return MeProfileOut(
        user={
            "id": row[0],
            "email": row[1],
            "phone": row[2],
            "created_at": row[3],
        },
        profile={
            "display_name": _normalize_optional_text(row[4], collapse_spaces=True),
            "bio": _normalize_optional_text(row[5]),
            "city": _normalize_optional_text(row[6], collapse_spaces=True),
            "preferred_address": _preferred_address_from_extra(row[7]),
              "home_location": _point_out({"lat": row[8], "lon": row[9]}) if row[8] is not None and row[9] is
not None else None,
        },
        stats=UserStatsOut(
            rating_avg=float(row[10] or 0),
            rating_count=int(row[11] or 0),
            completed_count=int(row[12] or 0),
            cancelled_count=int(row[13] or 0),
        ),
```

```
    )


def _fetch_profile_section(user_id: str) -> UserProfileOut:
    if user_id == "dev":
        return UserProfileOut(display_name="Dev User", preferred_address=None)

    _ensure_profile_and_stats(user_id)

    row = fetch_one(
        f"""
        SELECT
            display_name,
            bio,
            city,
            {_profile_extra_select_sql("")},
            {_profile_home_location_select_sql("")}
        FROM user_profiles
        WHERE user_id = %s
        """,
        (user_id,),
    )

    if not row:
        return UserProfileOut()

    return UserProfileOut(
        display_name=_normalize_optional_text(row[0], collapse_spaces=True),
        bio=_normalize_optional_text(row[1]),
        city=_normalize_optional_text(row[2], collapse_spaces=True),
        preferred_address=_preferred_address_from_extra(row[3]),
         home_location=_point_out({"lat": row[4], "lon": row[5]}) if row[4] is not None and row[5] is not None
else None,
    )


@app.get("/users/me", response_model=MeProfileOut)
def users_me(user: UserOut = Depends(get_current_user)) -> MeProfileOut:
    return _fetch_me_profile(user.id)


@app.patch("/users/me/profile", response_model=UserProfileOut)
def update_my_profile(
    payload: UpdateMyProfileIn,
    user: UserOut = Depends(get_current_user),
) -> UserProfileOut:
    display_name = _normalize_optional_text(payload.display_name, collapse_spaces=True)
    bio = _normalize_optional_text(payload.bio)
    city = _normalize_optional_text(payload.city, collapse_spaces=True)
    preferred_address = _normalize_optional_text(payload.preferred_address, collapse_spaces=True)
    if not display_name or len(display_name) < 2:
        raise HTTPException(status_code=422, detail="Display name must contain at least 2 characters")

    if user.id == "dev":
        return UserProfileOut(
            display_name=display_name,
            bio=bio,
            city=city,
            preferred_address=preferred_address,
            home_location=payload.home_location,
        )

    _ensure_profile_and_stats(user.id)

    current_profile = _fetch_profile_section(user.id)

    resolved_home_location = payload.home_location
    if resolved_home_location is None:
```

```
        if preferred_address:
            geocoded = geocode_address(preferred_address)
            if geocoded:
                resolved_home_location = GeoPointOut(lat=geocoded[0], lon=geocoded[1])
            else:
                resolved_home_location = current_profile.home_location
        else:
            resolved_home_location = None

    set_clauses = [
        "display_name = %s",
        "bio = %s",
        "city = %s",
    ]
    params: List[Any] = [display_name, bio, city]

    if _profile_has_extra_column():
        extra_payload = _fetch_profile_extra(user.id)
        if preferred_address:
            extra_payload["preferred_address"] = preferred_address
        else:
            extra_payload.pop("preferred_address", None)

        set_clauses.append("extra = %s::jsonb")
        params.append(json.dumps(extra_payload, ensure_ascii=False) if extra_payload else None)


    home_location_type = _profile_home_location_udt_name()
    if home_location_type in {"geography", "geometry"}:
        if resolved_home_location is not None:
            geometry_expression = "ST_SetSRID(ST_MakePoint(%s::double precision, %s::double precision), 4326)"
            if home_location_type == "geography":
                geometry_expression = f"{geometry_expression}::geography"

            set_clauses.append(f"home_location = {geometry_expression}")
            params.extend([resolved_home_location.lon, resolved_home_location.lat])
        else:
            set_clauses.append("home_location = NULL")
    elif home_location_type in {"jsonb", "json"}:
        if resolved_home_location is not None:
            set_clauses.append("home_location = %s::jsonb")
            params.append(
                json.dumps(
                    {"lat": resolved_home_location.lat, "lon": resolved_home_location.lon},
                    ensure_ascii=False,
                )
            )
        else:
            set_clauses.append("home_location = NULL")

    set_clauses.append("updated_at = now()")
    params.append(user.id)

    execute(
        f"""
        UPDATE user_profiles
        SET {", ".join(set_clauses)}
        WHERE user_id = %s
        """,
        tuple(params),
    )

    return _fetch_profile_section(user.id)


def _normalize_review_role(role: Optional[str]) -> str:
    normalized = (role or "").strip().lower()
    if normalized in VALID_REVIEW_ROLES:
```

```
        return normalized
    return REVIEW_ROLE_ALL


def _review_feed_summary(user_id: str, role: str) -> ReviewFeedSummaryOut:
    params: List[Any] = [user_id]
    sql = """
        SELECT COALESCE(AVG(r.stars), 0), COUNT(*)
        FROM reviews r
        WHERE r.to_user_id = %s
    """
    if role != REVIEW_ROLE_ALL:
        sql += " AND COALESCE(r.target_role, '') = %s"
        params.append(role)

    row = fetch_one(sql, tuple(params))
    return ReviewFeedSummaryOut(
        average=round(float(row[0] or 0), 1) if row else 0.0,
        count=int(row[1] or 0) if row else 0,
    )


def _review_context_for_user(task_id: str, user_id: str) -> Dict[str, Any]:
    row = fetch_one(
        """
        SELECT
            ta.task_id::text,
            ta.assignment_status,
            ta.execution_stage,
            ta.chat_thread_id::text,
            ta.customer_id::text,
            ta.performer_id::text,
            COALESCE(NULLIF(BTRIM(cup.display_name), ''), NULLIF(BTRIM(cu.phone), ''), NULLIF(BTRIM(cu.email),
''), 'Заказчик') AS customer_name,
            COALESCE(NULLIF(BTRIM(pup.display_name), ''), NULLIF(BTRIM(pu.phone), ''), NULLIF(BTRIM(pu.email),
''), 'Исполнитель') AS performer_name,
            a.title
        FROM task_assignments ta
        LEFT JOIN announcements a ON a.id::text = ta.task_id::text
        LEFT JOIN user_profiles cup ON cup.user_id = ta.customer_id
        LEFT JOIN users cu ON cu.id = ta.customer_id
        LEFT JOIN user_profiles pup ON pup.user_id = ta.performer_id
        LEFT JOIN users pu ON pu.id = ta.performer_id
        WHERE ta.task_id::text = %s
        ORDER BY
            CASE ta.assignment_status
                WHEN 'completed' THEN 0
                WHEN 'in_progress' THEN 1
                WHEN 'assigned' THEN 2
                ELSE 3
            END,
            ta.updated_at DESC,
            ta.created_at DESC
        LIMIT 1
        """,
        (task_id,),
    )
    if not row:
        raise HTTPException(status_code=404, detail="Контекст отзыва для задания не найден")

    assignment_status = str(row[1] or "")
    execution_stage = str(row[2] or "")
    thread_id = str(row[3]) if row[3] else None
    customer_id = str(row[4] or "")
    performer_id = str(row[5] or "")
    customer_name = str(row[6] or "Заказчик")
    performer_name = str(row[7] or "Исполнитель")
    announcement_title = _normalize_optional_text(row[8], collapse_spaces=True)
```

```
    if user_id == customer_id:
        counterpart_user_id = performer_id
        counterpart_display_name = performer_name
        counterpart_role = REVIEW_ROLE_PERFORMER
        author_role = REVIEW_ROLE_CUSTOMER
    elif user_id == performer_id:
        counterpart_user_id = customer_id
        counterpart_display_name = customer_name
        counterpart_role = REVIEW_ROLE_CUSTOMER
        author_role = REVIEW_ROLE_PERFORMER
    else:
        raise HTTPException(status_code=403, detail="Недостаточно прав для работы с отзывом")

    already_submitted = bool(
        fetch_one(
            """
            SELECT 1
            FROM reviews
            WHERE task_id = %s
              AND from_user_id = %s
            LIMIT 1
            """,
            (task_id, user_id),
        )
    )
    is_completed = assignment_status == "completed" or execution_stage == "completed"
    can_submit = is_completed and not already_submitted and bool(counterpart_user_id)

    message = None
    if already_submitted:
        message = "Ваш отзыв уже сохранён."
    elif not is_completed:
        message = "Оставить отзыв можно после завершения задания."

    return {
        "announcement_id": task_id,
        "announcement_title": announcement_title,
        "thread_id": thread_id,
        "counterpart_user_id": counterpart_user_id,
        "counterpart_display_name": counterpart_display_name,
        "counterpart_role": counterpart_role,
        "author_role": author_role,
        "already_submitted": already_submitted,
        "can_submit": can_submit,
        "message": message,
    }


@app.get("/users/me/reviews", response_model=ReviewFeedOut)
def my_reviews(
    limit: int = 2,
    offset: int = 0,
    role: str = REVIEW_ROLE_ALL,
    user: UserOut = Depends(get_current_user),
) -> ReviewFeedOut:
    selected_role = _normalize_review_role(role)
    if user.id == "dev":
        return ReviewFeedOut(items=[], selected_role=selected_role, summary=ReviewFeedSummaryOut())

    lim = max(1, min(int(limit), 100))
    off = max(0, int(offset))
    summary = _review_feed_summary(user.id, selected_role)

    params: List[Any] = [user.id]
    filter_sql = ""
    if selected_role != REVIEW_ROLE_ALL:
        filter_sql = " AND COALESCE(r.target_role, '') = %s"
```

```
        params.append(selected_role)
    params.extend([lim, off])

    rows = fetch_all(
        f"""
        SELECT
              COALESCE(r.id::text, r.from_user_id::text || '|' || COALESCE(r.created_at::text, '') || '|' ||
COALESCE(r.text, '')),
            COALESCE(NULLIF(up.display_name, ''), u.email, 'Пользователь') AS from_user_display_name,
            r.stars,
            r.text,
            r.created_at,
            r.target_role
        FROM reviews r
        LEFT JOIN user_profiles up ON up.user_id = r.from_user_id
        LEFT JOIN users u ON u.id = r.from_user_id
        WHERE r.to_user_id = %s
        {filter_sql}
        ORDER BY r.created_at DESC
        LIMIT %s OFFSET %s
        """,
        tuple(params),
    )

    return ReviewFeedOut(

        items=[
            ReviewFeedItemOut(
                id=str(row[0]),
                from_user_display_name=row[1],
                stars=int(row[2]),
                text=row[3],
                created_at=row[4],
                target_role=row[5],
            )
            for row in rows
        ],
        selected_role=selected_role,
        summary=summary,
    )


@app.get("/announcements/{ann_id}/review-context", response_model=ReviewEligibilityOut)
def announcement_review_context(
    ann_id: str,
    user: UserOut = Depends(get_current_user),
) -> ReviewEligibilityOut:
    if user.id == "dev":
        return ReviewEligibilityOut(announcement_id=ann_id, message="В dev-режиме отзывы отключены.")

    return ReviewEligibilityOut(**_review_context_for_user(ann_id, user.id))


@app.post("/announcements/{ann_id}/review", response_model=OKOut)
def submit_announcement_review(
    ann_id: str,
    payload: SubmitReviewIn,
    user: UserOut = Depends(get_current_user),
) -> OKOut:
    if user.id == "dev":
        return OKOut(ok=True)

    context = _review_context_for_user(ann_id, user.id)
    if context["already_submitted"]:
        raise HTTPException(status_code=409, detail="Отзыв уже отправлен")
    if not context["can_submit"]:
        raise HTTPException(status_code=409, detail=context["message"] or "Оставить отзыв пока нельзя")
```

```
    clean_text = _normalize_optional_text(payload.text)
    review_id = str(uuid.uuid4())
    inserted = fetch_one(
        """
          INSERT INTO reviews (id, task_id, from_user_id, to_user_id, stars, text, author_role, target_role,
created_at)
        VALUES (%s, %s, %s, %s, %s, %s, %s, %s, now())
        ON CONFLICT (task_id, from_user_id) DO NOTHING
        RETURNING id
        """,
        (
            review_id,
            ann_id,
            user.id,
            context["counterpart_user_id"],
            int(payload.stars),
            clean_text,
            context["author_role"],
            context["counterpart_role"],
        ),
    )
    if not inserted:
        raise HTTPException(status_code=409, detail="Отзыв уже отправлен")

    counterpart_user_id = str(context["counterpart_user_id"])
    _ensure_profile_and_stats(counterpart_user_id)
    overall_summary = _review_feed_summary(counterpart_user_id, REVIEW_ROLE_ALL)
    execute(
        """
        UPDATE user_stats
        SET rating_avg = %s,
            rating_count = %s,
            updated_at = now()
        WHERE user_id = %s
        """,
        (overall_summary.average, overall_summary.count, counterpart_user_id),
    )
    create_notification(
        user_id=counterpart_user_id,
        notif_type="review_received",
        body="Вам оставили отзыв по завершённому заданию.",
        payload={"announcement_id": ann_id, "from_user_id": user.id},
    )
    return OKOut(ok=True)


@app.post("/devices/register", response_model=OKOut)
def register_device(
    payload: DeviceRegisterIn,
    user: UserOut = Depends(get_current_user),
) -> OKOut:
    if user.id == "dev":
        return OKOut(ok=True)

    device_id = _normalize_optional_text(payload.device_id)
    if not device_id:
        raise HTTPException(status_code=422, detail="device_id is required")

    existing = fetch_one(
        """
        SELECT id
        FROM user_devices
        WHERE device_id = %s
        ORDER BY created_at DESC
        LIMIT 1
        """,
        (device_id,),
    )
```

```
    if existing:
        execute(
            """
            UPDATE user_devices
            SET user_id = %s,
                platform = %s,
                push_token = %s,
                locale = %s,
                timezone = %s,
                device_name = %s,
                last_seen_at = now(),
                deleted_at = NULL
            WHERE id = %s
            """,
            (
                user.id,
                payload.platform,
                payload.push_token,
                payload.locale,
                payload.timezone,
                payload.device_name,
                existing[0],
            ),
        )
    else:
        execute(
            """
            INSERT INTO user_devices (
                    id, user_id, platform, device_id, push_token, locale, timezone, device_name, created_at,
last_seen_at, deleted_at
            )
            VALUES (%s, %s, %s, %s, %s, %s, %s, %s, now(), now(), NULL)
            """,
            (
                str(uuid.uuid4()),
                user.id,
                payload.platform,
                device_id,
                payload.push_token,
                payload.locale,
                payload.timezone,
                payload.device_name,
            ),
        )
    return OKOut(ok=True)


@app.delete("/devices/me", response_model=OKOut)
def unregister_device(
    payload: DeviceUnregisterIn,
    user: UserOut = Depends(get_current_user),
) -> OKOut:
    if user.id == "dev":
        return OKOut(ok=True)

    device_id = _normalize_optional_text(payload.device_id)
    if not device_id:
        raise HTTPException(status_code=422, detail="device_id is required")

    execute(
        """
        UPDATE user_devices
        SET deleted_at = now(),
            last_seen_at = now(),
            push_token = NULL
        WHERE user_id = %s
          AND deleted_at IS NULL
```

```
          AND (
              device_id = %s
              OR (%s IS NOT NULL AND push_token = %s)
          )
        """,
        (user.id, device_id, payload.push_token, payload.push_token),
    )
    return OKOut(ok=True)


# ----------------------------
# Helpers (announcements / moderation)
# ----------------------------
TASK_ANNOUNCEMENT_SELECT = """
    SELECT
        t.id::text,
        t.customer_id::text,
        COALESCE(c.slug, COALESCE(t.extra->>'category', t.extra->>'main_group', 'help')) AS category_slug,
        t.title,
        t.extra,
        t.created_at,
        t.status::text,
        t.moderation_status::text,
        t.deleted_at,

        t.responses_count,
        t.address_text,
              CASE  WHEN  t.location_point  IS  NULL  THEN  NULL  ELSE  ST_Y(t.location_point::geometry)  END  AS
location_lat,
              CASE  WHEN  t.location_point  IS  NULL  THEN  NULL  ELSE  ST_X(t.location_point::geometry)  END  AS
location_lon,
        ta.id::text AS assignment_id,
        ta.assignment_status,
        ta.execution_stage,
        ta.performer_id::text AS assignment_performer_id,
        ta.chat_thread_id::text AS assignment_chat_thread_id,
        ta.route_visibility
    FROM tasks t
    LEFT JOIN categories c
      ON c.id = t.category_id
    LEFT JOIN task_assignments ta
      ON ta.task_id = t.id
     AND ta.assignment_status IN ('assigned', 'in_progress')
"""


def _default_category_slug(extra: Dict[str, Any]) -> str:
    return builder_category_slug(
        _normalize_optional_text(extra.get("category"))
        or _normalize_optional_text(extra.get("main_group"))
        or "help"
    )


def _task_row_to_announcement(row) -> AnnouncementOut:
    extra = _normalize_json_object(row[4])
    payload = {
        "id": row[0],
        "customer_id": row[1],
        "category_slug": row[2] or _default_category_slug(extra),
        "title": row[3],
        "extra": extra,
        "created_at": row[5],
        "task_status": row[6],
        "moderation_status": row[7],
        "deleted_at": row[8],
        "responses_count": row[9],
        "address_text": row[10],
```

```
        "location_lat": row[11],
        "location_lon": row[12],
        "assignment_id": row[13],
        "assignment_status": row[14],
        "execution_stage": row[15],
        "assignment_performer_id": row[16],
        "assignment_chat_thread_id": row[17],
        "route_visibility": row[18],
    }
    return AnnouncementOut(**task_row_to_announcement_dict(payload))


def _row_to_report(row) -> ReportOut:
    return ReportOut(
        id=row[0],
        reporter_id=row[1],
        target_type=row[2],
        target_id=row[3],
        reason_code=row[4],
        reason_text=row[5],
        status=row[6],
        resolution=row[7],
        resolved_by=row[8],
        moderator_comment=row[9],
        created_at=row[10],
        resolved_at=row[11],
    )


def _ensure_obj(x: Any) -> Dict[str, Any]:
    return x if isinstance(x, dict) else {}


def _ensure_list(x: Any) -> List[Any]:
    return x if isinstance(x, list) else []


def _get_mod(data: Dict[str, Any]) -> Dict[str, Any]:
    return _ensure_obj(data.get("moderation"))


def _set_mod(data: Dict[str, Any], mod: Dict[str, Any]) -> None:
    data["moderation"] = mod


def _set_decision(mod: Dict[str, Any], status: str, message: str) -> None:
    mod["decision"] = {"status": status, "message": message}


def _remove_reasons_for_field(mod: Dict[str, Any], field: str) -> None:
    reasons = _ensure_list(mod.get("reasons"))
    mod["reasons"] = [r for r in reasons if not (isinstance(r, dict) and r.get("field") == field)]


def _add_reason(mod: Dict[str, Any], field: str, code: str, details: str, can_appeal: bool) -> None:
    reasons = _ensure_list(mod.get("reasons"))
    reasons.append(
        {
            "field": field,
            "code": code,
            "details": details,
            "can_appeal": bool(can_appeal),
        }
    )
    mod["reasons"] = reasons


def _set_suggestions(mod: Dict[str, Any], suggestions: List[str]) -> None:
```

```
    mod["suggestions"] = [s for s in suggestions if isinstance(s, str) and s.strip()]


def _is_temporary_text_moderation_issue(reason: str) -> bool:
    normalized = (reason or "").strip().lower()
    if not normalized:
        return False

    technical_markers = (
        "ollama error",
        "timed out",
        "connection refused",
        "connection reset",
        "remote end closed connection",
        "non-json",
        "не-json",
    )
    return any(marker in normalized for marker in technical_markers)


def _status_priority(s: str) -> int:
    return {
        STATUS_ACTIVE: 0,
        STATUS_PENDING: 1,
        STATUS_NEEDS_FIX: 2,
        STATUS_REJECTED: 3,
        STATUS_ARCHIVED: 4,
        "draft": 1,
    }.get(s or "", 1)


def _keep_stricter(current: str, candidate: str) -> str:
    return current if _status_priority(current) >= _status_priority(candidate) else candidate


def _has_media_attachments(data: Dict[str, Any]) -> bool:
    for key in ("media", "images", "photos", "media_local_identifiers"):
        raw = data.get(key)

        if isinstance(raw, list):
            for item in raw:
                if isinstance(item, str) and item.strip():
                    return True
                if isinstance(item, dict) and item:
                    return True
            continue

        if isinstance(raw, str) and raw.strip():
            return True

        if isinstance(raw, dict) and raw:
            return True

    return False


def _normalize_address(value: Any) -> Optional[str]:
    if not isinstance(value, str):
        return None
    normalized = " ".join(value.strip().split())
    return normalized or None


def _normalize_message(value: Any) -> Optional[str]:
    if not isinstance(value, str):
        return None
    normalized = " ".join(value.strip().split())
    return normalized or None
```

```python
def _parse_float(value: Any) -> Optional[float]:
    if isinstance(value, (int, float)):
        return float(value)
    if isinstance(value, str):
        raw = value.strip().replace(",", ".")
        if not raw:
            return None
        try:
            return float(raw)
        except ValueError:
            return None
    return None


def _extract_point(value: Any) -> Optional[Tuple[float, float]]:
    if not isinstance(value, dict):
        return None
    lat = _parse_float(value.get("lat"))
    lon = _parse_float(value.get("lon"))
    if lat is None or lon is None:
        return None

    if not (-90 <= lat <= 90 and -180 <= lon <= 180):
        return None
    return lat, lon


def _point_obj(point: Tuple[float, float]) -> Dict[str, float]:
    return {"lat": point[0], "lon": point[1]}


def _normalize_budget_fields(data: Dict[str, Any]) -> None:
    for key in ("budget", "budget_min", "budget_max"):
        if key not in data:
            continue

        parsed = _parse_float(data.get(key))
        if parsed is None:
            if data.get(key) in ("", None):
                data[key] = None
            continue

        data[key] = int(parsed)


def _extract_primary_address(category: str, data: Dict[str, Any]) -> Optional[str]:
    cat = (category or "").strip().lower()
    if cat == "delivery":
        return _normalize_address(data.get("pickup_address"))
    if cat == "help":
        return _normalize_address(data.get("address"))
    return _normalize_address(data.get("address"))


def _resolve_point(
    data: Dict[str, Any],
    preferred_key: str,
    fallback_key: Optional[str],
    address: Optional[str],
) -> Optional[Tuple[float, float]]:
    point = _extract_point(data.get(preferred_key))
    if point:
        return point

    if fallback_key:
        point = _extract_point(data.get(fallback_key))
```

```
        if point:
            return point

    if address:
        return geocode_address(address)

    return None


def _announcement_point_for_storage(category: str, data: Dict[str, Any]) -> Optional[Tuple[float, float]]:
    normalized_category = (category or "").strip().lower()
    if normalized_category == "delivery":
        return _extract_point(data.get("pickup_point")) or _extract_point(data.get("point"))
    if normalized_category == "help":
        return _extract_point(data.get("help_point")) or _extract_point(data.get("point"))
    return _extract_point(data.get("point"))


def _save_upload(ann_id: str, file: UploadFile, content: bytes) -> str:
    safe_name = (file.filename or "image").replace("/", "_").replace("\\", "_")
    folder = UPLOADS_DIR / ann_id
    folder.mkdir(parents=True, exist_ok=True)
    out = folder / f"{uuid.uuid4().hex}_{safe_name}"
    out.write_bytes(content)
    return "/" + out.as_posix().lstrip("/")


def _utc_iso_now() -> str:
    return datetime.now(timezone.utc).isoformat()


def _offer_avatar_select_sql() -> str:
    if _profile_has_extra_column():
        return "up.extra->>'avatar_url' AS avatar_url"
    return "NULL AS avatar_url"


def _category_id_for_slug(slug: str) -> str:
    row = fetch_one(
        """
        SELECT id::text
        FROM categories
        WHERE slug = %s
        LIMIT 1
        """,
        (slug,),
    )
    if row and row[0]:
        return str(row[0])
    raise HTTPException(status_code=500, detail=f"Category slug '{slug}' is not configured")


def _offer_from_row(row) -> OfferOut:
    return OfferOut(**task_offer_row_to_legacy_dict(
        {
            "id": row[0],
            "task_id": row[1],
            "performer_id": row[2],
            "message": row[3],
            "proposed_price": row[4],
            "agreed_price": row[5],
            "pricing_mode": row[6],
            "minimum_price_accepted": row[7],
            "can_reoffer": row[8],
            "status": row[9],
            "created_at": row[10],
        }
    ))
```

```
def _offer_expanded_from_row(row) -> OfferOutExpanded:
    offer = _offer_from_row(row)
    return OfferOutExpanded(
        id=offer.id,
        announcement_id=offer.announcement_id,
        performer_id=offer.performer_id,
        message=offer.message,
        proposed_price=offer.proposed_price,
        agreed_price=offer.agreed_price,
        pricing_mode=offer.pricing_mode,
        minimum_price_accepted=offer.minimum_price_accepted,
        can_reoffer=offer.can_reoffer,
        status=offer.status,
        created_at=offer.created_at,
        performer_profile={
            "user_id": row[11],
            "display_name": row[12],
            "city": _normalize_optional_text(row[13], collapse_spaces=True),
            "contact": _normalize_optional_text(row[14], collapse_spaces=True),
            "avatar_url": row[15],
        },
        performer_stats={
            "rating_avg": float(row[16] or 0),
            "rating_count": int(row[17] or 0),
            "completed_count": int(row[18] or 0),
            "cancelled_count": int(row[19] or 0),
        },
    )


def _task_exists(task_id: str) -> bool:
    return bool(fetch_one("SELECT 1 FROM tasks WHERE id::text = %s", (task_id,)))


def _announcement_exists(task_id: str) -> bool:
    return _task_exists(task_id)


def _fetch_task_row_or_404(task_id: str):
    row = fetch_one(
        f"""
        {TASK_ANNOUNCEMENT_SELECT}
        WHERE t.id::text = %s
        """,
        (task_id,),
    )
    if not row:
        raise HTTPException(status_code=404, detail="Announcement not found")
    return row


def _fetch_announcement_or_404(ann_id: str) -> AnnouncementOut:
    return _task_row_to_announcement(_fetch_task_row_or_404(ann_id))


def _count_pending_offers(task_id: str) -> int:
    row = fetch_one(
        """
        SELECT COUNT(*)
        FROM task_offers
        WHERE task_id::text = %s
          AND status IN ('sent', 'accepted_by_customer')
        """,
        (task_id,),
    )
    return int(row[0] or 0) if row else 0
```

```
def _sync_announcement_offers_count(task_id: str) -> int:
    offers_count = _count_pending_offers(task_id)
    execute(
        """
        UPDATE tasks
        SET responses_count = %s,
            status = CASE
                WHEN accepted_offer_id IS NOT NULL THEN status
                WHEN %s > 0 AND status = 'published' THEN 'in_responses'
                WHEN %s = 0 AND status = 'in_responses' THEN 'published'
                ELSE status
            END,
            updated_at = now()
        WHERE id::text = %s
        """,
        (offers_count, offers_count, offers_count, task_id),
    )
    return offers_count



def _sync_task_route_points(task_id: str, data: Dict[str, Any]) -> None:
    execute("DELETE FROM task_route_points WHERE task_id::text = %s", (task_id,))
    for point in route_points_from_payload(task_id, data):
        point_value = point.get("point")
        if not isinstance(point_value, tuple):
            continue
        execute(
            """
            INSERT INTO task_route_points (
                id, task_id, point_order, title, address_text, point, point_kind, created_at
            )
            VALUES (
                %s, %s, %s, %s, %s,
                ST_SetSRID(ST_MakePoint(%s, %s), 4326)::geography,
                %s, now()
            )
            """,
            (
                str(uuid.uuid4()),
                task_id,
                int(point["point_order"]),
                point.get("title"),
                point.get("address_text"),
                point_value[1],
                point_value[0],
                point.get("point_kind"),
            ),
        )


def _fetch_expanded_offer(ann_id: str, offer_id: str) -> Optional[OfferOutExpanded]:
    row = fetch_one(
        f"""
        SELECT
            tf.id::text,
            tf.task_id::text,
            tf.performer_id::text,
            tf.message,
            tf.proposed_price,
            tf.agreed_price,
            tf.pricing_mode,
            tf.minimum_price_accepted,
            tf.can_reoffer,
            tf.status::text,
            tf.created_at,
```

```
            tf.performer_id::text,
            COALESCE(
                NULLIF(BTRIM(up.display_name), ''),
                NULLIF(BTRIM(u.phone), ''),
                NULLIF(BTRIM(u.email), ''),
                'Пользователь'
            ) AS performer_display_name,
            up.city,
            COALESCE(
                NULLIF(BTRIM(u.phone), ''),
                NULLIF(BTRIM(u.email), '')
            ) AS performer_contact,
            {_offer_avatar_select_sql()},
            COALESCE(us.rating_avg, 0),
            COALESCE(us.rating_count, 0),
            COALESCE(us.completed_count, 0),
            COALESCE(us.cancelled_count, 0)
        FROM task_offers tf
        JOIN users u
          ON u.id::text = tf.performer_id::text
        LEFT JOIN user_profiles up
          ON up.user_id::text = tf.performer_id::text
        LEFT JOIN user_stats us
          ON us.user_id::text = tf.performer_id::text
        WHERE tf.task_id::text = %s
          AND tf.id::text = %s
        """,
        (ann_id, offer_id),
    )
    if not row:
        return None
    return _offer_expanded_from_row(row)


def _task_columns_from_payload(
    *,
    user_id: str,
    category: str,
    title: str,
    legacy_status: str,
    data: Dict[str, Any],
    deleted_at: Any = None,
    has_accepted_offer: bool = False,
) -> Dict[str, Any]:
    normalized_data = ensure_task_payload(
        data,
        title=title,
        announcement_status=legacy_status,
        deleted_at=deleted_at,
    )
    task_status, moderation_status = announcement_status_to_task_fields(
        legacy_status,
        deleted=deleted_at is not None,
        has_accepted_offer=has_accepted_offer,
    )
    budget_min, budget_max = derive_budget_bounds(normalized_data)
    quick_offer_price = derive_quick_offer_price(normalized_data)
    reward_amount = derive_reward_amount(normalized_data)
    point = _announcement_point_for_storage(category, normalized_data) or primary_map_point(normalized_data)
       address_text  =  _normalize_optional_text(normalized_data.get("address_text"),  collapse_spaces=True)  or
primary_source_address(normalized_data)
       published_at  =  datetime.now(timezone.utc)  if  moderation_status  ==  "published"  and  task_status  in
{"published", "in_responses"} else None
       closed_at  =  datetime.now(timezone.utc)  if  task_status  in  {"closed",  "completed",  "cancelled"}  or
deleted_at is not None else None
    return {
        "category_id": _category_id_for_slug(builder_category_slug(category)),
        "data": normalized_data,
```

```
        "task_status": task_status,
        "moderation_status": moderation_status,
        "reward_amount": reward_amount,
        "budget_min": budget_min,
        "budget_max": budget_max,
        "quick_offer_price": quick_offer_price,
        "point": point,
        "address_text": address_text,
        "published_at": published_at,
        "closed_at": closed_at,
                                "reoffer_policy":    _normalize_optional_text(normalized_data.get("offer_policy",
{}).get("reoffer_policy")) or "blocked_after_reject",
        "customer_comment": _normalize_optional_text(normalized_data.get("notes")),
                    "description":  _normalize_optional_text(normalized_data.get("generated_description"))  or
_normalize_optional_text(normalized_data.get("notes")) or title,
    }


def _insert_task(task_id: str, user_id: str, category: str, title: str, legacy_status: str, data: Dict[str,
Any]) -> None:
    fields = _task_columns_from_payload(
        user_id=user_id,
        category=category,
        title=title,
        legacy_status=legacy_status,
        data=data,
    )
    point = fields["point"]
    execute(
        """
        INSERT INTO tasks (
            id, customer_id, title, description, category_id, reward_amount, currency, price_type,
            deadline_at, location_point, address_text, customer_comment, performer_preferences,
            status, moderation_status, views_count, favorites_count, responses_count, accepted_offer_id,
            extra, created_at, updated_at, published_at, closed_at, deleted_at,
            budget_min, budget_max, quick_offer_price, reoffer_policy
        )
        VALUES (
            %s, %s, %s, %s, %s, %s, 'RUB',
            %s,
            NULL,
            CASE
                WHEN %s::double precision IS NULL OR %s::double precision IS NULL THEN NULL
                ELSE ST_SetSRID(
                    ST_MakePoint(%s::double precision, %s::double precision),
                    4326
                )::geography
            END,
            %s, %s, NULL,
            %s, %s,
            0, 0, 0, NULL,
            %s::jsonb, now(), now(), %s, %s, NULL,
            %s, %s, %s, %s
        )
        """,
        (
            task_id,
            user_id,
            title,
            fields["description"],
            fields["category_id"],
            fields["reward_amount"],
             "negotiable" if fields["budget_min"] is not None or fields["budget_max"] is not None else ("free"
if fields["reward_amount"] == 0 else "fixed"),
            point[1] if point else None,
            point[0] if point else None,
            point[1] if point else None,
            point[0] if point else None,
```

```
            fields["address_text"],
            fields["customer_comment"],
            fields["task_status"],
            fields["moderation_status"],
            json.dumps(fields["data"], ensure_ascii=False),
            fields["published_at"],
            fields["closed_at"],
            fields["budget_min"],
            fields["budget_max"],
            fields["quick_offer_price"],
            fields["reoffer_policy"],
        ),
    )
    _sync_task_route_points(task_id, fields["data"])
    execute(
        """
        INSERT INTO task_status_events (id, task_id, from_status, to_status, changed_by, reason, created_at)
        VALUES (%s, %s, NULL, %s, %s, %s, now())
        """,
        (str(uuid.uuid4()), task_id, fields["task_status"], user_id, "created_from_announcement_api"),
    )


def _update_task_content(

    task_id: str,
    *,
    user_id: str,
    category: str,
    title: str,
    legacy_status: str,
    data: Dict[str, Any],
    deleted_at: Any = None,
) -> None:
    current = _fetch_task_row_or_404(task_id)
    fields = _task_columns_from_payload(
        user_id=user_id,
        category=category,
        title=title,
        legacy_status=legacy_status,
        data=data,
        deleted_at=deleted_at,
        has_accepted_offer=bool(current[13]),
    )
    point = fields["point"]
    execute(
        """
        UPDATE tasks
        SET title = %s,
            description = %s,
            category_id = %s,
            reward_amount = %s,
            price_type = %s,
            location_point = CASE
                WHEN %s::double precision IS NULL OR %s::double precision IS NULL THEN NULL
                ELSE ST_SetSRID(
                    ST_MakePoint(%s::double precision, %s::double precision),
                    4326
                )::geography
            END,
            address_text = %s,
            customer_comment = %s,
            status = %s,
            moderation_status = %s,
            extra = %s::jsonb,
            published_at = COALESCE(%s, published_at),
            closed_at = %s,
            deleted_at = %s,
```

```
            budget_min = %s,
            budget_max = %s,
            quick_offer_price = %s,
            reoffer_policy = %s,
            updated_at = now()
        WHERE id::text = %s
        """,
        (
            title,
            fields["description"],
            fields["category_id"],
            fields["reward_amount"],
             "negotiable" if fields["budget_min"] is not None or fields["budget_max"] is not None else ("free"
if fields["reward_amount"] == 0 else "fixed"),
            point[1] if point else None,
            point[0] if point else None,
            point[1] if point else None,
            point[0] if point else None,
            fields["address_text"],
            fields["customer_comment"],
            fields["task_status"],
            fields["moderation_status"],
            json.dumps(fields["data"], ensure_ascii=False),
            fields["published_at"],
            fields["closed_at"],
            deleted_at,
            fields["budget_min"],
            fields["budget_max"],
            fields["quick_offer_price"],
            fields["reoffer_policy"],
            task_id,
        ),
    )
    _sync_task_route_points(task_id, fields["data"])


def _active_assignment_for_task(task_id: str):
    return fetch_one(
        """
                 SELECT  id::text,  offer_id::text,  performer_id::text,  assignment_status,  execution_stage,
chat_thread_id::text
        FROM task_assignments
        WHERE task_id::text = %s
          AND assignment_status IN ('assigned', 'in_progress')
        ORDER BY created_at DESC
        LIMIT 1
        """,
        (task_id,),
    )


def _create_or_update_assignment(task_id: str, offer_id: str, customer_id: str, performer_id: str) -> str:
    existing = _active_assignment_for_task(task_id)
    if existing and str(existing[1]) == offer_id:
        return str(existing[0])
    if existing and str(existing[1]) != offer_id:
        raise HTTPException(status_code=409, detail="По объявлению уже выбран исполнитель")

    assignment_id = str(uuid.uuid4())
    execute(
        """
        INSERT INTO task_assignments (
            id, task_id, offer_id, customer_id, performer_id,
            assignment_status, execution_stage, route_visibility,
            created_at, updated_at
        )
        VALUES (
            %s, %s, %s, %s, %s,
```

```
            'assigned', 'accepted', 'performer_only',
            now(), now()
        )
        """,
        (assignment_id, task_id, offer_id, customer_id, performer_id),
    )
    execute(
        """
        INSERT INTO task_assignment_events (
            id, assignment_id, task_id, event_type, from_value, to_value, changed_by, payload
        )
        VALUES (%s, %s, %s, 'assignment_status', NULL, 'assigned', %s, '{}'::jsonb)
        """,
        (str(uuid.uuid4()), assignment_id, task_id, customer_id),
    )
    execute(
        """
        INSERT INTO task_assignment_events (
            id, assignment_id, task_id, event_type, from_value, to_value, changed_by, payload
        )
        VALUES (%s, %s, %s, 'execution_stage', NULL, 'accepted', %s, '{}'::jsonb)
        """,
        (str(uuid.uuid4()), assignment_id, task_id, customer_id),
    )
    execute(
        """
        UPDATE tasks
        SET accepted_offer_id = %s,
            status = 'agreed',
            updated_at = now()
        WHERE id::text = %s
        """,
        (offer_id, task_id),
    )
    execute(
        """
        INSERT INTO task_status_events (id, task_id, from_status, to_status, changed_by, reason, created_at)
        VALUES (%s, %s, 'in_responses', 'agreed', %s, %s, now())
        """,
        (str(uuid.uuid4()), task_id, customer_id, "offer_accepted"),
    )
    return assignment_id


# ----------------------------
@app.post("/announcements", response_model=AnnouncementOut, status_code=201)
def create_announcement(
    payload: CreateAnnouncementIn,
    user: UserOut = Depends(get_current_user),
) -> AnnouncementOut:
    ann_id = str(uuid.uuid4())

    data: Dict[str, Any] = dict(payload.data or {})
    category = (payload.category or "").strip().lower()
    _normalize_budget_fields(data)
    data["offers_count"] = int(_parse_float(data.get("offers_count")) or 0)

    if category == "delivery":
        pickup_address = _normalize_address(data.get("pickup_address"))
        dropoff_address = _normalize_address(data.get("dropoff_address"))
        if pickup_address:
            data["pickup_address"] = pickup_address
        if dropoff_address:
            data["dropoff_address"] = dropoff_address

        pickup_point = _resolve_point(
            data=data,
            preferred_key="pickup_point",
```

```
            fallback_key="point",
            address=pickup_address,
        )
        dropoff_point = _resolve_point(
            data=data,
            preferred_key="dropoff_point",
            fallback_key=None,
            address=dropoff_address,
        )

        if pickup_point:
            data["pickup_point"] = _point_obj(pickup_point)
            data["point"] = _point_obj(pickup_point)
        if pickup_address:
            data["address_text"] = pickup_address
        if dropoff_point:
            data["dropoff_point"] = _point_obj(dropoff_point)

    elif category == "help":
        help_address = _normalize_address(data.get("address"))
        if help_address:
            data["address"] = help_address


        help_point = _resolve_point(
            data=data,
            preferred_key="help_point",
            fallback_key="point",
            address=help_address,
        )

        if help_point:
            data["help_point"] = _point_obj(help_point)
            data["point"] = _point_obj(help_point)
        if help_address:
            data["address_text"] = help_address

    else:
        addr = _extract_primary_address(payload.category, data)
        point = _resolve_point(data=data, preferred_key="point", fallback_key=None, address=addr)
        if point:
            data["point"] = _point_obj(point)
        if addr:
            data["address_text"] = addr

    mod = _get_mod(data)

    title = (payload.title or "").strip()
    notes = data.get("notes")

    parts = [title]
    if isinstance(notes, str) and notes.strip():
        parts.append(notes.strip())

    text_for_check = "\n\n".join([p for p in parts if p])

    text_mod = classify_text(text_for_check) if text_for_check else {"label": "LEGAL", "reason": ""}
    label = str(text_mod.get("label", "UNKNOWN")).upper()
    reason = str(text_mod.get("reason", "")).strip()

    _remove_reasons_for_field(mod, "title")
    _remove_reasons_for_field(mod, "notes")

    status = STATUS_PENDING
    suggestions: List[str] = []
    has_media = _has_media_attachments(data)
                        text_moderation_temporarily_unavailable      =      label      ==      "UNKNOWN"      and
_is_temporary_text_moderation_issue(reason)
```

```
    mod["text"] = {
        "label": label,
        "reason": reason,
        "can_appeal": True if label in ("ILLEGAL", "UNKNOWN") else False,
    }

    if label == "ILLEGAL":
        status = STATUS_REJECTED
        _set_decision(mod, status, "Отказано: текст похож на запрещённый контент. Измените формулировку.")
        _add_reason(mod, "title", "TEXT_ILLEGAL", reason or "Запрещённый текст", True)
        if isinstance(notes, str) and notes.strip():
            _add_reason(mod, "notes", "TEXT_ILLEGAL", reason or "Запрещённый текст", True)
        suggestions = [
            "Переформулируйте название и описание.",
            "Уберите неоднозначные или запрещённые формулировки.",
        ]
    elif label != "LEGAL":
        if text_moderation_temporarily_unavailable:
            if has_media:
                status = STATUS_PENDING
                  _set_decision(mod, status, "На проверке: сначала проверим фото, затем объявление появится на
карте.")
            else:
                status = STATUS_ACTIVE
                _set_decision(mod, status, "Одобрено: объявление без фото активно и отображается на карте.")
        else:
            status = STATUS_NEEDS_FIX
            _set_decision(mod, status, "Нужно исправить: текст выглядит спорным. Уточните формулировку.")
            _add_reason(mod, "title", "TEXT_UNKNOWN", reason or "Спорный текст", True)
            if isinstance(notes, str) and notes.strip():
                _add_reason(mod, "notes", "TEXT_UNKNOWN", reason or "Спорный текст", True)
            suggestions = [
                "Сделайте описание более нейтральным и конкретным.",
                "Уберите фразы, которые можно трактовать неоднозначно.",
            ]
    else:
        if has_media:
            status = STATUS_PENDING
               _set_decision(mod, status, "На проверке: сначала проверим фото, затем объявление появится на
карте.")
        else:
            status = STATUS_ACTIVE
            _set_decision(mod, status, "Одобрено: объявление без фото активно и отображается на карте.")

    mod["image"] = {
        "max_nsfw": None,
        "items": [],
        "can_appeal": None,
        "has_media": has_media,
        "review_thr": NSFW_REVIEW,
        "hard_block_thr": NSFW_HARD_BLOCK,
    }

    if suggestions:
        _set_suggestions(mod, suggestions)

    _set_mod(data, mod)

    _insert_task(
        task_id=ann_id,
        user_id=user.id,
        category=payload.category,
        title=title,
        legacy_status=status,
        data=data,
    )
    return _fetch_announcement_or_404(ann_id)
```

```
# ----------------------------
# Announcements (Media)
# ----------------------------
@app.post("/announcements/{ann_id}/media", response_model=AnnouncementOut)
def upload_announcement_media(
    ann_id: str,
    files: List[UploadFile] = File(...),
    user: UserOut = Depends(get_current_user),
) -> AnnouncementOut:
    ann = _fetch_announcement_or_404(ann_id)
    if ann.user_id != user.id:
        raise HTTPException(status_code=403, detail="Not your announcement")

    data_obj: Dict[str, Any] = dict(ann.data or {})
    mod = _get_mod(data_obj)

    det = get_nsfw_detector()

    items: List[Dict[str, Any]] = []
    max_nsfw = 0.0

    for f in files:
        content = f.file.read()
        if not content:
            continue

        r = det.predict_bytes(content)
        max_nsfw = max(max_nsfw, float(r.nsfw))

        path = _save_upload(ann_id, f, content)
        items.append(
            {
                "filename": f.filename,
                "path": path,
                "nsfw": float(r.nsfw),
                "sfw": float(r.sfw),
                "top_label": r.top_label,
                "top_prob": float(r.top_prob),
                "infer_s": float(r.infer_seconds),
            }
        )

    data_obj["media"] = items
    _remove_reasons_for_field(mod, "media")

    current_status = ann.status or STATUS_PENDING
    new_status = current_status

    if max_nsfw >= NSFW_HARD_BLOCK:
        new_status = _keep_stricter(current_status, STATUS_REJECTED)
        _add_reason(mod, "media", "NSFW_HARD", f"NSFW {max_nsfw:.2f} ≥ {NSFW_HARD_BLOCK:.2f}", False)
        _set_decision(mod, new_status, "Отказано: фото содержит запрещённый/неприемлемый контент.")
        mod["penalty_stub"] = {"type": "warning", "points": 1, "applied_at": None}
        mod["image"] = {
            "max_nsfw": max_nsfw,
            "items": items,
            "can_appeal": False,
            "review_thr": NSFW_REVIEW,
            "hard_block_thr": NSFW_HARD_BLOCK,
        }
    elif max_nsfw >= NSFW_REVIEW:
        new_status = _keep_stricter(current_status, STATUS_NEEDS_FIX)
        _add_reason(mod, "media", "NSFW_REVIEW", f"NSFW {max_nsfw:.2f}", True)
          _set_decision(mod, new_status, "Нужно исправить: фото выглядит спорным. Замените фото и отправьте
заново.")
        mod["image"] = {
```

```
            "max_nsfw": max_nsfw,
            "items": items,
            "can_appeal": True,
            "review_thr": NSFW_REVIEW,
            "hard_block_thr": NSFW_HARD_BLOCK,
        }
    else:
        if _status_priority(current_status) <= _status_priority(STATUS_PENDING):
            new_status = STATUS_ACTIVE
            _set_decision(mod, new_status, "Одобрено: объявление активно и отображается на карте.")
        else:
            new_status = current_status
        mod["image"] = {
            "max_nsfw": max_nsfw,
            "items": items,
            "can_appeal": None,
            "review_thr": NSFW_REVIEW,
            "hard_block_thr": NSFW_HARD_BLOCK,
        }

    _set_mod(data_obj, mod)
    _update_task_content(
        ann_id,
        user_id=user.id,

        category=ann.category,
        title=ann.title,
        legacy_status=new_status,
        data=data_obj,
    )
    return _fetch_announcement_or_404(ann_id)


# ----------------------------
# Announcements (Appeal / Archive / Delete)
# ----------------------------
@app.post("/announcements/{ann_id}/appeal", response_model=AnnouncementOut)
def appeal_announcement(
    ann_id: str,
    payload: AppealIn,
    user: UserOut = Depends(get_current_user),
) -> AnnouncementOut:
    ann = _fetch_announcement_or_404(ann_id)
    if ann.user_id != user.id:
        raise HTTPException(status_code=403, detail="Not your announcement")

    data_obj: Dict[str, Any] = dict(ann.data or {})
    mod = _get_mod(data_obj)
    appeal = _ensure_obj(mod.get("appeal"))
    appeal["requested"] = True
    appeal["reason"] = (payload.reason or "").strip() or None
    appeal["requested_at"] = _utc_iso_now()
    appeal["resolved_at"] = None
    mod["appeal"] = appeal

    _set_decision(mod, STATUS_PENDING, "Апелляция принята и отправлена на повторную проверку.")
    _set_mod(data_obj, mod)
    _update_task_content(
        ann_id,
        user_id=user.id,
        category=ann.category,
        title=ann.title,
        legacy_status=STATUS_PENDING,
        data=data_obj,
    )
    ensure_appeal_report(user.id, ann_id, payload.reason)
    return _fetch_announcement_or_404(ann_id)
```

```
@app.patch("/announcements/{ann_id}/archive", response_model=AnnouncementOut)
def archive_announcement(
    ann_id: str,
    user: UserOut = Depends(get_current_user),
) -> AnnouncementOut:
    ann = _fetch_announcement_or_404(ann_id)
    if ann.user_id != user.id:
        raise HTTPException(status_code=403, detail="Not your announcement")
    _update_task_content(
        ann_id,
        user_id=user.id,
        category=ann.category,
        title=ann.title,
        legacy_status=STATUS_ARCHIVED,
        data=dict(ann.data or {}),
    )
    return _fetch_announcement_or_404(ann_id)


@app.delete("/announcements/{ann_id}")
def delete_announcement(
    ann_id: str,
    user: UserOut = Depends(get_current_user),
) -> Dict[str, bool]:
    if not _task_exists(ann_id):
        return {"ok": True}
    ann = _fetch_announcement_or_404(ann_id)
    if ann.user_id != user.id:
        raise HTTPException(status_code=403, detail="Not your announcement")
    now = datetime.now(timezone.utc)
    execute(
        """
        UPDATE task_assignments
        SET assignment_status = 'cancelled',
            execution_stage = 'cancelled',
            route_visibility = 'hidden',
            cancelled_at = now(),
            updated_at = now()
        WHERE task_id::text = %s
          AND assignment_status IN ('assigned', 'in_progress')
        """,
        (ann_id,),
    )
    execute(
        """
        UPDATE chat_threads
        SET archived_at = now()
        WHERE task_id::text = %s
        """,
        (ann_id,),
    )
    execute(
        """
        UPDATE chat_participants
        SET left_at = now()
        WHERE thread_id IN (
            SELECT id FROM chat_threads WHERE task_id::text = %s
        )
        """,
        (ann_id,),
    )
    _update_task_content(
        ann_id,
        user_id=user.id,
        category=ann.category,
        title=ann.title,
        legacy_status="deleted",
```

```
        data=dict(ann.data or {}),
        deleted_at=now,
    )
    return {"ok": True}


# ----------------------------
# Offers
# ----------------------------
@app.post("/announcements/{ann_id}/offers", response_model=OfferOut, status_code=201)
def create_offer(
    ann_id: str,
    payload: CreateOfferIn,
    user: UserOut = Depends(get_current_user),
) -> OfferOut:
    ann = _fetch_announcement_or_404(ann_id)

    if ann.user_id == user.id:
        raise HTTPException(status_code=403, detail="Нельзя откликаться на своё объявление")

    if ann.status != STATUS_ACTIVE:
        raise HTTPException(status_code=400, detail="Отклики доступны только для активных объявлений")

    if _active_assignment_for_task(ann_id):
        raise HTTPException(status_code=409, detail="По объявлению уже выбран исполнитель")

    message = _normalize_message(payload.message)
            pricing_mode   =   (payload.pricing_mode   or   "").strip().lower()   or   ("quick_min_price"   if
payload.proposed_price is None else "counter_price")
    quick_offer_price = derive_quick_offer_price(dict(ann.data or {}))
    minimum_price_accepted = bool(payload.minimum_price_accepted) or pricing_mode == "quick_min_price"
    proposed_price = payload.proposed_price
    agreed_price = payload.agreed_price

    if pricing_mode == "quick_min_price":
        minimum_price_accepted = True
        proposed_price = None
        agreed_price = quick_offer_price
    elif agreed_price is None and proposed_price is not None:
        agreed_price = proposed_price

    existing = fetch_one(
        """
        SELECT id::text, status::text, can_reoffer
        FROM task_offers
        WHERE task_id::text = %s
          AND performer_id::text = %s
        LIMIT 1
        """,
        (ann_id, user.id),
    )

    if existing:
        offer_id = str(existing[0])
        existing_status = str(existing[1] or "")
        can_reoffer = bool(existing[2])
        if existing_status == "accepted_by_customer":
            raise HTTPException(status_code=409, detail="Ваш отклик уже принят")
        if not can_reoffer and existing_status in {"rejected_by_customer", "withdrawn_by_sender"}:
            raise HTTPException(status_code=409, detail="Повторный отклик по этому заданию запрещён")
        execute(
            """
            UPDATE task_offers
            SET message = %s,
                proposed_price = %s,
                agreed_price = %s,
                pricing_mode = %s,
                minimum_price_accepted = %s,
```

```
                can_reoffer = TRUE,
                reoffer_block_reason = NULL,
                status = 'sent',
                rejected_at = NULL,
                withdrawn_at = NULL,
                updated_at = now()
            WHERE id::text = %s
            """,
            (message, proposed_price, agreed_price, pricing_mode, minimum_price_accepted, offer_id),
        )
    else:
        offer_id = str(uuid.uuid4())
        execute(
            """
            INSERT INTO task_offers (
                id, task_id, performer_id, message, proposed_price, currency, status, created_at, updated_at,
                pricing_mode, agreed_price, minimum_price_accepted, can_reoffer, reoffer_block_reason
            )
            VALUES (%s, %s, %s, %s, %s, 'RUB', 'sent', now(), now(), %s, %s, %s, TRUE, NULL)
            """,
                        (offer_id,  ann_id,  user.id,  message,  proposed_price,  pricing_mode,  agreed_price,
minimum_price_accepted),
        )


    _sync_announcement_offers_count(ann_id)
    create_notification(
        user_id=ann.user_id,
        notif_type="offer",
        body="Новый отклик на ваше объявление",
        payload={"announcement_id": ann_id, "offer_id": offer_id},
    )

    row = fetch_one(
        """
        SELECT
            id::text,
            task_id::text,
            performer_id::text,
            message,
            proposed_price,
            agreed_price,
            pricing_mode,
            minimum_price_accepted,
            can_reoffer,
            status::text,
            created_at
        FROM task_offers
        WHERE id::text = %s
        """,
        (offer_id,),
    )
    if not row:
        raise HTTPException(status_code=500, detail="Failed to create offer")

    return _offer_from_row(row)


@app.get("/announcements/{ann_id}/offers", response_model=List[OfferOutExpanded])
def list_announcement_offers(
    ann_id: str,
    user: UserOut = Depends(get_current_user),
) -> List[OfferOutExpanded]:
    ann = _fetch_announcement_or_404(ann_id)
    if ann.user_id != user.id:
        raise HTTPException(status_code=403, detail="Not your announcement")

    rows = fetch_all(
```

```
        f"""
        SELECT
            tf.id::text,
            tf.task_id::text,
            tf.performer_id::text,
            tf.message,
            tf.proposed_price,
            tf.agreed_price,
            tf.pricing_mode,
            tf.minimum_price_accepted,
            tf.can_reoffer,
            tf.status::text,
            tf.created_at,
            tf.performer_id::text,
            COALESCE(
                NULLIF(BTRIM(up.display_name), ''),
                NULLIF(BTRIM(u.phone), ''),
                NULLIF(BTRIM(u.email), ''),
                'Пользователь'
            ) AS performer_display_name,
            up.city,
            COALESCE(
                NULLIF(BTRIM(u.phone), ''),
                NULLIF(BTRIM(u.email), '')
            ) AS performer_contact,
            {_offer_avatar_select_sql()},
            COALESCE(us.rating_avg, 0),
            COALESCE(us.rating_count, 0),
            COALESCE(us.completed_count, 0),
            COALESCE(us.cancelled_count, 0)
        FROM task_offers tf
        JOIN users u
          ON u.id::text = tf.performer_id::text
        LEFT JOIN user_profiles up
          ON up.user_id::text = tf.performer_id::text
        LEFT JOIN user_stats us
          ON us.user_id::text = tf.performer_id::text
        WHERE tf.task_id::text = %s
          AND tf.status IN ('sent', 'accepted_by_customer', 'rejected_by_customer')
        ORDER BY CASE WHEN tf.status = 'accepted_by_customer' THEN 0 ELSE 1 END,
                 tf.created_at DESC
        """,
        (ann_id,),
    )

    return [_offer_expanded_from_row(row) for row in rows]


@app.post("/announcements/{ann_id}/offers/{offer_id}/accept", response_model=AcceptOfferOut)
def accept_announcement_offer(
    ann_id: str,
    offer_id: str,
    user: UserOut = Depends(get_current_user),
) -> AcceptOfferOut:
    ann = _fetch_announcement_or_404(ann_id)
    if ann.user_id != user.id:
        raise HTTPException(status_code=403, detail="Not your announcement")

    row = fetch_one(
        """
        SELECT performer_id::text, status::text
        FROM task_offers
        WHERE id::text = %s
          AND task_id::text = %s
        """,
        (offer_id, ann_id),
    )
    if not row:
```

```
    raise HTTPException(status_code=404, detail="Offer not found")

performer_id, status = row
if status == "rejected_by_customer":
    raise HTTPException(status_code=409, detail="Отклик уже отклонён")

did_accept_now = False
assignment_id = None

if status != "accepted_by_customer":
    if status != "sent":
        raise HTTPException(status_code=409, detail="Отклик нельзя принять в текущем статусе")

    execute(
        """
        UPDATE task_offers
        SET status = 'accepted_by_customer',
            can_reoffer = FALSE,
            accepted_at = now(),
            agreed_price = COALESCE(agreed_price, proposed_price, %s)
        WHERE id::text = %s
        """,
        (derive_quick_offer_price(dict(ann.data or {})), offer_id),
    )
    execute(
        """
        UPDATE task_offers
        SET status = 'rejected_by_customer',
            can_reoffer = FALSE,
            reoffer_block_reason = 'task_already_assigned',
            rejected_at = now(),
            updated_at = now()
        WHERE task_id::text = %s
          AND id::text <> %s
          AND status = 'sent'
        """,
        (ann_id, offer_id),
    )
    _sync_announcement_offers_count(ann_id)
    assignment_id = _create_or_update_assignment(ann_id, offer_id, ann.user_id, performer_id)
    did_accept_now = True
else:
    existing_assignment = _active_assignment_for_task(ann_id)
    assignment_id = str(existing_assignment[0]) if existing_assignment else None

thread_id = get_or_create_offer_thread(
    task_id=ann_id,
    offer_id=offer_id,
    assignment_id=assignment_id,
    owner_id=ann.user_id,
    performer_id=performer_id,
)

if did_accept_now:
    create_notification(
        user_id=performer_id,
        notif_type="offer_accepted",
        body="Ваш отклик принят. Открыт чат.",
        payload={"thread_id": thread_id, "announcement_id": ann_id, "offer_id": offer_id},
    )

expanded = _fetch_expanded_offer(ann_id, offer_id)
if expanded is None:
    raise HTTPException(status_code=500, detail="Failed to load accepted offer")

return AcceptOfferOut(thread_id=thread_id, offer=expanded)
```

```
@app.post("/announcements/{ann_id}/offers/{offer_id}/reject", response_model=OKOut)
def reject_announcement_offer(
    ann_id: str,
    offer_id: str,
    user: UserOut = Depends(get_current_user),
) -> OKOut:
    ann = _fetch_announcement_or_404(ann_id)
    if ann.user_id != user.id:
        raise HTTPException(status_code=403, detail="Not your announcement")

    row = fetch_one(
        """
        SELECT performer_id::text, status::text
        FROM task_offers
        WHERE id::text = %s
          AND task_id::text = %s
        """,
        (offer_id, ann_id),
    )
    if not row:

        raise HTTPException(status_code=404, detail="Offer not found")

    performer_id, status = row
    if status == "accepted_by_customer":
        raise HTTPException(status_code=409, detail="Нельзя отклонить уже принятый отклик")

    if status != "rejected_by_customer":
        execute(
            """
            UPDATE task_offers
            SET status = 'rejected_by_customer',
                can_reoffer = FALSE,
                reoffer_block_reason = 'blocked_after_reject',
                rejected_at = now(),
                updated_at = now()
            WHERE id::text = %s
            """,
            (offer_id,),
        )
        _sync_announcement_offers_count(ann_id)
        create_notification(
            user_id=performer_id,
            notif_type="offer_rejected",
            body="Ваш отклик отклонён.",
            payload={"announcement_id": ann_id, "offer_id": offer_id},
        )

    return OKOut(ok=True)


def _canonical_execution_stage(stage: str) -> str:
    normalized = (stage or "").strip().lower()
    mapping = {
        "accepted": "accepted",
        "heading": "en_route",
        "en_route": "en_route",
        "onsite": "on_site",
        "on_site": "on_site",
        "doing": "in_progress",
        "in_progress": "in_progress",
        "finishing": "handoff",
        "handoff": "handoff",
        "completed": "completed",
        "cancelled": "cancelled",
        "canceled": "cancelled",
    }
    return mapping.get(normalized, normalized)
```

```
@app.post("/announcements/{ann_id}/execution-stage", response_model=AnnouncementOut)
def update_announcement_execution_stage(
    ann_id: str,
    payload: ExecutionStageUpdateIn,
    user: UserOut = Depends(get_current_user),
) -> AnnouncementOut:
    ann = _fetch_announcement_or_404(ann_id)
    assignment = _active_assignment_for_task(ann_id)
    if not assignment:
        raise HTTPException(status_code=409, detail="Для задания пока нет активного исполнителя")

       assignment_id,  accepted_offer_id,  performer_id,  assignment_status,  current_stage,  chat_thread_id  =
assignment
    if user.id != performer_id and user.id != ann.user_id:
        raise HTTPException(status_code=403, detail="Недостаточно прав для смены этапа")

    new_stage = _canonical_execution_stage(payload.stage)
    allowed_stages = {"accepted", "en_route", "on_site", "in_progress", "handoff", "completed", "cancelled"}
    if new_stage not in allowed_stages:
        raise HTTPException(status_code=422, detail="Неизвестный execution-stage")

    stage_order = {
        "accepted": 0,
        "en_route": 1,
        "on_site": 2,
        "in_progress": 3,
        "handoff": 4,
        "completed": 5,
        "cancelled": 99,
    }
    current_stage = canonical_execution_status(
        execution_stage=current_stage,
        assignment_status=assignment_status,
        current_value="accepted",
    )
    if new_stage != "cancelled" and stage_order[new_stage] < stage_order[current_stage]:
        raise HTTPException(status_code=409, detail="Нельзя откатить execution-stage назад")
    if new_stage != "cancelled" and stage_order[new_stage] > stage_order[current_stage] + 1:
        raise HTTPException(status_code=409, detail="Нужно отмечать этапы последовательно")

    next_assignment_status = "assigned"
    next_task_status = "agreed"
    if new_stage in {"en_route", "on_site", "in_progress", "handoff"}:
        next_assignment_status = "in_progress"
        next_task_status = "in_progress"
    elif new_stage == "completed":
        next_assignment_status = "completed"
        next_task_status = "completed"
    elif new_stage == "cancelled":
        next_assignment_status = "cancelled"
        next_task_status = "cancelled"

    accepted_confirmed = new_stage != "cancelled"
    data_obj: Dict[str, Any] = dict(ann.data or {})
    task_obj = _ensure_obj(data_obj.get("task"))
    task_execution = _ensure_obj(task_obj.get("execution"))
    task_assignment = _ensure_obj(task_obj.get("assignment"))
    legacy_execution = _ensure_obj(data_obj.get("execution"))

    task_execution["status"] = new_stage
    task_execution["accepted_confirmed"] = accepted_confirmed
    task_assignment["execution_status"] = new_stage
    task_assignment["accepted_confirmed"] = accepted_confirmed
    task_obj["execution"] = task_execution
    task_obj["assignment"] = task_assignment
    data_obj["task"] = task_obj
```

```
    legacy_execution["status"] = new_stage
    legacy_execution["accepted_confirmed"] = accepted_confirmed
    data_obj["execution"] = legacy_execution
    data_obj["execution_status"] = new_stage
    data_obj["execution_status_confirmed"] = accepted_confirmed

    next_route_visibility = route_visibility_for_execution(new_stage)
    execute(
        """
        UPDATE task_assignments
        SET assignment_status = %s,
            execution_stage = %s,
            route_visibility = %s,
            started_at = CASE
                WHEN started_at IS NULL AND %s = 'in_progress' THEN now()
                ELSE started_at
            END,
            completed_at = CASE WHEN %s = 'completed' THEN now() ELSE completed_at END,
            cancelled_at = CASE WHEN %s = 'cancelled' THEN now() ELSE cancelled_at END,
            updated_at = now()
        WHERE id::text = %s
        """,
        (
            next_assignment_status,
            new_stage,
            next_route_visibility,
            next_assignment_status,
            next_assignment_status,
            next_assignment_status,
            assignment_id,
        ),
    )
    execute(
        """
           INSERT INTO task_assignment_events (id, assignment_id, task_id, event_type, from_value, to_value,
changed_by, payload)
        VALUES (%s, %s, %s, 'execution_stage', %s, %s, %s, '{}'::jsonb)
        """,
        (str(uuid.uuid4()), assignment_id, ann_id, current_stage, new_stage, user.id),
    )
    if assignment_status != next_assignment_status:
        execute(
            """
             INSERT INTO task_assignment_events (id, assignment_id, task_id, event_type, from_value, to_value,
changed_by, payload)
            VALUES (%s, %s, %s, 'assignment_status', %s, %s, %s, '{}'::jsonb)
            """,
            (str(uuid.uuid4()), assignment_id, ann_id, assignment_status, next_assignment_status, user.id),
        )

    current_task = fetch_one("SELECT status::text FROM tasks WHERE id::text = %s", (ann_id,))
    previous_task_status = str(current_task[0]) if current_task and current_task[0] else "agreed"
    execute(
        """
        UPDATE tasks
        SET status = %s,
            extra = %s::jsonb,
            closed_at = CASE WHEN %s IN ('completed', 'cancelled') THEN now() ELSE NULL END,
            updated_at = now()
        WHERE id::text = %s
        """,
        (next_task_status, json.dumps(data_obj, ensure_ascii=False), next_task_status, ann_id),
    )
    if previous_task_status != next_task_status:
        execute(
            """
                  INSERT INTO task_status_events (id, task_id, from_status, to_status, changed_by, reason,
```

```
created_at)
            VALUES (%s, %s, %s, %s, %s, %s, now())
            """,
                                (str(uuid.uuid4()),  ann_id,  previous_task_status,  next_task_status,  user.id,
"execution_stage_updated"),
        )

    legacy_announcement_status = STATUS_ACTIVE
    if new_stage == "completed":
        legacy_announcement_status = "completed"
    elif new_stage == "cancelled":
        legacy_announcement_status = "cancelled"
    execute(
        """
        UPDATE announcements
        SET status = %s,
            updated_at = now()
        WHERE id::text = %s
          AND deleted_at IS NULL
        """,
        (legacy_announcement_status, ann_id),
    )


    if chat_thread_id:
        if new_stage == "completed":
            post_system_thread_message(
                chat_thread_id,
                 "Заказ выполнен. Если возникли проблемы, позже здесь можно будет открыть спор. Если всё прошло
хорошо, оставьте отзыв друг о друге."
            )
        anyio.from_thread.run(broadcast_thread_preview_to_user, chat_thread_id, ann.user_id)
        anyio.from_thread.run(broadcast_thread_preview_to_user, chat_thread_id, performer_id)
    return _fetch_announcement_or_404(ann_id)


# ----------------------------
# Chat (offer threads)
# ----------------------------
@app.get("/chats", response_model=List[ChatThreadOut])
def get_chats(user: UserOut = Depends(get_current_user)) -> List[ChatThreadOut]:
    if user.id == "dev":
        return []

    threads = list_user_threads(user.id)
    return [ChatThreadOut(**thread) for thread in threads]


@app.get("/chats/{thread_id}/messages", response_model=List[ChatMessageOut])
def get_chat_messages(
    thread_id: str,
    limit: int = 50,
    before: Optional[datetime] = None,
    user: UserOut = Depends(get_current_user),
) -> List[ChatMessageOut]:
    if user.id == "dev":
        return []

    messages = list_thread_messages(thread_id=thread_id, user_id=user.id, limit=limit, before=before)
    return [ChatMessageOut(**message) for message in messages]


@app.post("/chats/{thread_id}/messages", response_model=ChatMessageOut, status_code=201)
async def send_chat_message(
    thread_id: str,
    payload: ChatMessageIn,
    user: UserOut = Depends(get_current_user),
) -> ChatMessageOut:
```

```
    if user.id == "dev":
        raise HTTPException(status_code=400, detail="DEV chat is not available")

    await run_in_threadpool(assert_thread_access, thread_id, user.id)
    message = await run_in_threadpool(post_thread_message, thread_id, user.id, payload.text)
    await broadcast_chat_message(thread_id=thread_id, message=message)
    return ChatMessageOut(**message)


def _parse_ws_chat_text(raw_message: str) -> Optional[str]:
    raw = (raw_message or "").strip()
    if not raw:
        return None

    if not raw.startswith("{"):
        return raw

    try:
        payload = json.loads(raw)
    except Exception:
        return raw

    if not isinstance(payload, dict):
        return None

    message_type = str(payload.get("type") or "").lower()
    if message_type == "ping":
        return None

    text_value = payload.get("text")
    if isinstance(text_value, str):
        normalized = text_value.strip()
        return normalized or None

    return None


@app.websocket("/ws/chats/{thread_id}")
async def chat_websocket(thread_id: str, websocket: WebSocket) -> None:
    try:
        user = _ws_current_user(websocket)
    except HTTPException:
        await websocket.close(code=4401)
        return

    if user.id == "dev":
        await websocket.close(code=4403)
        return

    try:
        await run_in_threadpool(assert_thread_access, thread_id, user.id)
    except HTTPException:
        await websocket.close(code=4403)
        return

    await connect_chat_socket(thread_id, websocket)
    try:
        await websocket.send_json({"type": "ready", "thread_id": thread_id})
        while True:
            incoming = await websocket.receive_text()
            text = _parse_ws_chat_text(incoming)
            if text is None:
                await websocket.send_json({"type": "pong"})
                continue

            message = await run_in_threadpool(post_thread_message, thread_id, user.id, text)
            await broadcast_chat_event(
                thread_id=thread_id,
```

```
                payload={"type": "message", "payload": message},
            )
    except WebSocketDisconnect:
        pass
    except Exception:
        try:
            await websocket.send_json({"type": "error", "message": "Ошибка чата"})
        except Exception:
            pass
    finally:
        await disconnect_chat_socket(thread_id, websocket)


# ----------------------------
# Reports
# ----------------------------
@app.post("/reports", response_model=ReportOut, status_code=201)
def submit_report(
    payload: ReportCreateIn,
    user: UserOut = Depends(get_current_user),
) -> ReportOut:
    allowed_targets = {"announcement", "message", "user", "task"}
    if payload.target_type not in allowed_targets:
        raise HTTPException(status_code=400, detail="Unsupported target type")

    report_id = create_report(
        reporter_id=user.id,
        target_type=payload.target_type,
        target_id=payload.target_id,
        reason_code=payload.reason_code,
        reason_text=payload.reason_text,
    )
    row = fetch_one(
        f"""
        SELECT id, reporter_id, target_type, target_id, reason_code, reason_text, status,
               resolution, resolved_by, moderator_comment, created_at, resolved_at
        FROM reports
        WHERE id = %s
        """.replace("status", f"{report_status_select_sql('reports')} AS status", 1),
        (report_id,),
    )
    if not row:
        raise HTTPException(status_code=500, detail="Failed to create report")
    return _row_to_report(row)


# ----------------------------
# Support chat (public API)
# ----------------------------
@app.get("/support/thread", response_model=SupportThreadOut)
def get_support_thread(user: UserOut = Depends(get_current_user)) -> SupportThreadOut:
    return SupportThreadOut(thread_id=get_or_create_support_thread(user.id))


@app.get("/support/thread/{thread_id}/messages", response_model=List[SupportMessageOut])
def get_support_thread_messages(
    thread_id: str,
    limit: int = 50,
    before: Optional[datetime] = None,
    user: UserOut = Depends(get_current_user),
) -> List[SupportMessageOut]:
    messages = list_support_messages(thread_id=thread_id, user_id=user.id, limit=limit, before=before)
    return [SupportMessageOut(**message) for message in messages]


@app.post("/support/thread/{thread_id}/messages", response_model=SupportMessageOut, status_code=201)
def send_support_thread_message(
    thread_id: str,
```

```
    payload: SupportMessageIn,
    user: UserOut = Depends(get_current_user),
) -> SupportMessageOut:
    message = post_support_message(
        thread_id=thread_id,
        sender_id=user.id,
        text=payload.text,
        sender_role=user.role,
    )
    return SupportMessageOut(**message)


# ----------------------------
# Lists
# ----------------------------
@app.get("/announcements/me", response_model=List[AnnouncementOut])
def my_announcements(user: UserOut = Depends(get_current_user)) -> List[AnnouncementOut]:
    rows = fetch_all(
        f"""
        {TASK_ANNOUNCEMENT_SELECT}

        WHERE t.customer_id::text = %s
          AND t.deleted_at IS NULL
        ORDER BY t.created_at DESC
        """,
        (user.id,),
    )
    return [_task_row_to_announcement(r) for r in rows]


@app.get("/announcements/public", response_model=List[AnnouncementOut])
def public_announcements(limit: int = 200) -> List[AnnouncementOut]:
    lim = max(1, min(int(limit), 500))
    rows = fetch_all(
        f"""
        {TASK_ANNOUNCEMENT_SELECT}
        WHERE t.deleted_at IS NULL
          AND t.moderation_status = 'published'
          AND t.status IN ('published', 'in_responses')
          AND t.customer_id::text <> 'dev'
          AND NOT EXISTS (
              SELECT 1
              FROM task_assignments ta
              WHERE ta.task_id = t.id
                AND ta.assignment_status IN ('assigned', 'in_progress')
          )
        ORDER BY t.created_at DESC
        LIMIT %s
        """,
        (lim,),
    )
    return [_task_row_to_announcement(r) for r in rows]


def _user_can_fetch_announcement(ann_id: str, user_id: str, owner_id: str) -> bool:
    if owner_id == user_id:
        return True

    assignment = fetch_one(
        """
        SELECT 1
        FROM task_assignments
        WHERE task_id::text = %s
          AND performer_id::text = %s
          AND assignment_status IN ('assigned', 'in_progress', 'completed')
        LIMIT 1
        """,
        (ann_id, user_id),
```

```
    )
    if assignment:
        return True

    public_row = fetch_one(
        """
        SELECT 1
        FROM tasks t
        WHERE t.id::text = %s
          AND t.deleted_at IS NULL
          AND t.moderation_status = 'published'
          AND t.status IN ('published', 'in_responses')
          AND t.customer_id::text <> 'dev'
          AND NOT EXISTS (
              SELECT 1
              FROM task_assignments ta
              WHERE ta.task_id = t.id
                AND ta.assignment_status IN ('assigned', 'in_progress')
          )
        LIMIT 1
        """,
        (ann_id,),
    )
    return public_row is not None


@app.get("/announcements/{ann_id}", response_model=AnnouncementOut)
def get_announcement(
    ann_id: str,
    user: UserOut = Depends(get_current_user),
) -> AnnouncementOut:
    ann = _fetch_announcement_or_404(ann_id)
    if not _user_can_fetch_announcement(ann_id, user.id, ann.user_id):
        raise HTTPException(status_code=404, detail="Announcement not found")
    return ann



// ===== File 7/28: app/moderation_image.py =====
# app/moderation_image.py
from __future__ import annotations

import os
import time
from dataclasses import dataclass
from io import BytesIO
from typing import Any, Dict, Optional, Tuple

NSFW_MODEL_ID = os.getenv("NSFW_MODEL_ID", "hf_hub:Marqo/nsfw-image-detection-384")
NSFW_DEVICE = os.getenv("NSFW_DEVICE", "cpu")  # cpu / mps (если есть)

_detector = None  # singleton

@dataclass
class NsfwResult:
    nsfw: float
    sfw: float
    top_label: str
    top_prob: float
    infer_seconds: float

def _pick_device(requested: str) -> str:
    d = (requested or "cpu").lower()
    if d == "mps":
        try:
            import torch
            if torch.backends.mps.is_available() and torch.backends.mps.is_built():
                return "mps"
```

```
        except Exception:
            pass
    return "cpu"

class NsfwTimmDetector:
    def __init__(self, model_id: str, device: str = "cpu"):
        import timm
        import torch

        self.model_id = model_id
        self.device = _pick_device(device)

        t0 = time.perf_counter()
        self.model = timm.create_model(self.model_id, pretrained=True)
        self.model.eval()
        if self.device != "cpu":
            self.model.to(self.device)

        data_config = timm.data.resolve_model_data_config(self.model)
        self.transforms = timm.data.create_transform(**data_config, is_training=False)

        cfg = getattr(self.model, "pretrained_cfg", {}) or {}
        self.class_names = cfg.get("label_names", ["NSFW", "SFW"])  # часто ["NSFW","SFW"]
        self.load_seconds = time.perf_counter() - t0

        # индекс класса NSFW
        self.nsfw_index = 0
        for i, name in enumerate(self.class_names):
            if str(name).upper() == "NSFW":
                self.nsfw_index = i
                break

    def predict_bytes(self, image_bytes: bytes) -> NsfwResult:
        import torch
        from PIL import Image

        img = Image.open(BytesIO(image_bytes)).convert("RGB")
        x = self.transforms(img).unsqueeze(0)

        if self.device != "cpu":
            x = x.to(self.device)

        t0 = time.perf_counter()
        with torch.no_grad():
            out = self.model(x).softmax(dim=-1).detach().cpu()[0]
        infer_dt = time.perf_counter() - t0

        probs = [float(v) for v in out.tolist()]
        # безопасно: если внезапно 1 класс — считаем его nsfw
        nsfw = probs[self.nsfw_index] if self.nsfw_index < len(probs) else float(probs[0])
        sfw = 1.0 - nsfw if len(probs) >= 2 else float(1.0 - nsfw)

        # top
        top_i = max(range(len(probs)), key=lambda i: probs[i])
        top_label = str(self.class_names[top_i]) if top_i < len(self.class_names) else str(top_i)
        top_prob = float(probs[top_i])

        return NsfwResult(
            nsfw=float(nsfw),
            sfw=float(sfw),
            top_label=top_label,
            top_prob=top_prob,
            infer_seconds=float(infer_dt),
        )

def get_nsfw_detector() -> NsfwTimmDetector:
    global _detector
    if _detector is None:
```

```
        _detector = NsfwTimmDetector(model_id=NSFW_MODEL_ID, device=NSFW_DEVICE)
    return _detector


// ===== File 8/28: app/moderation_text.py =====
# app/moderation_text.py
from __future__ import annotations

import json
import os
import time
import urllib.request
from functools import lru_cache
from typing import Any, Dict, Optional

OLLAMA_URL = os.getenv("OLLAMA_URL", "http://localhost:11434/api/generate")
OLLAMA_MODEL = os.getenv("OLLAMA_MODEL", "shieldgemma:2b")
OLLAMA_TIMEOUT = float(os.getenv("OLLAMA_TIMEOUT", "15"))

SYSTEM = (
    "You are a legality classification model. "
    "Classify the text into LEGAL or ILLEGAL. "
    "Return ONLY JSON: {\"label\":\"LEGAL|ILLEGAL\",\"reason\":\"...\"}."
)

def _to_json_or_none(text: str) -> Optional[Dict[str, Any]]:
    text = (text or "").strip()
    if text.startswith("{") and text.endswith("}"):
        try:
            return json.loads(text)
        except Exception:
            return None
    # иногда модель может “обрамлять” JSON — вытаскиваем по скобкам
    i = text.find("{")
    j = text.rfind("}")
    if i != -1 and j != -1 and j > i:
        try:
            return json.loads(text[i : j + 1])
        except Exception:
            return None
    return None

def _normalize_text(text: str) -> str:
    # чтобы кэш работал лучше: убираем лишние пробелы
    return " ".join((text or "").strip().split())

@lru_cache(maxsize=4096)
def classify_text(text: str) -> Dict[str, Any]:
    """
    Локальная (на сервере) проверка текста через Ollama.
    Кэшируем результат, потому что “очень много текстов”.
    """
    normalized = _normalize_text(text)
    if not normalized:
        return {"label": "LEGAL", "reason": "empty"}

    prompt = f"{SYSTEM}\nText: {normalized}\n"
    payload = {
        "model": OLLAMA_MODEL,
        "prompt": prompt,
        "stream": False,
        "format": "json",
        "options": {"temperature": 0},
    }

    data = json.dumps(payload, ensure_ascii=False).encode("utf-8")
    req = urllib.request.Request(
        OLLAMA_URL,
```

```
        data=data,
        headers={"Content-Type": "application/json", "Accept": "application/json"},
        method="POST",
    )

    t0 = time.perf_counter()
    try:
        with urllib.request.urlopen(req, timeout=OLLAMA_TIMEOUT) as resp:
            raw = resp.read().decode("utf-8")
        obj = json.loads(raw)
        model_text = obj.get("response", "") or ""
        parsed = _to_json_or_none(model_text)
        if not parsed:
            return {"label": "UNKNOWN", "reason": f"Model ответила не-JSON: {model_text[:200]}"}
        # нормализуем выход
        label = str(parsed.get("label", "UNKNOWN")).upper()
        reason = str(parsed.get("reason", "")).strip()
        dt = time.perf_counter() - t0
        return {"label": label, "reason": reason, "t": dt}
    except Exception as e:
        return {"label": "UNKNOWN", "reason": f"Ollama error: {e!s}"}


// ===== File 9/28: app/ops.py =====
from __future__ import annotations

import json
import uuid
from typing import Any, Dict, Optional

from app.db import execute, fetch_one
from app.schema_compat import get_table_columns, table_has_column


def _build_insert_sql(table_name: str, values: Dict[str, Any], jsonb_columns: set[str] | None = None) ->
tuple[str, tuple[Any, ...]]:
    jsonb_columns = jsonb_columns or set()
    columns = list(values.keys())
    placeholders = []
    params: list[Any] = []
    for column in columns:
        if column in jsonb_columns:
            placeholders.append("%s::jsonb")
            params.append(json.dumps(values[column], ensure_ascii=False))
        else:
            placeholders.append("%s")
            params.append(values[column])
    sql = f"INSERT INTO {table_name} ({', '.join(columns)}) VALUES ({', '.join(placeholders)})"
    return sql, tuple(params)


def _report_open_filter_sql() -> str:
    if table_has_column("reports", "status"):
        return "status = 'open'"
    if table_has_column("reports", "resolved_at"):
        return "resolved_at IS NULL"
    if table_has_column("reports", "resolution"):
        return "resolution IS NULL"
    return "1 = 1"


def report_status_select_sql(alias: str = "reports") -> str:
    if table_has_column("reports", "status"):
        return f"{alias}.status"
    if table_has_column("reports", "resolved_at"):
        return f"CASE WHEN {alias}.resolved_at IS NULL THEN 'open' ELSE 'resolved' END"
    if table_has_column("reports", "resolution"):
        return f"CASE WHEN {alias}.resolution IS NULL THEN 'open' ELSE 'resolved' END"
```

```
    return "'open'"


def _notification_user_id_requires_uuid() -> bool:
    row = fetch_one(
        """
        SELECT udt_name
        FROM information_schema.columns
        WHERE table_schema = 'public'
          AND table_name = 'notifications'
          AND column_name = 'user_id'
        """,
    )
    if not row or not row[0]:
        return False
    return str(row[0]).lower() == "uuid"


def create_notification(
    user_id: str,
    notif_type: str,
    body: str,
    payload: Optional[Dict[str, Any]] = None,
) -> str:
    notification_id = str(uuid.uuid4())
    if _notification_user_id_requires_uuid():
        try:
            uuid.UUID(str(user_id))
        except Exception:
            return notification_id

    columns = get_table_columns("notifications")
    values: Dict[str, Any] = {"id": notification_id, "user_id": user_id, "type": notif_type, "body": body}
    if "payload" in columns:
        values["payload"] = payload or {}
    if "is_read" in columns:
        values["is_read"] = False
     sql, params = _build_insert_sql("notifications", values, jsonb_columns={"payload"} if "payload" in values
else set())
    execute(sql, params)
    return notification_id


def create_report(
    reporter_id: str,
    target_type: str,
    target_id: str,
    reason_code: str,
    reason_text: Optional[str] = None,
) -> str:
    report_id = str(uuid.uuid4())
    columns = get_table_columns("reports")
    values: Dict[str, Any] = {
        "id": report_id,
        "reporter_id": reporter_id,
        "target_type": target_type,
        "target_id": target_id,
        "reason_code": reason_code,
    }
    if "reason_text" in columns:
        values["reason_text"] = reason_text
    if "status" in columns:
        values["status"] = "open"
    sql, params = _build_insert_sql("reports", values)
    execute(sql, params)
    return report_id
```

```
def ensure_appeal_report(
    reporter_id: str,
    announcement_id: str,
    reason_text: Optional[str] = None,
) -> str:
    existing = fetch_one(
        f"""
        SELECT id
        FROM reports
        WHERE reporter_id = %s
          AND target_type = 'announcement'
          AND target_id = %s
          AND reason_code = 'APPEAL'
          AND {_report_open_filter_sql()}
        ORDER BY created_at DESC
        LIMIT 1
        """,
        (reporter_id, announcement_id),
    )
    if existing:
        return existing[0]
    return create_report(
        reporter_id=reporter_id,
        target_type="announcement",
        target_id=announcement_id,
        reason_code="APPEAL",
        reason_text=reason_text,
    )


def log_moderation_action(
    moderator_id: str,
    action_type: str,
    target_type: str,
    target_id: str,
    reason: Optional[str] = None,
    payload: Optional[Dict[str, Any]] = None,
) -> str:
    action_id = str(uuid.uuid4())
    columns = get_table_columns("moderation_actions")
    values: Dict[str, Any] = {
        "id": action_id,
        "moderator_id": moderator_id,
        "action_type": action_type,
        "target_type": target_type,
        "target_id": target_id,
    }
    if "reason" in columns:
        values["reason"] = reason
    if "payload" in columns:
        values["payload"] = payload or {}
    sql, params = _build_insert_sql(
        "moderation_actions",
        values,
        jsonb_columns={"payload"} if "payload" in values else set(),
    )
    execute(sql, params)
    return action_id



// ===== File 10/28: app/routes_module/__init__.py =====
from .router import router

__all__ = ["router"]
```

```
// ===== File 11/28: app/routes_module/router.py =====
from __future__ import annotations

from fastapi import APIRouter, Depends, HTTPException
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer

from app.db import fetch_one
from app.schemas import UserOut
from app.security import decode_token

from .schemas import RouteBuildIn, RouteContextOut, RouteDetailsOut
from .service import (
    DEFAULT_ROUTE_LIMIT,
    DEFAULT_ROUTE_RADIUS_METERS,
    build_route_for_announcement,
    build_route_for_current_user,
    build_route_from_polyline,
    build_route_context_for_announcement,
    build_route_context_for_current_user,
)

router = APIRouter(tags=["routes"])
bearer = HTTPBearer(auto_error=True)


def _user_from_token(token: str) -> UserOut:
    if token == "DEV_TOKEN":
        return UserOut(id="dev", email="dev@localdomain.com", role="user")

    try:
        payload = decode_token(token)
    except Exception:
        raise HTTPException(status_code=401, detail="Invalid token")

    user_id = payload.get("sub")
    if not user_id:
        raise HTTPException(status_code=401, detail="Invalid token payload")

    row = fetch_one("SELECT id::text, email, role FROM users WHERE id = %s", (user_id,))
    if not row:
        raise HTTPException(status_code=401, detail="User not found")

    return UserOut(id=row[0], email=row[1], role=row[2])


def get_current_user(creds: HTTPAuthorizationCredentials = Depends(bearer)) -> UserOut:
    return _user_from_token(creds.credentials)


@router.get("/announcements/{ann_id}/route", response_model=RouteDetailsOut)
def announcement_route(
    ann_id: str,
    radius_m: int = DEFAULT_ROUTE_RADIUS_METERS,
    limit: int = DEFAULT_ROUTE_LIMIT,
    user: UserOut = Depends(get_current_user),
) -> RouteDetailsOut:
    normalized_radius = max(50, min(int(radius_m), 5000))
    normalized_limit = max(1, min(int(limit), 100))
    return build_route_for_announcement(
        announcement_id=ann_id,
        user_id=user.id,
        radius_m=normalized_radius,
        limit=normalized_limit,
    )


@router.get("/routes/me/current", response_model=RouteDetailsOut)
def my_current_route(
```

```
    radius_m: int = DEFAULT_ROUTE_RADIUS_METERS,
    limit: int = DEFAULT_ROUTE_LIMIT,
    user: UserOut = Depends(get_current_user),
) -> RouteDetailsOut:
    normalized_radius = max(50, min(int(radius_m), 5000))
    normalized_limit = max(1, min(int(limit), 100))
    return build_route_for_current_user(
        user_id=user.id,
        radius_m=normalized_radius,
        limit=normalized_limit,
    )


@router.get("/announcements/{ann_id}/route/context", response_model=RouteContextOut)
def announcement_route_context(
    ann_id: str,
    radius_m: int = DEFAULT_ROUTE_RADIUS_METERS,
    user: UserOut = Depends(get_current_user),
) -> RouteContextOut:
    normalized_radius = max(50, min(int(radius_m), 5000))
    return build_route_context_for_announcement(
        announcement_id=ann_id,
        user_id=user.id,
        radius_m=normalized_radius,
    )


@router.get("/routes/me/current/context", response_model=RouteContextOut)
def my_current_route_context(
    radius_m: int = DEFAULT_ROUTE_RADIUS_METERS,
    user: UserOut = Depends(get_current_user),
) -> RouteContextOut:
    normalized_radius = max(50, min(int(radius_m), 5000))
    return build_route_context_for_current_user(
        user_id=user.id,
        radius_m=normalized_radius,
    )


@router.post("/route/build", response_model=RouteDetailsOut)
def route_build(
    request: RouteBuildIn,
    limit: int = DEFAULT_ROUTE_LIMIT,
    user: UserOut = Depends(get_current_user),
) -> RouteDetailsOut:
    normalized_limit = max(1, min(int(limit), 100))
    return build_route_from_polyline(
        announcement_id=request.announcement_id,
        user_id=user.id,
        polyline=request.polyline,
        start_address=request.start_address,
        end_address=request.end_address,
        distance_meters=request.distance_meters,
        duration_seconds=request.duration_seconds,
        radius_m=request.radius_m,
        limit=normalized_limit,
        travel_mode=request.travel_mode,
    )



// ===== File 12/28: app/routes_module/schemas.py =====
from __future__ import annotations

from typing import List, Optional

from pydantic import BaseModel, Field
```

```
class CoordinateIn(BaseModel):
    lat: float
    lon: float


class CoordinateOut(BaseModel):
    lat: float
    lon: float


class RouteBuildIn(BaseModel):
    announcement_id: str | None = None
    polyline: List[List[float]] = Field(default_factory=list)
    start_address: str | None = None
    end_address: str | None = None
    distance_meters: int | None = None
    duration_seconds: int | None = None
    radius_m: int = Field(default=500, ge=50, le=5_000)
    travel_mode: str = Field(default="driving")


class RouteContextOut(BaseModel):
    entity_id: str
    start_address: str
    end_address: str
    start: CoordinateOut
    end: CoordinateOut
    radius_m: int = Field(default=500)
    travel_mode: str = Field(default="driving")


class RouteTaskByPathOut(BaseModel):
    id: str
    title: str
    category: Optional[str] = None
    address_text: Optional[str] = None
    latitude: Optional[float] = None
    longitude: Optional[float] = None
    distance_to_route_meters: float = Field(default=0)
    price_text: Optional[str] = None
    preview_image_url: Optional[str] = None
    status: Optional[str] = None


class RouteDetailsOut(BaseModel):
    entity_id: str
    start_address: str
    end_address: str
    distance_meters: int = 0
    duration_seconds: int = 0
    distance_text: str
    duration_text: str
    polyline: List[List[float]] = Field(default_factory=list)
    tasks_by_route: List[RouteTaskByPathOut] = Field(default_factory=list)



// ===== File 13/28: app/routes_module/service.py =====
from __future__ import annotations

import json
import math
import os
from typing import Any

from fastapi import HTTPException
```

```
from app.db import fetch_all, fetch_one
from app.geocoding import geocode_address
from app.task_compat import ensure_task_payload

from .schemas import CoordinateOut, RouteContextOut, RouteDetailsOut, RouteTaskByPathOut
from .sql import (
    FIND_CURRENT_ROUTE_TASK_SQL,
    FIND_TASK_ROUTE_CONTEXT_SQL,
    NEARBY_TASKS_BY_ROUTE_SQL,
)

EARTH_RADIUS_M = 6_371_008.8
DEFAULT_ROUTE_RADIUS_METERS = int(os.getenv("ROUTE_TASK_RADIUS_METERS", "500"))
DEFAULT_ROUTE_LIMIT = int(os.getenv("ROUTE_TASKS_LIMIT", "50"))
DEFAULT_TRAVEL_MODE = os.getenv("ROUTE_DEFAULT_TRAVEL_MODE", "driving")
SUPPORTED_TRAVEL_MODES = {"driving", "walking", "truck", "transit", "bicycle", "scooter"}


def build_route_for_current_user(
    user_id: str,
    *,
    radius_m: int = DEFAULT_ROUTE_RADIUS_METERS,
    limit: int = DEFAULT_ROUTE_LIMIT,
) -> RouteDetailsOut:
    announcement_id = _resolve_current_announcement_id(user_id)
    return build_route_for_announcement(
        announcement_id=announcement_id,
        user_id=user_id,
        radius_m=radius_m,
        limit=limit,
    )


def build_route_for_announcement(
    announcement_id: str,
    user_id: str,
    *,
    radius_m: int = DEFAULT_ROUTE_RADIUS_METERS,
    limit: int = DEFAULT_ROUTE_LIMIT,
) -> RouteDetailsOut:
    context = build_route_context_for_announcement(
        announcement_id=announcement_id,
        user_id=user_id,
        radius_m=radius_m,
    )
    return build_route_from_polyline(
        announcement_id=announcement_id,
        user_id=user_id,
        polyline=[
            [context.start.lat, context.start.lon],
            [context.end.lat, context.end.lon],
        ],
        start_address=context.start_address,
        end_address=context.end_address,
        distance_meters=None,
        duration_seconds=None,
        radius_m=radius_m,
        limit=limit,
        travel_mode=context.travel_mode,
    )


def build_route_context_for_current_user(
    user_id: str,
    *,
    radius_m: int = DEFAULT_ROUTE_RADIUS_METERS,
) -> RouteContextOut:
    announcement_id = _resolve_current_announcement_id(user_id)
```

```
    return build_route_context_for_announcement(
        announcement_id=announcement_id,
        user_id=user_id,
        radius_m=radius_m,
    )


def build_route_context_for_announcement(
    announcement_id: str,
    user_id: str,
    *,
    radius_m: int = DEFAULT_ROUTE_RADIUS_METERS,
) -> RouteContextOut:
    context = _load_route_context(announcement_id=announcement_id, user_id=user_id)
    start_point = context["start_point"]
    end_point = context["end_point"]

    return RouteContextOut(
        entity_id=context["announcement_id"],
        start_address=context["start_address"],
        end_address=context["end_address"],
        start=CoordinateOut(lat=start_point[0], lon=start_point[1]),
        end=CoordinateOut(lat=end_point[0], lon=end_point[1]),
        radius_m=max(50, int(radius_m)),
        travel_mode=context["travel_mode"],
    )


def build_route_from_polyline(
    *,
    announcement_id: str | None,
    user_id: str,
    polyline: list[list[float]],
    start_address: str | None,
    end_address: str | None,
    distance_meters: int | None,
    duration_seconds: int | None,
    radius_m: int = DEFAULT_ROUTE_RADIUS_METERS,
    limit: int = DEFAULT_ROUTE_LIMIT,
    travel_mode: str = DEFAULT_TRAVEL_MODE,
) -> RouteDetailsOut:
    resolved_announcement_id = announcement_id or _resolve_current_announcement_id(user_id)
    context = _load_route_context(announcement_id=resolved_announcement_id, user_id=user_id)

    normalized_polyline = _parse_input_polyline(polyline)
    if len(normalized_polyline) < 2:
        raise HTTPException(status_code=422, detail="Polyline маршрута должна содержать минимум 2 точки")

    normalized_mode = _normalize_travel_mode(travel_mode)
    effective_distance = (
        int(distance_meters)
        if distance_meters is not None and int(distance_meters) > 0
        else int(round(_polyline_length_meters(normalized_polyline)))
    )
    effective_duration = (
        int(duration_seconds)
        if duration_seconds is not None and int(duration_seconds) > 0
        else _estimate_duration_seconds(effective_distance, normalized_mode)
    )

    resolved_start_address = _normalize_address(start_address) or context["start_address"]
    resolved_end_address = _normalize_address(end_address) or context["end_address"]
    tasks_by_route = _fetch_tasks_by_route(
        current_announcement_id=context["announcement_id"],
        current_user_id=user_id,
        route_polyline=normalized_polyline,
        radius_m=radius_m,
        limit=limit,
```

```
    )

    return RouteDetailsOut(
        entity_id=context["announcement_id"],
        start_address=resolved_start_address,
        end_address=resolved_end_address,
        distance_meters=effective_distance,
        duration_seconds=effective_duration,
        distance_text=_format_distance(effective_distance),
        duration_text=_format_duration(effective_duration),
        polyline=normalized_polyline,
        tasks_by_route=tasks_by_route,
    )


def _resolve_current_announcement_id(user_id: str) -> str:
    row = fetch_one(FIND_CURRENT_ROUTE_TASK_SQL, (user_id,))
    if not row:
        raise HTTPException(status_code=404, detail="Для вас пока нет активного маршрута")
    return str(row[0])


def _load_route_context(*, announcement_id: str, user_id: str) -> dict[str, Any]:
    row = fetch_one(FIND_TASK_ROUTE_CONTEXT_SQL, (announcement_id,))
    if not row:
        raise HTTPException(status_code=404, detail="Задание не найдено")

    ann_id = str(row[0])
    ann_owner_id = str(row[1])
    category = str(row[2] or "")
    title = str(row[3] or "")
    performer_id = str(row[5]) if row[5] else None
    assignment_status = str(row[6] or "")
    execution_stage = str(row[7] or "")
    route_visibility = str(row[8] or "")
    data = ensure_task_payload(
        _coerce_data(row[4]),
        title=title,
        announcement_status=_route_announcement_status(
            assignment_status=assignment_status,
            execution_stage=execution_stage,
        ),
        assignment={
            "performer_id": performer_id,
            "assignment_status": assignment_status,
            "execution_stage": execution_stage,
            "route_visibility": route_visibility,
        },
    )

    _assert_route_access(
        owner_id=ann_owner_id,
        performer_id=performer_id,
        assignment_status=assignment_status,

        user_id=user_id,
    )

    if assignment_status not in {"assigned", "in_progress"}:
        raise HTTPException(status_code=409, detail="Маршрут появится после принятия активного задания")

    route_points = _extract_route_points(data, category)
    if not route_points:
        raise HTTPException(
            status_code=422,
            detail="Не удалось определить старт и финиш маршрута. Проверьте адреса/координаты объявления.",
        )
```

```
    start_point, end_point = route_points
    start_address, end_address = _resolve_route_addresses(data, category, start_point, end_point)

    return {
        "announcement_id": ann_id,
        "category": category,
        "data": data,
        "start_point": start_point,
        "end_point": end_point,
        "start_address": start_address,
        "end_address": end_address,
        "travel_mode": _task_travel_mode(data),
    }


def _assert_route_access(
    *,
    owner_id: str,
    performer_id: str | None,
    assignment_status: str,
    user_id: str,
) -> None:
    if owner_id == user_id:
        return

    if performer_id == user_id and assignment_status in {"assigned", "in_progress"}:
        return

    raise HTTPException(status_code=403, detail="У вас нет доступа к маршруту этого объявления")


def _route_announcement_status(*, assignment_status: str, execution_stage: str) -> str:
    normalized_assignment = (assignment_status or "").strip().lower()
    normalized_stage = (execution_stage or "").strip().lower()

    if normalized_stage in {"en_route", "on_site", "in_progress", "handoff"}:
        return "in_progress"
    if normalized_assignment == "assigned":
        return "assigned"
    if normalized_assignment == "in_progress":
        return "in_progress"
    return "active"


def _task_travel_mode(data: dict[str, Any]) -> str:
    task = data.get("task") if isinstance(data.get("task"), dict) else {}
    route = task.get("route") if isinstance(task.get("route"), dict) else {}
    return _normalize_travel_mode(route.get("travel_mode") or data.get("travel_mode") or DEFAULT_TRAVEL_MODE)


def _coerce_data(value: Any) -> dict[str, Any]:
    if isinstance(value, dict):
        return dict(value)
    if isinstance(value, str):
        try:
            parsed = json.loads(value)
            if isinstance(parsed, dict):
                return parsed
        except Exception:
            return {}
    return {}


def _parse_float(value: Any) -> float | None:
    if isinstance(value, (int, float)):
        return float(value)
    if isinstance(value, str):
        raw = value.strip().replace(",", ".")
```

```
        if not raw:
            return None
        try:
            return float(raw)
        except ValueError:
            return None
    return None


def _extract_point(value: Any) -> tuple[float, float] | None:
    if not isinstance(value, dict):
        return None
    lat = _parse_float(value.get("lat"))
    lon = _parse_float(value.get("lon"))
    if lat is None or lon is None:
        return None
    if not (-90 <= lat <= 90 and -180 <= lon <= 180):
        return None
    return lat, lon


def _normalize_address(value: Any) -> str | None:
    if not isinstance(value, str):
        return None
    normalized = " ".join(value.strip().split())
    return normalized or None


def _extract_route_points(
    data: dict[str, Any],
    category: str,
) -> tuple[tuple[float, float], tuple[float, float]] | None:
    normalized_category = category.strip().lower()
    task = data.get("task") if isinstance(data.get("task"), dict) else {}
    route = task.get("route") if isinstance(task.get("route"), dict) else {}
    route_source = route.get("source") if isinstance(route.get("source"), dict) else {}
    route_destination = route.get("destination") if isinstance(route.get("destination"), dict) else {}

    start = (
        _extract_point(data.get("pickup_point"))
        or _extract_point(data.get("help_point"))
        or _extract_point(data.get("start_point"))
        or _extract_point(data.get("point"))
        or _extract_point(route_source.get("point"))
    )
    end = (
        _extract_point(data.get("dropoff_point"))
        or _extract_point(data.get("end_point"))
        or _extract_point(data.get("destination_point"))
        or _extract_point(data.get("to_point"))
        or _extract_point(route_destination.get("point"))
    )

    start_address = (
        _normalize_address(data.get("pickup_address"))
        or _normalize_address(data.get("address"))
        or _normalize_address(data.get("start_address"))
        or _normalize_address(data.get("address_text"))
        or _normalize_address(route_source.get("address"))
    )
    end_address = (
        _normalize_address(data.get("dropoff_address"))
        or _normalize_address(data.get("end_address"))
        or _normalize_address(data.get("to_address"))
        or _normalize_address(data.get("destination_address"))
        or _normalize_address(route_destination.get("address"))
    )
```

```
    if normalized_category == "delivery":
        if start is None and start_address:
            start = geocode_address(start_address)
        if end is None and end_address:
            end = geocode_address(end_address)
    else:
        if start is None and start_address:
            start = geocode_address(start_address)
        if end is None and end_address:
            end = geocode_address(end_address)

    if start is None or end is None:
        return None
    return start, end


def _resolve_route_addresses(
    data: dict[str, Any],
    category: str,
    start_point: tuple[float, float],
    end_point: tuple[float, float],
) -> tuple[str, str]:
    category_key = category.strip().lower()
    task = data.get("task") if isinstance(data.get("task"), dict) else {}
    route = task.get("route") if isinstance(task.get("route"), dict) else {}
    route_source = route.get("source") if isinstance(route.get("source"), dict) else {}
    route_destination = route.get("destination") if isinstance(route.get("destination"), dict) else {}
    if category_key == "delivery":
                                    start_address    =    _normalize_address(data.get("pickup_address"))    or
_normalize_address(route_source.get("address"))
                                     end_address    =    _normalize_address(data.get("dropoff_address"))    or
_normalize_address(route_destination.get("address"))
    else:
        start_address = (
            _normalize_address(data.get("start_address"))
            or _normalize_address(data.get("address"))
            or _normalize_address(data.get("address_text"))
            or _normalize_address(route_source.get("address"))
        )
        end_address = (
            _normalize_address(data.get("end_address"))
            or _normalize_address(data.get("to_address"))
            or _normalize_address(data.get("destination_address"))
            or _normalize_address(route_destination.get("address"))
        )

    if not start_address:
        start_address = _format_point(start_point)
    if not end_address:
        end_address = _format_point(end_point)

    return start_address, end_address



def _format_point(point: tuple[float, float]) -> str:
    return f"{point[0]:.6f}, {point[1]:.6f}"


def _normalize_travel_mode(value: str | None) -> str:
    normalized = (value or DEFAULT_TRAVEL_MODE).strip().lower()
    aliases = {
        "pedestrian": "walking",
        "foot": "walking",
        "car": "driving",
    }
    normalized = aliases.get(normalized, normalized)
    if normalized not in SUPPORTED_TRAVEL_MODES:
```

```
        raise HTTPException(
            status_code=422,
            detail=(
                f"Неподдерживаемый travel_mode: {value!r}. "
                f"Поддерживаются: {', '.join(sorted(SUPPORTED_TRAVEL_MODES))}"
            ),
        )
    return normalized


def _parse_input_polyline(raw_polyline: list[list[float]]) -> list[list[float]]:
    polyline: list[list[float]] = []
    for raw_pair in raw_polyline:
        parsed = _parse_lat_lon_pair(raw_pair)
        if parsed is not None:
            polyline.append([parsed[0], parsed[1]])
    return polyline


def _parse_lat_lon_pair(raw_pair: Any) -> tuple[float, float] | None:
    if not isinstance(raw_pair, (list, tuple)) or len(raw_pair) < 2:
        return None

    first = _parse_float(raw_pair[0])
    second = _parse_float(raw_pair[1])
    if first is None or second is None:
        return None

    lat, lon = first, second
    if not (-90 <= lat <= 90 and -180 <= lon <= 180):
        lat, lon = second, first

    if not (-90 <= lat <= 90 and -180 <= lon <= 180):
        return None
    return lat, lon


def _polyline_length_meters(polyline: list[list[float]]) -> float:
    if len(polyline) < 2:
        return 0.0
    return sum(
        _haversine_distance_meters(tuple(start), tuple(end))
        for start, end in zip(polyline, polyline[1:])
    )


def _haversine_distance_meters(
    start: tuple[float, float],
    end: tuple[float, float],
) -> float:
    lat1 = math.radians(start[0])
    lat2 = math.radians(end[0])
    d_lat = lat2 - lat1
    d_lon = math.radians(end[1] - start[1])

    sin_lat = math.sin(d_lat / 2.0)
    sin_lon = math.sin(d_lon / 2.0)
    a = sin_lat * sin_lat + math.cos(lat1) * math.cos(lat2) * sin_lon * sin_lon
    return 2.0 * EARTH_RADIUS_M * math.asin(math.sqrt(max(0.0, min(1.0, a))))


def _estimate_duration_seconds(distance_meters: int, travel_mode: str) -> int:
    speeds = {
        "walking": 1.4,
        "bicycle": 4.5,
        "scooter": 5.5,
        "transit": 6.0,
        "driving": 8.0,
```

```
        "truck": 7.0,
    }
    speed = speeds.get(travel_mode, 6.0)
    return max(1, int(round(distance_meters / max(0.1, speed))))


def _fetch_tasks_by_route(
    *,
    current_announcement_id: str,
    current_user_id: str,
    route_polyline: list[list[float]],
    radius_m: int,
    limit: int,
) -> list[RouteTaskByPathOut]:
    route_geojson = {
        "type": "LineString",
        "coordinates": [[point[1], point[0]] for point in route_polyline],
    }
    rows = fetch_all(
        NEARBY_TASKS_BY_ROUTE_SQL,
        (
            json.dumps(route_geojson, ensure_ascii=False),
            current_announcement_id,
            current_user_id,
            int(radius_m),
            int(limit),
        ),
    )

    result: list[RouteTaskByPathOut] = []
    for row in rows:
        data = _coerce_data(row[4])
        result.append(
            RouteTaskByPathOut(
                id=str(row[0]),
                title=str(row[1] or "Без названия"),
                category=_normalize_address(row[2]),
                status=_normalize_address(row[3]),
                address_text=_extract_address_text(data, str(row[2] or "")),
                latitude=_parse_float(row[5]),
                longitude=_parse_float(row[6]),
                distance_to_route_meters=float(row[7] or 0),
                price_text=_extract_price_text(data),
                preview_image_url=_extract_preview_image_url(data),
            )
        )
    return result


def _extract_address_text(data: dict[str, Any], category: str) -> str | None:
    if category.strip().lower() == "delivery":
        pickup = _normalize_address(data.get("pickup_address"))
        dropoff = _normalize_address(data.get("dropoff_address"))
        if pickup and dropoff:
            return f"{pickup} → {dropoff}"
        return pickup or dropoff or _normalize_address(data.get("address_text"))
    return (
        _normalize_address(data.get("address"))
        or _normalize_address(data.get("address_text"))
        or _normalize_address(data.get("pickup_address"))
    )


def _extract_price_text(data: dict[str, Any]) -> str | None:
    budget_min = _to_int(data.get("budget_min"))
    budget_max = _to_int(data.get("budget_max"))
    budget = _to_int(data.get("budget"))
```

```
    if budget_min is not None and budget_max is not None:
        if budget_min == budget_max:
            return _format_price(budget_min)
        return f"{_format_price_raw(budget_min)}–{_format_price_raw(budget_max)} "
    if budget_min is not None:
        return f"от {_format_price_raw(budget_min)} "
    if budget_max is not None:
        return f"до {_format_price_raw(budget_max)} "
    if budget is not None:
        return _format_price(budget)
    return None


def _format_price(value: int) -> str:
    return f"{_format_price_raw(value)} "


def _format_price_raw(value: int) -> str:
    return f"{value:,}".replace(",", " ")


def _to_int(value: Any) -> int | None:
    parsed = _parse_float(value)
    if parsed is None:
        return None
    return int(round(parsed))


def _extract_preview_image_url(data: dict[str, Any]) -> str | None:
    media_candidates: list[Any] = []

    for key in ("media", "images", "photos"):
        raw_value = data.get(key)
        if isinstance(raw_value, list):
            media_candidates.extend(raw_value)
        elif raw_value is not None:
            media_candidates.append(raw_value)

    for candidate in media_candidates:
        url = _extract_media_url(candidate)
        if url:
            return url
    return None


def _extract_media_url(value: Any) -> str | None:
    if isinstance(value, str):
        normalized = value.strip()
        return normalized or None


    if isinstance(value, dict):
        for key in (
            "preview_url",
            "previewUrl",
            "thumbnail_url",
            "thumbnailUrl",
            "url",
            "image_url",
            "imageUrl",
            "file_url",
            "fileUrl",
            "path",
        ):
            candidate = value.get(key)
            if isinstance(candidate, str) and candidate.strip():
                return candidate.strip()
```

```
        nested = value.get("file")
        return _extract_media_url(nested)

    return None


def _format_distance(distance_meters: int) -> str:
    value = max(0, int(distance_meters))
    if value < 1000:
        return f"{value} м"
    km = value / 1000.0
    return f"{km:.1f} км"


def _format_duration(duration_seconds: int) -> str:
    seconds = max(0, int(duration_seconds))
    minutes = max(1, int(round(seconds / 60.0)))

    if minutes < 60:
        return f"{minutes} мин"

    hours = minutes // 60
    left_minutes = minutes % 60
    if left_minutes == 0:
        return f"{hours} ч"
    return f"{hours} ч {left_minutes} мин"



// ===== File 14/28: app/routes_module/sql.py =====
from __future__ import annotations

FIND_TASK_ROUTE_CONTEXT_SQL = """
SELECT
    t.id::text,
    t.customer_id::text,
    COALESCE(c.slug, 'help') AS category_slug,
    t.title,
    t.extra,
    ta.performer_id::text,
    ta.assignment_status::text,
    ta.execution_stage::text,
    COALESCE(ta.route_visibility::text, 'performer_only')
FROM tasks t
LEFT JOIN categories c
  ON c.id = t.category_id
LEFT JOIN LATERAL (
    SELECT performer_id, assignment_status, execution_stage, route_visibility
    FROM task_assignments
    WHERE task_id::text = t.id::text
      AND assignment_status IN ('assigned', 'in_progress')
    ORDER BY updated_at DESC NULLS LAST, created_at DESC NULLS LAST
    LIMIT 1
) ta ON TRUE
WHERE t.id::text = %s
  AND t.deleted_at IS NULL
LIMIT 1
"""

FIND_CURRENT_ROUTE_TASK_SQL = """
SELECT ta.task_id::text
FROM task_assignments ta
JOIN tasks t
  ON t.id::text = ta.task_id::text
WHERE ta.performer_id::text = %s
  AND ta.assignment_status IN ('assigned', 'in_progress')
  AND COALESCE(ta.route_visibility::text, 'performer_only') <> 'hidden'
  AND t.deleted_at IS NULL
```

```
ORDER BY ta.updated_at DESC NULLS LAST, ta.created_at DESC NULLS LAST
LIMIT 1
"""

NEARBY_TASKS_BY_ROUTE_SQL = """
WITH route AS (
    SELECT ST_SetSRID(ST_GeomFromGeoJSON(%s), 4326)::geography AS geog
),
candidates AS (
    SELECT
        a.id,
        a.title,
        a.category,
        a.status,
        a.data,
        COALESCE(
            a.location_point,
            CASE
                WHEN jsonb_typeof(a.data -> 'point') = 'object'
                     AND COALESCE(a.data -> 'point' ->> 'lat', '') ~ '^-?[0-9]+(\\.[0-9]+)?$'
                     AND COALESCE(a.data -> 'point' ->> 'lon', '') ~ '^-?[0-9]+(\\.[0-9]+)?$'
                THEN ST_SetSRID(
                    ST_MakePoint(
                        (a.data -> 'point' ->> 'lon')::double precision,
                        (a.data -> 'point' ->> 'lat')::double precision
                    ),
                    4326
                )::geography
                ELSE NULL
            END
        ) AS point_geog
FROM announcements a
WHERE a.deleted_at IS NULL
  AND a.status = 'active'
  AND a.id::text <> %s
  AND a.user_id::text <> %s
  AND NOT EXISTS (
      SELECT 1
      FROM announcement_offers ao
      WHERE ao.announcement_id::text = a.id::text
        AND ao.status = 'accepted'
        AND ao.deleted_at IS NULL
  )
)
SELECT
    c.id,
    c.title,
    c.category,
    c.status,
    c.data,
    ST_Y(c.point_geog::geometry) AS latitude,
    ST_X(c.point_geog::geometry) AS longitude,
    ST_Distance(c.point_geog, (SELECT geog FROM route)) AS distance_to_route_meters
FROM candidates c
WHERE c.point_geog IS NOT NULL
  AND ST_DWithin(c.point_geog, (SELECT geog FROM route), %s)
ORDER BY distance_to_route_meters ASC
LIMIT %s
"""



// ===== File 15/28: app/schema_compat.py =====
from __future__ import annotations

from functools import lru_cache
from typing import Iterable
```

```
from app.db import fetch_all


@lru_cache(maxsize=128)
def get_table_columns(table_name: str) -> frozenset[str]:
    rows = fetch_all(
        """
        SELECT column_name
        FROM information_schema.columns
        WHERE table_schema = 'public' AND table_name = %s
        """,
        (table_name,),
    )
    return frozenset(str(row[0]) for row in rows)


def clear_schema_cache() -> None:
    get_table_columns.cache_clear()


def table_has_column(table_name: str, column_name: str) -> bool:
    return column_name in get_table_columns(table_name)


def table_has_columns(table_name: str, column_names: Iterable[str]) -> bool:
    columns = get_table_columns(table_name)
    return all(column_name in columns for column_name in column_names)



// ===== File 16/28: app/schemas.py =====
from __future__ import annotations

from datetime import datetime
from typing import Any, Dict, Optional

from pydantic import BaseModel, EmailStr, Field


class RegisterIn(BaseModel):
    email: EmailStr
    password: str


class LoginIn(BaseModel):
    email: EmailStr
    password: str


class TokenOut(BaseModel):
    access_token: str
    token_type: str = "bearer"


class UserOut(BaseModel):
    id: str
    email: EmailStr
    role: str


class GeoPointOut(BaseModel):
    lat: float = Field(..., ge=-90, le=90)
    lon: float = Field(..., ge=-180, le=180)


class CurrentUserDetailsOut(BaseModel):
    id: str
    email: Optional[EmailStr] = None
```

```
    phone: Optional[str] = None
    created_at: datetime


class UserProfileOut(BaseModel):
    display_name: Optional[str] = None
    bio: Optional[str] = None
    city: Optional[str] = None
    preferred_address: Optional[str] = None
    home_location: Optional[GeoPointOut] = None


class UserStatsOut(BaseModel):
    rating_avg: float = 0.0
    rating_count: int = 0
    completed_count: int = 0
    cancelled_count: int = 0


class MeProfileOut(BaseModel):
    user: CurrentUserDetailsOut
    profile: UserProfileOut
    stats: UserStatsOut


class UpdateMyProfileIn(BaseModel):
    display_name: str = Field(..., min_length=2, max_length=80)
    bio: Optional[str] = Field(default=None, max_length=300)
    city: Optional[str] = Field(default=None, max_length=120)
    preferred_address: Optional[str] = Field(default=None, max_length=180)
    home_location: Optional[GeoPointOut] = None


class UserReviewOut(BaseModel):
    from_user_display_name: str
    stars: int = Field(..., ge=1, le=5)
    text: Optional[str] = None
    created_at: datetime


class UserReviewListOut(BaseModel):
    items: list[UserReviewOut] = Field(default_factory=list)


class DeviceRegisterIn(BaseModel):
    device_id: str = Field(..., min_length=1, max_length=200)
    platform: str = Field(..., min_length=1, max_length=32)
    push_token: Optional[str] = Field(default=None, max_length=500)
    locale: Optional[str] = Field(default=None, max_length=50)
    timezone: Optional[str] = Field(default=None, max_length=120)
    device_name: Optional[str] = Field(default=None, max_length=120)


class DeviceUnregisterIn(BaseModel):
    device_id: str = Field(..., min_length=1, max_length=200)
    push_token: Optional[str] = Field(default=None, max_length=500)


class OKOut(BaseModel):
    ok: bool = True


# ----------------------------
# Announcements (Ads)
# ----------------------------
class CreateAnnouncementIn(BaseModel):
    category: str = Field(..., min_length=1, max_length=64)
    title: str = Field(..., min_length=1, max_length=200)
```

```
    status: str = Field(default="active", max_length=32)
    data: Dict[str, Any] = Field(default_factory=dict)


class AnnouncementOut(BaseModel):
    id: str
    user_id: str
    category: str
    title: str
    status: str
    data: Dict[str, Any] = Field(default_factory=dict)
    created_at: datetime


class CreateOfferIn(BaseModel):
    message: Optional[str] = Field(default=None, max_length=1000)
    proposed_price: Optional[int] = Field(default=None, ge=0)
    pricing_mode: Optional[str] = Field(default=None, max_length=64)
    agreed_price: Optional[int] = Field(default=None, ge=0)
    minimum_price_accepted: Optional[bool] = None


class OfferPerformerProfileOut(BaseModel):
    user_id: str
    display_name: str
    city: Optional[str] = None
    contact: Optional[str] = None
    avatar_url: Optional[str] = None


class OfferPerformerStatsOut(BaseModel):
    rating_avg: float = 0.0
    rating_count: int = 0
    completed_count: int = 0
    cancelled_count: int = 0


class OfferOut(BaseModel):
    id: str
    announcement_id: str
    performer_id: str
    message: Optional[str] = None
    proposed_price: Optional[int] = None
    agreed_price: Optional[int] = None
    pricing_mode: Optional[str] = None
    minimum_price_accepted: bool = False
    can_reoffer: bool = True
    status: str
    created_at: datetime


class OfferOutExpanded(OfferOut):
    performer_profile: OfferPerformerProfileOut
    performer_stats: OfferPerformerStatsOut


class AcceptOfferOut(BaseModel):
    thread_id: str
    offer: OfferOutExpanded


class AppealIn(BaseModel):
    reason: Optional[str] = Field(default=None, max_length=2000)


class ReportCreateIn(BaseModel):
    target_type: str = Field(..., min_length=1, max_length=32)
    target_id: str = Field(..., min_length=1, max_length=128)
```

```
    reason_code: str = Field(..., min_length=1, max_length=64)
    reason_text: Optional[str] = Field(default=None, max_length=2000)


class ReportOut(BaseModel):
    id: str
    reporter_id: str
    target_type: str
    target_id: str
    reason_code: str
    reason_text: Optional[str] = None
    status: str
    resolution: Optional[str] = None
    resolved_by: Optional[str] = None
    moderator_comment: Optional[str] = None
    created_at: datetime
    resolved_at: Optional[datetime] = None


class SupportThreadOut(BaseModel):
    thread_id: str


class ChatThreadOut(BaseModel):
    thread_id: str
    kind: str

    partner_id: Optional[str] = None
    partner_display_name: str
    partner_avatar_url: Optional[str] = None
    last_message_text: Optional[str] = None
    last_message_at: Optional[datetime] = None
    unread_count: int = 0
    announcement_id: Optional[str] = None
    announcement_title: Optional[str] = None


class SupportMessageIn(BaseModel):
    text: str = Field(..., min_length=1, max_length=5000)


class ChatMessageIn(BaseModel):
    text: str = Field(..., min_length=1, max_length=5000)


class ExecutionStageUpdateIn(BaseModel):
    stage: str = Field(..., min_length=1, max_length=64)


class ChatMessageOut(BaseModel):
    id: str
    thread_id: str
    sender_id: str
    text: str
    created_at: datetime


class SupportMessageOut(BaseModel):
    id: str
    thread_id: str
    sender_id: str
    type: str
    text: str
    is_blocked: bool = False
    blocked_reason: Optional[str] = None
    created_at: datetime
    edited_at: Optional[datetime] = None
    deleted_at: Optional[datetime] = None
```

```
class TextCheckIn(BaseModel):
    text: str


class TextCheckOut(BaseModel):
    label: str
    reason: str = ""
    t: float | None = None


class MediaUploadOut(BaseModel):
    announcement: AnnouncementOut
    max_nsfw: float
    decision: str
    can_appeal: bool
    message: str



// ===== File 17/28: app/security.py =====
# app/security.py

import hashlib
import os
from datetime import datetime, timedelta, timezone
from typing import Any, Dict
from uuid import UUID

import bcrypt
from dotenv import load_dotenv
from jose import JWTError, jwt

load_dotenv()


_BCRYPT_SHA256_PREFIX = "bcrypt_sha256$"


def _password_bytes(password: str) -> bytes:
    return (password or "").encode("utf-8")


def _bcrypt_sha256_input(password: str) -> bytes:
    return hashlib.sha256(_password_bytes(password)).hexdigest().encode("ascii")


def _json_safe(value: Any) -> Any:
    if isinstance(value, UUID):
        return str(value)
    if isinstance(value, dict):
        return {str(k): _json_safe(v) for k, v in value.items()}
    if isinstance(value, (list, tuple, set)):
        return [_json_safe(item) for item in value]
    return value


def hash_password(password: str) -> str:
    digest = _bcrypt_sha256_input(password)
    hashed = bcrypt.hashpw(digest, bcrypt.gensalt())
    return f"{_BCRYPT_SHA256_PREFIX}{hashed.decode('utf-8')}"


def verify_password(password: str, password_hash: str) -> bool:
    if not password_hash:
        return False
```

```
    encoded_hash = password_hash.encode("utf-8")

    if password_hash.startswith(_BCRYPT_SHA256_PREFIX):
        raw_hash = password_hash[len(_BCRYPT_SHA256_PREFIX) :].encode("utf-8")
        return bcrypt.checkpw(_bcrypt_sha256_input(password), raw_hash)

    try:
        return bcrypt.checkpw(_password_bytes(password), encoded_hash)
    except ValueError:
        # Legacy bcrypt hashes may have been created by libraries that silently
        # truncated passwords to 72 bytes. Keep verification compatible.
        return bcrypt.checkpw(_password_bytes(password)[:72], encoded_hash)


JWT_SECRET = os.getenv("JWT_SECRET", "DEV_JWT_SECRET_CHANGE_ME")
JWT_ALG = os.getenv("JWT_ALG", "HS256")
JWT_EXPIRE_MINUTES = int(os.getenv("JWT_EXPIRE_MINUTES", "10080"))


def create_access_token(payload: Dict[str, Any], expires_minutes: int | None = None) -> str:
    data = _json_safe(dict(payload))

    exp_minutes = expires_minutes if expires_minutes is not None else JWT_EXPIRE_MINUTES
    exp = datetime.now(timezone.utc) + timedelta(minutes=exp_minutes)

    data["exp"] = exp
    data["iat"] = datetime.now(timezone.utc)

    return jwt.encode(data, JWT_SECRET, algorithm=JWT_ALG)


def decode_token(token: str) -> Dict[str, Any]:
    try:
        return jwt.decode(token, JWT_SECRET, algorithms=[JWT_ALG])
    except JWTError as e:
        raise ValueError("Invalid token") from e



// ===== File 18/28: app/support.py =====
from __future__ import annotations

import uuid
from datetime import datetime
from typing import Any, Dict, List, Optional

from fastapi import HTTPException

from app.db import execute, fetch_all, fetch_one
from app.ops import create_notification


STAFF_ROLES = ("admin", "moderator", "support")


def _message_row_to_dict(row) -> Dict[str, Any]:
    return {
        "id": row[0],
        "thread_id": row[1],
        "sender_id": row[2],
        "type": row[3],
        "text": row[4],
        "is_blocked": bool(row[5]),
        "blocked_reason": row[6],
        "created_at": row[7],
        "edited_at": row[8],
        "deleted_at": row[9],
    }
```

```
def _normalize_text(text: str) -> str:
    return " ".join((text or "").strip().split())


def _pick_default_staff_user_id() -> Optional[str]:
    row = fetch_one(
        """
        SELECT id
        FROM users
        WHERE role IN ('support', 'admin', 'moderator')
        ORDER BY CASE role
            WHEN 'support' THEN 0
            WHEN 'admin' THEN 1
            ELSE 2
        END, created_at ASC
        LIMIT 1
        """
    )
    return row[0] if row else None


def ensure_support_participant(thread_id: str, user_id: str, role: str) -> None:
    execute(
        """
        INSERT INTO chat_participants (thread_id, user_id, role)
        VALUES (%s, %s, %s)
        ON CONFLICT (thread_id, user_id)
        DO UPDATE SET role = EXCLUDED.role, left_at = NULL
        """,
        (thread_id, user_id, role),
    )


def get_or_create_support_thread(user_id: str) -> str:
    existing = fetch_one(
        """
        SELECT ct.id
        FROM chat_threads ct
        JOIN chat_participants cp
          ON cp.thread_id = ct.id
        WHERE ct.kind = 'support'
          AND cp.user_id = %s
          AND cp.role = 'user'
          AND cp.left_at IS NULL
        ORDER BY ct.created_at DESC
        LIMIT 1
        """,
        (user_id,),
    )
    if existing:
        return existing[0]

    thread_id = str(uuid.uuid4())
    execute(
        """
        INSERT INTO chat_threads (id, kind, task_id, offer_id, last_message_at)
        VALUES (%s, 'support', NULL, NULL, NULL)
        """,
        (thread_id,),
    )
    ensure_support_participant(thread_id, user_id, "user")

    staff_id = _pick_default_staff_user_id()
    if staff_id and staff_id != user_id:
        ensure_support_participant(thread_id, staff_id, "support")
```

```
    return thread_id


def assert_support_access(thread_id: str, user_id: str) -> None:
    row = fetch_one(
        """
        SELECT 1
        FROM chat_threads ct
        JOIN chat_participants cp
          ON cp.thread_id = ct.id
        WHERE ct.id = %s
          AND ct.kind = 'support'
          AND cp.user_id = %s
          AND cp.left_at IS NULL
        """,
        (thread_id, user_id),
    )
    if not row:
        raise HTTPException(status_code=403, detail="Thread access denied")


def list_support_messages(
    thread_id: str,
    user_id: str,
    limit: int = 50,
    before: Optional[datetime] = None,
) -> List[Dict[str, Any]]:
    assert_support_access(thread_id, user_id)

    lim = max(1, min(int(limit), 100))
    params: List[Any] = [thread_id]
    sql = """
            SELECT id, thread_id, sender_id, type, text, is_blocked, blocked_reason, created_at, edited_at,
deleted_at
        FROM chat_messages
        WHERE thread_id = %s
          AND deleted_at IS NULL
    """
    if before is not None:
        sql += " AND created_at < %s"
        params.append(before)
    sql += " ORDER BY created_at DESC LIMIT %s"
    params.append(lim)

    rows = fetch_all(sql, tuple(params))
    ordered = list(reversed(rows))
    if ordered:
        execute(
            """
            UPDATE chat_participants
            SET last_read_message_id = %s
            WHERE thread_id = %s AND user_id = %s
            """,
            (ordered[-1][0], thread_id, user_id),
        )
    return [_message_row_to_dict(row) for row in ordered]


def post_support_message(
    thread_id: str,
    sender_id: str,
    text: str,
    sender_role: str,
) -> Dict[str, Any]:
    clean_text = _normalize_text(text)
    if not clean_text:
        raise HTTPException(status_code=400, detail="Message text is required")
```

```
    assert_support_access(thread_id, sender_id)

    message_id = str(uuid.uuid4())
    execute(
        """
        INSERT INTO chat_messages (id, thread_id, sender_id, type, text)
        VALUES (%s, %s, %s, 'text', %s)
        """,
        (message_id, thread_id, sender_id, clean_text),
    )
    execute(
        "UPDATE chat_threads SET last_message_at = now() WHERE id = %s",
        (thread_id,),
    )
    execute(
        """
        UPDATE chat_participants
        SET last_read_message_id = %s
        WHERE thread_id = %s AND user_id = %s
        """,
        (message_id, thread_id, sender_id),
    )

    if sender_role in STAFF_ROLES:
        user_participant = fetch_one(
            """
            SELECT user_id
            FROM chat_participants
            WHERE thread_id = %s
              AND role = 'user'
              AND left_at IS NULL
            LIMIT 1
            """,
            (thread_id,),
        )
        if user_participant:

            create_notification(
                user_id=user_participant[0],
                notif_type="support",
                body=clean_text,
                payload={"thread_id": thread_id, "message_id": message_id},
            )

    row = fetch_one(
        """
            SELECT id, thread_id, sender_id, type, text, is_blocked, blocked_reason, created_at, edited_at,
deleted_at
        FROM chat_messages
        WHERE id = %s
        """,
        (message_id,),
    )
    if not row:
        raise HTTPException(status_code=500, detail="Message was not saved")
    return _message_row_to_dict(row)



// ===== File 19/28: app/task_compat.py =====
from __future__ import annotations

import copy
import uuid
from datetime import datetime
from typing import Any, Dict, Iterable, Optional
```

```
TASK_PUBLIC_STATUSES = {"published", "in_responses"}
TASK_ASSIGNMENT_ACTIVE_STATUSES = {"assigned", "in_progress"}
EXECUTION_CUSTOMER_VISIBLE = {"en_route", "on_site", "in_progress", "handoff"}
EXECUTION_TERMINAL = {"completed", "cancelled"}

LEGACY_TO_CANONICAL_OFFER_STATUS = {
    "pending": "sent",
    "accepted": "accepted_by_customer",
    "rejected": "rejected_by_customer",
    "withdrawn": "withdrawn_by_sender",
}

CANONICAL_TO_LEGACY_OFFER_STATUS = {
    "sent": "pending",
    "accepted_by_customer": "accepted",
    "rejected_by_customer": "rejected",
    "withdrawn_by_sender": "withdrawn",
    "cancelled_by_sender": "withdrawn",
}


def is_uuid_like(value: Any) -> bool:
    try:
        uuid.UUID(str(value))
        return True
    except Exception:
        return False


def normalize_optional_text(value: Any, *, collapse_spaces: bool = False) -> Optional[str]:
    if value is None:
        return None

    normalized = str(value).strip()
    if collapse_spaces:
        normalized = " ".join(normalized.split())
    return normalized or None


def normalize_json_object(value: Any) -> Dict[str, Any]:
    if isinstance(value, dict):
        return copy.deepcopy(value)
    return {}


def normalize_json_list(value: Any) -> list[Any]:
    if isinstance(value, list):
        return list(value)
    return []


def parse_float(value: Any) -> Optional[float]:
    if isinstance(value, (int, float)):
        return float(value)
    if isinstance(value, str):
        raw = value.strip().replace(" ", "").replace(",", ".")
        if not raw:
            return None
        try:
            return float(raw)
        except ValueError:
            return None
    return None


def parse_int(value: Any) -> Optional[int]:
    parsed = parse_float(value)
    if parsed is None:
```

```
        return None
    return int(round(parsed))


def extract_point(value: Any) -> Optional[tuple[float, float]]:
    if not isinstance(value, dict):
        return None

    lat = parse_float(value.get("lat"))
    lon = parse_float(value.get("lon"))
    if lat is None or lon is None:
        return None
    if not (-90 <= lat <= 90 and -180 <= lon <= 180):
        return None
    return lat, lon


def point_json(point: tuple[float, float]) -> Dict[str, float]:
    return {"lat": point[0], "lon": point[1]}


def current_iso() -> str:
    return datetime.utcnow().isoformat() + "Z"


def builder_category_slug(raw_category: Optional[str]) -> str:
    normalized = (raw_category or "").strip().lower()
    if normalized == "delivery":
        return "delivery"
    if normalized in {"help", "errands"}:
        return "errands"
    if normalized in {"shopping", "buy"}:
        return "shopping"
    return "errands"


def announcement_status_to_task_fields(
    announcement_status: Optional[str],
    *,
    deleted: bool = False,
    has_accepted_offer: bool = False,
) -> tuple[str, str]:
    if deleted:
        return "closed", "published"

    normalized = (announcement_status or "").strip().lower()
    if normalized in {"pending_review", "pending", "review", "in_review"}:
        return "review", "pending"
    if normalized == "needs_fix":
        return "draft", "needs_fix"
    if normalized in {"rejected", "declined"}:
        return "draft", "rejected"
    if normalized == "archived":
        return "closed", "published"
    if normalized in {"completed", "done"}:
        return "completed", "published"
    if normalized in {"cancelled", "canceled"}:
        return "cancelled", "published"
    if normalized in {"assigned", "agreed"}:
        return "agreed", "published"
    if normalized in {"in_progress", "executing"}:
        return "in_progress", "published"
    if normalized in {"active", "published", "open"}:
        return ("agreed" if has_accepted_offer else "published"), "published"
    return "draft", "pending"


def task_to_announcement_status(
```

```
    task_status: Optional[str],
    moderation_status: Optional[str],
    deleted_at: Any,
    *,
    assignment_status: Optional[str] = None,
    execution_stage: Optional[str] = None,
) -> str:
    if deleted_at is not None:
        return "deleted"

    moderation = (moderation_status or "").strip().lower()
    if moderation == "pending":
        return "pending_review"
    if moderation == "needs_fix":
        return "needs_fix"
    if moderation in {"rejected", "blocked"}:
        return "rejected"

    normalized_task_status = (task_status or "").strip().lower()
    normalized_assignment_status = (assignment_status or "").strip().lower()
    normalized_execution_stage = canonical_execution_status(
        execution_stage=execution_stage,
        assignment_status=normalized_assignment_status,
        current_value=None,
    )

    if normalized_task_status == "closed":
        return "archived"
    if normalized_task_status == "cancelled" or normalized_execution_stage == "cancelled":
        return "cancelled"
    if normalized_task_status == "completed" or normalized_execution_stage == "completed":
        return "completed"
     if normalized_task_status == "in_progress" or normalized_execution_stage in EXECUTION_CUSTOMER_VISIBLE -
{"completed"}:
        return "in_progress"
    if normalized_task_status == "agreed" or normalized_assignment_status == "assigned":
        return "assigned"
    if normalized_task_status in TASK_PUBLIC_STATUSES:
        return "active"
    if normalized_task_status in {"draft", "review", "deferred"}:
        return "pending_review"
    return "active"


def canonical_execution_status(
    *,
    execution_stage: Optional[str],
    assignment_status: Optional[str],
    current_value: Optional[str],
) -> str:
    explicit = normalize_optional_text(execution_stage)
    if explicit:
        normalized = explicit.lower()
        if normalized in {
            "open",
            "accepted",

            "awaiting_acceptance",
            "en_route",
            "on_site",
            "in_progress",
            "handoff",
            "completed",
            "cancelled",
            "disputed",
        }:
            return normalized
```

```
    assignment = (assignment_status or "").strip().lower()
    if assignment == "in_progress":
        return "in_progress"
    if assignment == "assigned":
        return "accepted"
    if assignment == "completed":
        return "completed"
    if assignment == "cancelled":
        return "cancelled"

    current = (current_value or "").strip().lower()
    return current or "open"


def route_visibility_for_execution(execution_stage: Optional[str]) -> str:
    normalized = canonical_execution_status(
        execution_stage=execution_stage,
        assignment_status=None,
        current_value=None,
    )
    if normalized in EXECUTION_TERMINAL:
        return "hidden"
    if normalized in EXECUTION_CUSTOMER_VISIBLE:
        return "customer_visible"
    return "performer_only"


def legacy_offer_status_to_canonical(status: Optional[str]) -> str:
    normalized = (status or "").strip().lower()
    return LEGACY_TO_CANONICAL_OFFER_STATUS.get(normalized, "sent")


def canonical_offer_status_to_legacy(status: Optional[str]) -> str:
    normalized = (status or "").strip().lower()
    return CANONICAL_TO_LEGACY_OFFER_STATUS.get(normalized, "pending")


def primary_source_address(data: Dict[str, Any]) -> Optional[str]:
    for key in ("pickup_address", "address", "source_address", "start_address", "address_text"):
        value = normalize_optional_text(data.get(key), collapse_spaces=True)
        if value:
            return value
    return None


def primary_destination_address(data: Dict[str, Any]) -> Optional[str]:
    for key in ("dropoff_address", "destination_address", "end_address", "to_address"):
        value = normalize_optional_text(data.get(key), collapse_spaces=True)
        if value:
            return value
    return None


def primary_map_point(data: Dict[str, Any]) -> Optional[tuple[float, float]]:
    for key in ("point", "pickup_point", "help_point", "source_point"):
        point = extract_point(data.get(key))
        if point:
            return point
    return None


def destination_point(data: Dict[str, Any]) -> Optional[tuple[float, float]]:
    for key in ("dropoff_point", "destination_point", "end_point", "to_point"):
        point = extract_point(data.get(key))
        if point:
            return point
    return None
```

```python
def derive_quick_offer_price(data: Dict[str, Any]) -> Optional[int]:
    task = normalize_json_object(data.get("task"))
    offer_policy = normalize_json_object(task.get("offer_policy"))
    budget = normalize_json_object(task.get("budget"))
    return (
        parse_int(offer_policy.get("quick_offer_price"))
        or parse_int(data.get("quick_offer_price"))
        or parse_int(data.get("budget_min"))
        or parse_int(budget.get("min"))
        or parse_int(data.get("budget"))
        or parse_int(budget.get("amount"))
        or parse_int(data.get("budget_max"))
        or parse_int(budget.get("max"))
    )


def derive_budget_bounds(data: Dict[str, Any]) -> tuple[Optional[int], Optional[int]]:
    task = normalize_json_object(data.get("task"))
    budget = normalize_json_object(task.get("budget"))
    budget_min = parse_int(budget.get("min")) or parse_int(data.get("budget_min"))
    budget_max = (
        parse_int(budget.get("max"))
        or parse_int(data.get("budget_max"))
        or parse_int(budget.get("amount"))
        or parse_int(data.get("budget"))
    )
    if budget_min is not None and budget_max is not None and budget_min > budget_max:
        budget_min, budget_max = budget_max, budget_min
    return budget_min, budget_max


def derive_reward_amount(data: Dict[str, Any]) -> int:
    task = normalize_json_object(data.get("task"))
    budget = normalize_json_object(task.get("budget"))
    return max(
        0,
        parse_int(budget.get("amount"))
        or parse_int(data.get("budget"))
        or derive_quick_offer_price(data)
        or derive_budget_bounds(data)[0]
        or derive_budget_bounds(data)[1]
        or 0,
    )


def ensure_task_payload(
    raw_data: Dict[str, Any],
    *,
    title: str,
    announcement_status: str,
    deleted_at: Any = None,
    assignment: Optional[Dict[str, Any]] = None,
) -> Dict[str, Any]:
    data = normalize_json_object(raw_data)
    task = normalize_json_object(data.get("task"))
    builder = normalize_json_object(task.get("builder"))
    attributes = normalize_json_object(task.get("attributes"))
    budget = normalize_json_object(task.get("budget"))
    route = normalize_json_object(task.get("route"))
    contacts = normalize_json_object(task.get("contacts"))
    search = normalize_json_object(task.get("search"))
    offer_policy = normalize_json_object(task.get("offer_policy"))
    execution = normalize_json_object(task.get("execution"))
    lifecycle = normalize_json_object(task.get("lifecycle"))

    budget_min, budget_max = derive_budget_bounds(data)
    quick_offer_price = derive_quick_offer_price(data)
```

```
    source_address = primary_source_address(data)
    destination_address = primary_destination_address(data)
    source_point = primary_map_point(data)
    destination = destination_point(data)

    normalized_status = task_to_announcement_status(
        lifecycle.get("status"),
        normalize_optional_text(data.get("moderation"), collapse_spaces=False),
        deleted_at,
    )
    lifecycle_status = {
        "active": "open",
        "assigned": "assigned",
        "in_progress": "in_progress",
        "completed": "completed",
        "cancelled": "cancelled",
        "archived": "archived",
        "rejected": "rejected",
        "needs_fix": "needs_fix",
        "pending_review": "pending_review",
    }.get((announcement_status or normalized_status or "").strip().lower(), "open")

    assignment_info = normalize_json_object(assignment)
    execution_status = canonical_execution_status(
        execution_stage=assignment_info.get("execution_stage"),
        assignment_status=assignment_info.get("assignment_status"),
        current_value=normalize_optional_text(execution.get("status"))
        or normalize_optional_text(data.get("execution_status")),
    )

                     builder.setdefault("main_group",     normalize_optional_text(data.get("main_group"))     or
normalize_optional_text(data.get("category")) or "help")
                   builder.setdefault("action_type",    normalize_optional_text(data.get("action_type"))    or
normalize_optional_text(data.get("user_action_type")))
           builder.setdefault("resolved_category",   normalize_optional_text(data.get("resolved_category"))   or
normalize_optional_text(data.get("category")))
    builder.setdefault("item_type", normalize_optional_text(data.get("item_type")))
    builder.setdefault("purchase_type", normalize_optional_text(data.get("purchase_type")))
    builder.setdefault("help_type", normalize_optional_text(data.get("help_type")))
    builder.setdefault("source_kind", normalize_optional_text(data.get("source_kind")))
    builder.setdefault("destination_kind", normalize_optional_text(data.get("destination_kind")))
    builder.setdefault("urgency", normalize_optional_text(data.get("urgency")))
    builder.setdefault("task_brief", normalize_optional_text(data.get("task_brief")))
    builder.setdefault("notes", normalize_optional_text(data.get("notes")))

    for key in (
        "requires_vehicle",
        "needs_trunk",
        "requires_careful_handling",
        "needs_loader",
        "requires_lift_to_floor",
        "has_elevator",
        "wait_on_site",
        "contactless",
        "requires_receipt",
        "requires_confirmation_code",

        "call_before_arrival",
        "photo_report_required",
    ):
        if key not in attributes and key in data:
            attributes[key] = data.get(key)

    attributes.setdefault("weight_category", normalize_optional_text(data.get("weight_category")))
    attributes.setdefault("size_category", normalize_optional_text(data.get("size_category")))
    attributes.setdefault(
        "cargo",
        {
```

```
            "length_cm": parse_int(data.get("cargo_length_cm")) or parse_int(data.get("cargo_length")),
            "width_cm": parse_int(data.get("cargo_width_cm")) or parse_int(data.get("cargo_width")),
            "height_cm": parse_int(data.get("cargo_height_cm")) or parse_int(data.get("cargo_height")),
        },
    )
    attributes.setdefault("estimated_task_minutes", parse_int(data.get("estimated_task_minutes")))
    attributes.setdefault("waiting_minutes", parse_int(data.get("waiting_minutes")))
    attributes.setdefault("floor", parse_int(data.get("floor")))

    budget.setdefault("currency", "RUB")
    budget.setdefault("recommended_min", parse_int(data.get("recommended_price_min")))
    budget.setdefault("recommended_max", parse_int(data.get("recommended_price_max")))
    budget["min"] = budget.get("min") if budget.get("min") is not None else budget_min
    budget["max"] = budget.get("max") if budget.get("max") is not None else budget_max
             budget["amount"]   =   budget.get("amount")   if   budget.get("amount")   is   not   None   else
derive_reward_amount(data)

    route.setdefault("travel_mode", normalize_optional_text(data.get("travel_mode")) or "driving")
    route.setdefault("start_at", normalize_optional_text(data.get("start_at")))
    route.setdefault("has_end_time", bool(data.get("has_end_time")))
    route.setdefault("end_at", normalize_optional_text(data.get("end_at")))
    route.setdefault(
        "source",
        {
            "address": source_address,
            "kind": normalize_optional_text(data.get("source_kind")),
            "point": point_json(source_point) if source_point else None,
        },
    )
    route.setdefault(
        "destination",
        {
            "address": destination_address,
            "kind": normalize_optional_text(data.get("destination_kind")),
            "point": point_json(destination) if destination else None,
        },
    )

    contacts.setdefault("name", normalize_optional_text(data.get("contact_name")))
    contacts.setdefault("phone", normalize_optional_text(data.get("contact_phone")))
    contacts.setdefault("method", normalize_optional_text(data.get("contact_method")))
    contacts.setdefault("audience", normalize_optional_text(data.get("audience")))

              search.setdefault("generated_title",   normalize_optional_text(data.get("generated_title"))   or
normalize_optional_text(title))
    search.setdefault(
        "generated_description",
                                              normalize_optional_text(data.get("generated_description"))     or
normalize_optional_text(data.get("notes")),
    )
    search.setdefault("generated_tags", normalize_json_list(data.get("generated_tags")))
    search.setdefault("hints", normalize_json_list(data.get("ai_hints")))

    offer_policy.setdefault("quick_offer_enabled", True)
    offer_policy.setdefault("quick_offer_price", quick_offer_price)
    offer_policy.setdefault("counter_price_allowed", True)
            offer_policy.setdefault("reoffer_policy",   normalize_optional_text(data.get("reoffer_policy"))   or
"blocked_after_reject")

    execution["status"] = execution_status
    execution["assignment_id"] = assignment_info.get("id")
    execution["performer_user_id"] = assignment_info.get("performer_id")
    if assignment_info.get("chat_thread_id"):
        execution["chat_thread_id"] = assignment_info.get("chat_thread_id")
    if assignment_info.get("route_visibility"):
        execution["route_visibility"] = assignment_info.get("route_visibility")

    lifecycle["status"] = lifecycle_status
```

```
    lifecycle["deleted_at"] = deleted_at.isoformat() if hasattr(deleted_at, "isoformat") else deleted_at

    task["schema_version"] = 2
    task["lifecycle"] = lifecycle
    task["builder"] = builder
    task["attributes"] = attributes
    task["budget"] = budget
    task["route"] = route
    task["contacts"] = contacts
    task["search"] = search
    task["offer_policy"] = offer_policy
    task["execution"] = execution
    task["assignment"] = {
        "assignment_status": assignment_info.get("assignment_status"),
        "execution_status": execution_status,
        "performer_user_id": assignment_info.get("performer_id"),
        "chat_thread_id": assignment_info.get("chat_thread_id"),
        "route_visibility": assignment_info.get("route_visibility"),
    }

    data["task"] = task
    data["offer_policy"] = offer_policy
    data["execution"] = execution
    data["search"] = search
    if budget_min is not None:
        data["budget_min"] = budget_min
    if budget_max is not None:
        data["budget_max"] = budget_max
    if quick_offer_price is not None:
        data["quick_offer_price"] = quick_offer_price
    if route.get("source", {}).get("address"):
        data.setdefault("source_address", route["source"]["address"])
    if route.get("destination", {}).get("address"):
        data.setdefault("destination_address", route["destination"]["address"])
    return data


def task_row_to_announcement_dict(row: Dict[str, Any]) -> Dict[str, Any]:
    assignment = {
        "id": row.get("assignment_id"),
        "assignment_status": row.get("assignment_status"),
        "execution_stage": row.get("execution_stage"),
        "performer_id": row.get("assignment_performer_id"),
        "chat_thread_id": row.get("assignment_chat_thread_id"),
        "route_visibility": row.get("route_visibility"),
    }

    announcement_status = task_to_announcement_status(
        row.get("task_status"),
        row.get("moderation_status"),
        row.get("deleted_at"),
        assignment_status=row.get("assignment_status"),
        execution_stage=row.get("execution_stage"),
    )
    data = ensure_task_payload(
        normalize_json_object(row.get("extra")),
        title=row.get("title") or "",
        announcement_status=announcement_status,
        deleted_at=row.get("deleted_at"),
        assignment=assignment,
    )

    data["offers_count"] = int(row.get("responses_count") or 0)
    if row.get("address_text"):
        data["address_text"] = row.get("address_text")

    if row.get("location_lat") is not None and row.get("location_lon") is not None:
        data.setdefault("point", {"lat": float(row["location_lat"]), "lon": float(row["location_lon"])})
```

```
    return {
        "id": str(row.get("id")),
        "user_id": str(row.get("customer_id")),
        "category": row.get("category_slug") or builder_category_slug(data.get("category")),
        "title": row.get("title") or "",
        "status": announcement_status,
        "data": data,
        "created_at": row.get("created_at"),
    }


def task_offer_row_to_legacy_dict(row: Dict[str, Any]) -> Dict[str, Any]:
    status = canonical_offer_status_to_legacy(row.get("status"))
    if status == "rejected" and bool(row.get("can_reoffer")) is False:
        status = "blocked"

    proposed_price = row.get("proposed_price")
    if proposed_price is not None:
        proposed_price = int(round(float(proposed_price)))

    agreed_price = row.get("agreed_price")
    if agreed_price is not None:
        agreed_price = int(round(float(agreed_price)))

    return {
        "id": str(row.get("id")),
        "announcement_id": str(row.get("task_id")),
        "performer_id": str(row.get("performer_id")),
        "message": normalize_optional_text(row.get("message")),
        "proposed_price": proposed_price,
        "agreed_price": agreed_price,
        "pricing_mode": row.get("pricing_mode") or "counter_price",
        "minimum_price_accepted": bool(row.get("minimum_price_accepted")),
        "can_reoffer": bool(row.get("can_reoffer")),
        "status": status,
        "created_at": row.get("created_at"),
    }


def route_points_from_payload(task_id: str, data: Dict[str, Any]) -> list[Dict[str, Any]]:
    points: list[Dict[str, Any]] = []

    source = primary_map_point(data)
    if source:
        points.append(
            {
                "task_id": task_id,
                "point_order": 0,
                "title": "Старт",
                "address_text": primary_source_address(data),
                "point": source,
                "point_kind": "source",
            }
        )

    destination = destination_point(data)

    if destination:
        points.append(
            {
                "task_id": task_id,
                "point_order": 1,
                "title": "Финиш",
                "address_text": primary_destination_address(data),
                "point": destination,
                "point_kind": "destination",
            }
```

```
        )

    return points


def first_value(values: Iterable[Any]) -> Any:
    for value in values:
        if value is not None:
            return value
    return None



// ===== File 20/28: services/admin_panel/app/__init__.py =====
# Admin panel package



// ===== File 21/28: services/admin_panel/app/auth.py =====
from __future__ import annotations

from dataclasses import dataclass
from typing import Optional

from fastapi import HTTPException, Request
from sqlalchemy import select, text
from sqladmin.authentication import AuthenticationBackend

from app.security import create_access_token, decode_token, verify_password

from .db import SessionLocal
from .models_sqlalchemy import User


STAFF_ROLES = {"admin", "moderator", "support"}


@dataclass
class StaffUser:
    id: str
    email: str
    role: str


def _as_str(value: object) -> str:
    return "" if value is None else str(value)


def _extract_token(request: Request) -> Optional[str]:
    auth_header = request.headers.get("authorization", "")
    if auth_header.lower().startswith("bearer "):
        return auth_header.split(" ", 1)[1].strip()
    if hasattr(request, "session"):
        token = request.session.get("admin_token")
        if token:
            return str(token)
    token = request.cookies.get("admin_token")
    return token or None


def _load_staff_user(token: str) -> Optional[StaffUser]:
    try:
        payload = decode_token(token)
    except Exception:
        return None

    user_id = payload.get("sub")
    role = payload.get("role")
```

```
    if not user_id or role not in STAFF_ROLES:
        return None

    with SessionLocal() as session:
        row = session.execute(
            text(
                """
                SELECT id::text, email, role
                FROM users
                WHERE id::text = :user_id
                LIMIT 1
                """
            ),
            {"user_id": str(user_id)},
        ).first()
        if not row or row[2] not in STAFF_ROLES:
            return None
        return StaffUser(id=str(row[0]), email=str(row[1]), role=str(row[2]))


class AdminAuth(AuthenticationBackend):
    def __init__(self, secret_key: str) -> None:
        super().__init__(secret_key=secret_key)

    async def login(self, request: Request) -> bool:
        form = await request.form()
        email = str(form.get("email") or form.get("username") or "").strip().lower()
        password = str(form.get("password", ""))
        if not email or not password:
            return False

        with SessionLocal() as session:
            user = session.scalar(select(User).where(User.email == email))
            if not user or user.role not in STAFF_ROLES:
                return False
            if not verify_password(password, user.password_hash):
                return False

            user_id = _as_str(user.id)
            user_role = _as_str(user.role)
            user_email = _as_str(user.email)
            token = create_access_token({"sub": user_id, "role": user_role})

        request.session.update(
            {
                "admin_token": token,
                "admin_user_id": user_id,
                "admin_email": user_email,
                "admin_role": user_role,
            }
        )
        return True

    async def logout(self, request: Request) -> bool:
        request.session.clear()
        return True

    async def authenticate(self, request: Request) -> bool:
        token = _extract_token(request)
        if not token:
            return False

        user = _load_staff_user(token)
        if not user:
            request.session.clear()
            return False

        request.state.admin_user = user
```

```
        request.session.update(
            {
                "admin_token": token,
                "admin_user_id": user.id,
                "admin_email": user.email,
                "admin_role": user.role,
            }
        )
        return True


def get_staff_user(request: Request) -> StaffUser:
    user = getattr(request.state, "admin_user", None)
    if isinstance(user, StaffUser):
        return user

    token = _extract_token(request)
    if not token:
        raise HTTPException(status_code=401, detail="Admin auth required")

    loaded = _load_staff_user(token)
    if not loaded:
        raise HTTPException(status_code=401, detail="Admin auth required")
    request.state.admin_user = loaded
    return loaded


def require_staff_user(request: Request) -> StaffUser:
    return get_staff_user(request)


def require_admin_user(request: Request) -> StaffUser:
    user = get_staff_user(request)
    if user.role != "admin":
        raise HTTPException(status_code=403, detail="Admin role required")
    return user



// ===== File 22/28: services/admin_panel/app/crud.py =====
from __future__ import annotations

import json
import uuid
from datetime import datetime, timezone
from typing import Any, Dict, Iterable, Optional, Sequence

from sqlalchemy import select, text
from sqlalchemy.orm import Session
from sqlalchemy.orm.attributes import flag_modified

from .models_sqlalchemy import (
    Announcement,
    ChatMessage,
    ChatParticipant,
    ChatThread,
    User,
)


STAFF_ROLES = {"admin", "moderator", "support"}
QUEUE_STATUSES = {"pending_review", "needs_fix", "rejected"}
_SCHEMA_CACHE: dict[str, frozenset[str]] = {}
_COLUMN_INFO_CACHE: dict[tuple[str, str], tuple[Optional[str], Optional[str]]] = {}
_ENUM_LABELS_CACHE: dict[str, tuple[str, ...]] = {}
_OPENISH_STATUSES = (
    "open",
    "pending",
```

```
    "new",
    "created",
    "active",
    "review",
    "under_review",
    "in_review",
    "queued",
)
_RESOLVEDISH_STATUSES = (
    "resolved",
    "closed",
    "done",
    "processed",
    "valid",
    "invalid",
    "approved",
    "rejected",
    "revoked",
    "completed",
)


def _now() -> datetime:
    return datetime.now(timezone.utc)


def _now_iso() -> str:
    return _now().isoformat()


def _get_table_columns(session: Session, table_name: str) -> frozenset[str]:
    cached = _SCHEMA_CACHE.get(table_name)
    if cached is not None:
        return cached
    rows = session.execute(
        text(
            """
            SELECT column_name
            FROM information_schema.columns
            WHERE table_schema = 'public' AND table_name = :table_name
            """
        ),
        {"table_name": table_name},
    ).scalars().all()
    columns = frozenset(str(row) for row in rows)
    _SCHEMA_CACHE[table_name] = columns
    return columns


def _has_column(session: Session, table_name: str, column_name: str) -> bool:
    return column_name in _get_table_columns(session, table_name)


def _text_eq(left_expr: str, right_expr: str) -> str:
    return f"({left_expr})::text = ({right_expr})::text"


def _text_expr(expr: str) -> str:
    return f"({expr})::text"


def _quoted(values: Sequence[str]) -> str:
    return ", ".join(f"'{value}'" for value in values)


def _get_column_info(session: Session, table_name: str, column_name: str) -> tuple[Optional[str],
Optional[str]]:
    cache_key = (table_name, column_name)
```

```
    cached = _COLUMN_INFO_CACHE.get(cache_key)
    if cached is not None:
        return cached
    row = session.execute(
        text(
            """
            SELECT data_type, udt_name
            FROM information_schema.columns
            WHERE table_schema = 'public'
              AND table_name = :table_name
              AND column_name = :column_name
            """
        ),
        {"table_name": table_name, "column_name": column_name},
    ).first()
    value = (str(row[0]), str(row[1])) if row else (None, None)
    _COLUMN_INFO_CACHE[cache_key] = value
    return value


def _get_enum_labels(session: Session, enum_name: str) -> tuple[str, ...]:
    cached = _ENUM_LABELS_CACHE.get(enum_name)
    if cached is not None:
        return cached
    rows = session.execute(
        text(
            """
            SELECT e.enumlabel
            FROM pg_type t
            JOIN pg_enum e ON e.enumtypid = t.oid
            JOIN pg_namespace n ON n.oid = t.typnamespace
            WHERE n.nspname = 'public'
              AND t.typname = :enum_name
            ORDER BY e.enumsortorder
            """
        ),
        {"enum_name": enum_name},
    ).scalars().all()
    labels = tuple(str(row) for row in rows)
    _ENUM_LABELS_CACHE[enum_name] = labels
    return labels


def _map_status_assignment(
    session: Session,
    table_name: str,
    column_name: str,
    target_status: str,
) -> Optional[str]:
    if not _has_column(session, table_name, column_name):
        return None
    data_type, udt_name = _get_column_info(session, table_name, column_name)
    if data_type != "USER-DEFINED" or not udt_name:
        return target_status
    labels = _get_enum_labels(session, udt_name)
    by_lower = {label.lower(): label for label in labels}
    candidates = (target_status,) + (
        _OPENISH_STATUSES if target_status == "open" else _RESOLVEDISH_STATUSES
    )
    for candidate in candidates:
        value = by_lower.get(candidate.lower())
        if value:
            return value
    return None


def _report_status_expr(session: Session, alias: str = "r") -> str:
    if _has_column(session, "reports", "status"):
```

```
        return (
            f"CASE WHEN lower({_text_expr(f'{alias}.status')}) "
            f"IN ({_quoted(_RESOLVEDISH_STATUSES)}) THEN 'resolved' ELSE 'open' END"
        )
    if _has_column(session, "reports", "resolved_at"):
        return f"CASE WHEN {alias}.resolved_at IS NULL THEN 'open' ELSE 'resolved' END"
    if _has_column(session, "reports", "resolution"):
        return f"CASE WHEN {alias}.resolution IS NULL THEN 'open' ELSE 'resolved' END"
    return "'open'"


def _report_open_condition(session: Session, alias: str = "reports") -> str:
    return f"{_report_status_expr(session, alias)} = 'open'"


def _restriction_status_expr(session: Session, alias: str = "r") -> str:
    if _has_column(session, "user_restrictions", "status"):
        return (
            f"CASE WHEN lower({_text_expr(f'{alias}.status')}) "
            f"IN ('revoked', 'inactive', 'closed', 'disabled') THEN 'revoked' ELSE 'active' END"
        )
    if _has_column(session, "user_restrictions", "revoked_at"):
        return f"CASE WHEN {alias}.revoked_at IS NULL THEN 'active' ELSE 'revoked' END"
    return "'active'"


def _get_user_row(session: Session, user_id: str) -> Optional[dict[str, Any]]:
    row = session.execute(
        text(
            """
            SELECT id::text AS id,
                   email,
                   password_hash,
                   role,
                   created_at
            FROM users

            WHERE id::text = :user_id
            """
        ),
        {"user_id": user_id},
    ).mappings().first()
    return dict(row) if row else None


def _get_chat_user_filter(session: Session, participant_alias: str, user_alias: str) -> str:
    parts = []
    if _has_column(session, "chat_participants", "left_at"):
        parts.append(f"{participant_alias}.left_at IS NULL")
    if _has_column(session, "chat_participants", "role"):
        parts.append(f"{participant_alias}.role = 'user'")
    elif _has_column(session, "users", "role"):
            parts.append(f"COALESCE({_text_expr(f'{user_alias}.role')}, 'user') NOT IN ('admin', 'moderator',
'support')")
    return " AND ".join(parts) if parts else "1 = 1"


def _insert_row(
    session: Session,
    table_name: str,
    values: Dict[str, Any],
    jsonb_columns: Optional[set[str]] = None,
) -> None:
    jsonb_columns = jsonb_columns or set()
    columns = list(values.keys())
    placeholders: list[str] = []
    params: dict[str, Any] = {}
    for index, column in enumerate(columns):
```

```
        param_name = f"p_{index}"
        if column in jsonb_columns:
            placeholders.append(f":{param_name}::jsonb")
            params[param_name] = json.dumps(values[column], ensure_ascii=False)
        else:
            placeholders.append(f":{param_name}")
            params[param_name] = values[column]
    sql = f"INSERT INTO {table_name} ({', '.join(columns)}) VALUES ({', '.join(placeholders)})"
    session.execute(text(sql), params)


def _restriction_timestamp_expr(session: Session, alias: str = "r") -> str:
    if _has_column(session, "user_restrictions", "starts_at"):
        return f"{alias}.starts_at"
    if _has_column(session, "user_restrictions", "created_at"):
        return f"{alias}.created_at"
    return "NULL::timestamptz"


def _ensure_obj(value: Any) -> Dict[str, Any]:
    return value if isinstance(value, dict) else {}


def _ensure_list(value: Any) -> list[Any]:
    return value if isinstance(value, list) else []


def _set_decision(mod: Dict[str, Any], status: str, message: str) -> None:
    mod["decision"] = {"status": status, "message": message}


def _normalize_reason(reason: Dict[str, Any]) -> Dict[str, Any]:
    return {
        "field": str(reason.get("field", "")).strip(),
        "code": str(reason.get("code", "")).strip(),
        "details": str(reason.get("details", "")).strip(),
        "can_appeal": bool(reason.get("can_appeal", True)),
    }


def _merge_reasons(existing: Iterable[Any], incoming: Optional[Sequence[Dict[str, Any]]]) -> list[Dict[str,
Any]]:
    merged: list[Dict[str, Any]] = []
    seen: set[tuple[str, str, str, bool]] = set()

    for raw in list(existing) + list(incoming or []):
        if not isinstance(raw, dict):
            continue
        item = _normalize_reason(raw)
        if not item["field"] or not item["code"] or not item["details"]:
            continue
        key = (item["field"], item["code"], item["details"], item["can_appeal"])
        if key in seen:
            continue
        seen.add(key)
        merged.append(item)
    return merged


def _normalize_suggestions(suggestions: Optional[Sequence[str]]) -> list[str]:
    if suggestions is None:
        return []
    clean: list[str] = []
    for suggestion in suggestions:
        value = str(suggestion or "").strip()
        if value:
            clean.append(value)
    return clean
```

```
def _reset_appeal(mod: Dict[str, Any]) -> None:
    appeal = _ensure_obj(mod.get("appeal"))
    if appeal:
        appeal["requested"] = False
        appeal["resolved_at"] = _now_iso()
        mod["appeal"] = appeal


def _has_hard_block_reason(reasons: Sequence[Dict[str, Any]]) -> bool:
    for reason in reasons:
        code = str(reason.get("code", "")).lower()
        details = str(reason.get("details", "")).lower()
        if "hard" in code or "hard-block" in code or "hard-block" in details:
            return True
    return False


def _add_action(
    session: Session,
    moderator_id: str,
    action_type: str,
    target_type: str,
    target_id: str,
    reason: Optional[str] = None,
    payload: Optional[Dict[str, Any]] = None,
) -> str:
    columns = _get_table_columns(session, "moderation_actions")
    action_id = str(uuid.uuid4())
    values: Dict[str, Any] = {
        "id": action_id,
        "moderator_id": moderator_id,
        "action_type": action_type,
        "target_type": target_type,
        "target_id": target_id,
    }
    if "reason" in columns:
        values["reason"] = reason
    if "payload" in columns:
        values["payload"] = payload or {}
     _insert_row(session, "moderation_actions", values, jsonb_columns={"payload"} if "payload" in values else
set())
    return action_id


def _add_notification(
    session: Session,
    user_id: str,
    notif_type: str,
    body: str,
    payload: Optional[Dict[str, Any]] = None,
) -> str:
    columns = _get_table_columns(session, "notifications")
    notification_id = str(uuid.uuid4())
    values: Dict[str, Any] = {"id": notification_id, "user_id": user_id, "type": notif_type, "body": body}
    if "payload" in columns:
        values["payload"] = payload or {}
    if "is_read" in columns:
        values["is_read"] = False
    _insert_row(session, "notifications", values, jsonb_columns={"payload"} if "payload" in values else set())
    return notification_id


def list_users(session: Session, search: Optional[str] = None) -> list[dict[str, Any]]:
    stmt = select(User).order_by(User.created_at.desc())
    if search:
        stmt = stmt.where(User.email.ilike(f"%{search.strip()}%"))
```

```
    rows = session.scalars(stmt).all()
    return [
        {
            "id": row.id,
            "email": row.email,
            "role": row.role,
            "created_at": row.created_at,
        }
        for row in rows
    ]


def get_user_detail(session: Session, user_id: str) -> dict[str, Any]:
    user = _get_user_row(session, user_id)
    if not user:
        raise ValueError("User not found")

    restrictions = list_restrictions(session, search=str(user["email"]), status=None)
    return {
        "id": user["id"],
        "email": user["email"],
        "role": user["role"],
        "created_at": user["created_at"],
        "restrictions": [item for item in restrictions if str(item["user_id"]) == str(user["id"])],
    }


def update_user_role(
    session: Session,
    target_user_id: str,
    role: str,
    actor_id: str,
    actor_role: str,
) -> dict[str, Any]:
    if actor_role != "admin":
        raise PermissionError("Only admin can change roles")

    if role not in {"user", "support", "moderator", "admin"}:
        raise ValueError("Unsupported role")

    user = _get_user_row(session, target_user_id)
    if not user:
        raise ValueError("User not found")

    previous_role = str(user["role"])
    session.execute(
        text("UPDATE users SET role = :role WHERE id::text = :user_id"),
        {"role": role, "user_id": target_user_id},
    )
    _add_action(
        session=session,
        moderator_id=actor_id,
        action_type="user_role_change",
        target_type="user",
        target_id=str(user["id"]),
        reason=f"{previous_role} -> {role}",
        payload={"old_role": previous_role, "new_role": role},
    )
    _add_notification(
        session=session,
        user_id=str(user["id"]),
        notif_type="moderation",
        body=f"Роль учётной записи изменена: {role}",
        payload={"user_id": str(user["id"]), "new_role": role},
    )
    session.commit()
    return {
        "id": user["id"],
```

```
        "email": user["email"],
        "role": role,
        "created_at": user["created_at"],
    }


def list_moderation_announcements(
    session: Session,
    status_filter: Optional[str] = None,
    appeals_only: bool = False,
    search: Optional[str] = None,
) -> list[dict[str, Any]]:
    sql = """
        SELECT a.id,
               a.user_id,
               u.email AS user_email,
               a.category,
               a.title,
               a.status,
               a.data,
               a.created_at,
               a.updated_at,
               COALESCE((a.data->'moderation'->'appeal'->>'requested')::boolean, FALSE) AS appeal_requested
        FROM announcements a
        LEFT JOIN users u ON {user_join}
        WHERE a.deleted_at IS NULL
    """
    params: dict[str, Any] = {}
    sql = sql.format(user_join=_text_eq("u.id", "a.user_id"))

    if status_filter and status_filter in QUEUE_STATUSES | {"active", "archived"}:
        sql += " AND a.status = :status_filter"
        params["status_filter"] = status_filter
    else:
        sql += " AND a.status IN ('pending_review', 'needs_fix', 'rejected')"

    if appeals_only:
        sql += " AND COALESCE((a.data->'moderation'->'appeal'->>'requested')::boolean, FALSE) = TRUE"

    if search:
        params["search"] = f"%{search.strip()}%"
           sql += " AND (a.title ILIKE :search OR a.category ILIKE :search OR COALESCE(u.email, '') ILIKE
:search)"

    sql += " ORDER BY appeal_requested DESC, a.updated_at DESC, a.created_at DESC LIMIT 300"
    rows = session.execute(text(sql), params).mappings().all()
    return [dict(row) for row in rows]


def get_announcement_detail(session: Session, ann_id: str) -> dict[str, Any]:
    row = session.execute(
        text(
            """
            SELECT a.id,
                   a.user_id,
                   u.email AS user_email,
                   a.category,
                   a.title,
                   a.status,
                   a.data,
                   a.created_at,
                   a.updated_at,
                   a.deleted_at
            FROM announcements a
            LEFT JOIN users u ON {user_join}
            WHERE a.id::text = :ann_id
            """
            .format(user_join=_text_eq("u.id", "a.user_id"))
```

```
        ),
        {"ann_id": ann_id},
    ).mappings().first()
    if not row:
        raise ValueError("Announcement not found")
    return dict(row)


def apply_announcement_decision(
    session: Session,
    ann_id: str,
    moderator_id: str,
    decision: str,
    message: str,
    reasons: Optional[Sequence[Dict[str, Any]]] = None,
    suggestions: Optional[Sequence[str]] = None,
) -> Announcement:
    announcement = session.execute(
        text(
            """
            SELECT id::text AS id,
                   user_id::text AS user_id,
                   status,
                   data,
                   created_at,
                   updated_at,
                   deleted_at
            FROM announcements
            WHERE id::text = :ann_id
              AND deleted_at IS NULL
            """
        ),
        {"ann_id": ann_id},
    ).mappings().first()
    if not announcement:
        raise ValueError("Announcement not found")

    if decision not in {"approve", "needs_fix", "reject", "archive", "delete"}:
        raise ValueError("Unsupported decision")

    old_status = str(announcement["status"])
    payload_data = dict(announcement["data"] or {})
    moderation = _ensure_obj(payload_data.get("moderation"))
    merged_reasons = _merge_reasons(_ensure_list(moderation.get("reasons")), reasons)
    clean_suggestions = _normalize_suggestions(suggestions)
    new_status = old_status
    now = _now()

    if decision == "approve":
        new_status = "active"
        _set_decision(moderation, new_status, message)
        moderation["reasons"] = []
        moderation["suggestions"] = []
        moderation.pop("penalty_stub", None)
        _reset_appeal(moderation)
    elif decision == "needs_fix":
        new_status = "needs_fix"
        _set_decision(moderation, new_status, message)
        moderation["reasons"] = merged_reasons
        moderation["suggestions"] = clean_suggestions
        _reset_appeal(moderation)
    elif decision == "reject":
        new_status = "rejected"
        _set_decision(moderation, new_status, message)
        moderation["reasons"] = merged_reasons
        if suggestions is not None:
            moderation["suggestions"] = clean_suggestions
        if _has_hard_block_reason(moderation["reasons"]):
```

```
        moderation["penalty_stub"] = {"type": "warning", "points": 1, "applied_at": None}
    _reset_appeal(moderation)
elif decision == "archive":
    new_status = "archived"
    _set_decision(moderation, new_status, message)
    _reset_appeal(moderation)
elif decision == "delete":
    _set_decision(moderation, old_status, message)
    _reset_appeal(moderation)

payload_data["moderation"] = moderation
update_clauses = [
    "data = CAST(:data AS jsonb)",
    "updated_at = :updated_at",
]
params: dict[str, Any] = {
    "ann_id": ann_id,
    "data": json.dumps(payload_data, ensure_ascii=False),
    "updated_at": now,
}
if decision == "delete":
    update_clauses.append("deleted_at = :deleted_at")
    params["deleted_at"] = now
else:
    update_clauses.append("status = :new_status")
    params["new_status"] = new_status
session.execute(
    text(f"UPDATE announcements SET {', '.join(update_clauses)} WHERE id::text = :ann_id"),
    params,
)

if decision != "delete":
    report_columns = _get_table_columns(session, "reports")
    report_resolution = "valid" if decision in {"approve", "needs_fix"} else "invalid"

    update_clauses: list[str] = []
    params: dict[str, Any] = {"target_id": str(announcement["id"])}
    status_value = _map_status_assignment(session, "reports", "status", "resolved")
    if "status" in report_columns and status_value is not None:
        update_clauses.append("status = :status_value")
        params["status_value"] = status_value
    if "resolution" in report_columns:
        update_clauses.append("resolution = :resolution")
        params["resolution"] = report_resolution
    if "resolved_by" in report_columns:
        update_clauses.append("resolved_by = :moderator_id")
        params["moderator_id"] = moderator_id
    if "moderator_comment" in report_columns:
        update_clauses.append("moderator_comment = :comment")
        params["comment"] = message
    if "resolved_at" in report_columns:
        update_clauses.append("resolved_at = :resolved_at")
        params["resolved_at"] = now
    if update_clauses:
        session.execute(
            text(
                f"""
                UPDATE reports
                SET {', '.join(update_clauses)}
                WHERE target_type = 'announcement'
                  AND target_id::text = :target_id
                  AND reason_code = 'APPEAL'
                  AND {_report_open_condition(session)}
                """
            ),
            params,
        )
```

```
    _add_action(
        session=session,
        moderator_id=moderator_id,
        action_type=decision,
        target_type="announcement",
        target_id=str(announcement["id"]),
        reason=message,
        payload={
            "old_status": old_status,
            "new_status": new_status if decision != "delete" else old_status,
            "message": message,
            "reasons": merged_reasons if decision in {"needs_fix", "reject"} else [],
                                "suggestions":   clean_suggestions   if   suggestions   is   not   None   else
_ensure_list(moderation.get("suggestions")),
            "deleted": decision == "delete",
        },
    )
    _add_notification(
        session=session,
        user_id=str(announcement["user_id"]),
        notif_type="moderation",
        body=message,
        payload={"ann_id": str(announcement["id"]), "decision": decision},
    )

    session.commit()
    refreshed = session.execute(
        text(
            """
            SELECT id::text AS id,
                   user_id::text AS user_id,
                   category,
                   title,
                   status,
                   data,
                   created_at,
                   updated_at,
                   deleted_at
            FROM announcements
            WHERE id::text = :ann_id
            """
        ),
        {"ann_id": ann_id},
    ).mappings().first()
    if not refreshed:
        raise ValueError("Announcement not found after update")
    return Announcement(
        id=str(refreshed["id"]),
        user_id=str(refreshed["user_id"]),
        category=str(refreshed["category"]),
        title=str(refreshed["title"]),
        status=str(refreshed["status"]),
        data=dict(refreshed["data"] or {}),
        created_at=refreshed["created_at"],
        updated_at=refreshed["updated_at"],
        deleted_at=refreshed["deleted_at"],
    )


def list_reports(
    session: Session,
    search: Optional[str] = None,
    status: Optional[str] = "open",
) -> list[dict[str, Any]]:
    status_expr = _report_status_expr(session, "r")
    resolved_by_join = (
        f"LEFT JOIN users mu ON {_text_eq('mu.id', 'r.resolved_by')}"
        if _has_column(session, "reports", "resolved_by")
```

```
        else ""
    )
      resolved_by_expr = _text_expr("r.resolved_by") if _has_column(session, "reports", "resolved_by") else
"NULL::text"
    resolved_by_email_expr = "mu.email" if _has_column(session, "reports", "resolved_by") else "NULL::text"
       resolution_expr = _text_expr("r.resolution") if _has_column(session, "reports", "resolution") else
"NULL::text"
    moderator_comment_expr = (
        "r.moderator_comment" if _has_column(session, "reports", "moderator_comment") else "NULL::text"
    )
           resolved_at_expr = "r.resolved_at" if _has_column(session, "reports", "resolved_at") else
"NULL::timestamptz"
    sql = """
        SELECT r.id,
               r.reporter_id,
               ru.email AS reporter_email,
               r.target_type,
               r.target_id,
               r.reason_code,
               r.reason_text,
               {status_expr} AS status,
               {resolution_expr} AS resolution,
               {resolved_by_expr} AS resolved_by,
               {resolved_by_email_expr} AS resolved_by_email,
               {moderator_comment_expr} AS moderator_comment,
               r.created_at,
               {resolved_at_expr} AS resolved_at
        FROM reports r
        LEFT JOIN users ru ON {reporter_join}
        {resolved_by_join}
        WHERE 1 = 1
    """.format(
        status_expr=status_expr,
        resolution_expr=resolution_expr,
        resolved_by_expr=resolved_by_expr,
        resolved_by_email_expr=resolved_by_email_expr,
        moderator_comment_expr=moderator_comment_expr,
        resolved_at_expr=resolved_at_expr,
        resolved_by_join=resolved_by_join,
        reporter_join=_text_eq("ru.id", "r.reporter_id"),
    )
    params: dict[str, Any] = {}
    if status:
        sql += f" AND {status_expr} = :status"
        params["status"] = status
    if search:
        sql += (
            " AND (COALESCE(ru.email, '') ILIKE :search"
            " OR r.target_id::text ILIKE :search"
            " OR r.reason_code::text ILIKE :search)"
        )
        params["search"] = f"%{search.strip()}%"
    sql += " ORDER BY r.created_at DESC LIMIT 300"
    rows = session.execute(text(sql), params).mappings().all()
    return [dict(row) for row in rows]


def get_report_detail(session: Session, report_id: str) -> dict[str, Any]:
    status_expr = _report_status_expr(session, "r")
    resolved_by_join = (
        f"LEFT JOIN users mu ON {_text_eq('mu.id', 'r.resolved_by')}"
        if _has_column(session, "reports", "resolved_by")
        else ""
    )
      resolved_by_expr = _text_expr("r.resolved_by") if _has_column(session, "reports", "resolved_by") else
"NULL::text"
    resolved_by_email_expr = "mu.email" if _has_column(session, "reports", "resolved_by") else "NULL::text"
       resolution_expr = _text_expr("r.resolution") if _has_column(session, "reports", "resolution") else
```

```
"NULL::text"
    moderator_comment_expr = (
        "r.moderator_comment" if _has_column(session, "reports", "moderator_comment") else "NULL::text"
    )
           resolved_at_expr   =   "r.resolved_at"   if   _has_column(session,   "reports",   "resolved_at")   else
"NULL::timestamptz"
    row = session.execute(
        text(
            """
            SELECT r.id,
                   r.reporter_id,
                   ru.email AS reporter_email,
                   r.target_type,
                   r.target_id,
                   r.reason_code,
                   r.reason_text,
                   {status_expr} AS status,
                   {resolution_expr} AS resolution,
                   {resolved_by_expr} AS resolved_by,
                   {resolved_by_email_expr} AS resolved_by_email,
                   {moderator_comment_expr} AS moderator_comment,
                   r.created_at,
                   {resolved_at_expr} AS resolved_at
            FROM reports r
            LEFT JOIN users ru ON {reporter_join}
            {resolved_by_join}
            WHERE r.id::text = :report_id
            """.format(
                status_expr=status_expr,
                resolution_expr=resolution_expr,
                resolved_by_expr=resolved_by_expr,
                resolved_by_email_expr=resolved_by_email_expr,
                moderator_comment_expr=moderator_comment_expr,
                resolved_at_expr=resolved_at_expr,
                resolved_by_join=resolved_by_join,
                reporter_join=_text_eq("ru.id", "r.reporter_id"),
            )
        ),

        {"report_id": report_id},
    ).mappings().first()
    if not row:
        raise ValueError("Report not found")
    return dict(row)


def resolve_report(
    session: Session,
    report_id: str,
    moderator_id: str,
    resolution: str,
    moderator_comment: Optional[str] = None,
) -> dict[str, Any]:
    if resolution not in {"valid", "invalid"}:
        raise ValueError("Unsupported resolution")

    report = session.execute(
        text(
            """
            SELECT id::text AS id,
                   reporter_id::text AS reporter_id,
                   target_type,
                   target_id
            FROM reports
            WHERE id::text = :report_id
            """
        ),
        {"report_id": report_id},
```

```
    ).mappings().first()
    if not report:
        raise ValueError("Report not found")

    columns = _get_table_columns(session, "reports")
    now = _now()
    update_clauses: list[str] = []
    params: dict[str, Any] = {"report_id": report_id}
    status_value = _map_status_assignment(session, "reports", "status", "resolved")
    if "status" in columns and status_value is not None:
        update_clauses.append("status = :status_value")
        params["status_value"] = status_value
    if "resolution" in columns:
        update_clauses.append("resolution = :resolution")
        params["resolution"] = resolution
    if "resolved_by" in columns:
        update_clauses.append("resolved_by = :moderator_id")
        params["moderator_id"] = moderator_id
    if "moderator_comment" in columns:
        update_clauses.append("moderator_comment = :moderator_comment")
        params["moderator_comment"] = moderator_comment
    if "resolved_at" in columns:
        update_clauses.append("resolved_at = :resolved_at")
        params["resolved_at"] = now
    if update_clauses:
         session.execute(text(f"UPDATE reports SET {', '.join(update_clauses)} WHERE id::text = :report_id"),
params)

    _add_action(
        session=session,
        moderator_id=moderator_id,
        action_type="report_resolve",
        target_type="report",
        target_id=str(report["id"]),
        reason=moderator_comment,
        payload={
            "resolution": resolution,
            "target_type": report["target_type"],
            "target_id": report["target_id"],
        },
    )
    _add_notification(
        session=session,
        user_id=str(report["reporter_id"]),
        notif_type="report",
        body=f"Жалоба обработана: {resolution}",
        payload={"report_id": str(report["id"]), "resolution": resolution},
    )
    session.commit()
    return get_report_detail(session, report_id)


def list_support_threads(session: Session, search: Optional[str] = None) -> list[dict[str, Any]]:
    user_filter = _get_chat_user_filter(session, "up", "uu")
    sql = """
        SELECT ct.id,
               ct.kind,
               ct.created_at,
               ct.last_message_at,
               up.user_id AS user_id,
               uu.email AS user_email,
               lm.text AS last_message_text,
               lm.created_at AS last_message_created_at
        FROM chat_threads ct
        JOIN chat_participants up
          ON up.thread_id = ct.id
        LEFT JOIN users uu ON {user_join}
        LEFT JOIN LATERAL (
```

```
            SELECT m.text, m.created_at
            FROM chat_messages m
            WHERE m.thread_id = ct.id
              AND m.deleted_at IS NULL
            ORDER BY m.created_at DESC
            LIMIT 1
        ) lm ON TRUE
        WHERE ct.kind = 'support'
    """
    sql = sql.format(user_join=_text_eq("uu.id", "up.user_id"))
    params: dict[str, Any] = {}
    if user_filter:
        sql += f" AND {user_filter}"
    if search:
        sql += " AND COALESCE(uu.email, '') ILIKE :search"
        params["search"] = f"%{search.strip()}%"
    sql += " ORDER BY COALESCE(ct.last_message_at, ct.created_at) DESC LIMIT 300"
    rows = session.execute(text(sql), params).mappings().all()
    return [dict(row) for row in rows]


def get_support_thread(session: Session, thread_id: str) -> dict[str, Any]:
    user_filter = _get_chat_user_filter(session, "up", "uu")
    row = session.execute(
        text(
            """
            SELECT ct.id,
                   ct.kind,
                   ct.created_at,
                   ct.last_message_at,
                   up.user_id AS user_id,
                   uu.email AS user_email
            FROM chat_threads ct
            JOIN chat_participants up
              ON up.thread_id = ct.id
            LEFT JOIN users uu ON {user_join}
            WHERE ct.id::text = :thread_id
              AND ct.kind = 'support'
              AND {user_filter}
            """
            .format(user_join=_text_eq("uu.id", "up.user_id"), user_filter=user_filter)
        ),
        {"thread_id": thread_id},
    ).mappings().first()
    if not row:
        raise ValueError("Support thread not found")
    return dict(row)


def get_support_messages(session: Session, thread_id: str) -> list[dict[str, Any]]:
    thread = session.execute(
        text("SELECT id::text AS id, kind FROM chat_threads WHERE id::text = :thread_id"),
        {"thread_id": thread_id},
    ).mappings().first()
    if not thread or str(thread["kind"]) != "support":
        raise ValueError("Support thread not found")
    sender_role_expr = (
        "COALESCE(cp.role::text, u.role::text, 'user')"
        if _has_column(session, "chat_participants", "role")
        else "COALESCE(u.role::text, 'user')"
    )
    cp_join = (
            f"LEFT JOIN chat_participants cp ON cp.thread_id = m.thread_id AND {_text_eq('cp.user_id',
'm.sender_id')}"
        if _has_column(session, "chat_participants", "role")
        else ""
    )
    rows = session.execute(
```

```
        text(
            """
            SELECT m.id,
                   m.thread_id,
                   m.sender_id,
                   u.email AS sender_email,
                   {sender_role_expr} AS sender_role,
                   m.type,
                   m.text,
                   m.is_blocked,
                   m.blocked_reason,
                   m.created_at,
                   m.edited_at,
                   m.deleted_at
            FROM chat_messages m
            LEFT JOIN users u ON {user_join}
            {cp_join}
            WHERE m.thread_id::text = :thread_id
              AND m.deleted_at IS NULL
            ORDER BY m.created_at ASC
            """
            .format(
                sender_role_expr=sender_role_expr,
                user_join=_text_eq("u.id", "m.sender_id"),
                cp_join=cp_join,
            )
        ),
        {"thread_id": thread_id},
    ).mappings().all()
    return [dict(row) for row in rows]


def post_support_message(
    session: Session,
    thread_id: str,
    sender_id: str,
    text_value: str,

) -> dict[str, Any]:
    thread = session.execute(
        text("SELECT id::text AS id, kind FROM chat_threads WHERE id::text = :thread_id"),
        {"thread_id": thread_id},
    ).mappings().first()
    if not thread or str(thread["kind"]) != "support":
        raise ValueError("Support thread not found")

    sender = _get_user_row(session, sender_id)
    if not sender or str(sender["role"]) not in STAFF_ROLES:
        raise PermissionError("Support sender must be staff")

    clean_text = " ".join((text_value or "").strip().split())
    if not clean_text:
        raise ValueError("Message text is required")

    participant_exists = session.execute(
        text(
            """
            SELECT 1
            FROM chat_participants
            WHERE thread_id::text = :thread_id
              AND user_id::text = :user_id
            LIMIT 1
            """
        ),
        {"thread_id": thread_id, "user_id": sender_id},
    ).first()
    participant_columns = _get_table_columns(session, "chat_participants")
    message_columns = _get_table_columns(session, "chat_messages")
```

```
    message_id = str(uuid.uuid4())
    created_at = _now()

    if not participant_exists:
        participant_values: Dict[str, Any] = {"thread_id": thread_id, "user_id": sender_id}
        if "role" in participant_columns:
            participant_values["role"] = "support"
        if "joined_at" in participant_columns:
            participant_values["joined_at"] = created_at
        if "left_at" in participant_columns:
            participant_values["left_at"] = None
        if "last_read_message_id" in participant_columns:
            participant_values["last_read_message_id"] = None
        _insert_row(session, "chat_participants", participant_values)

    message_values: Dict[str, Any] = {
        "id": message_id,
        "thread_id": thread_id,
        "sender_id": sender_id,
        "type": "text",
        "text": clean_text,
    }
    if "is_blocked" in message_columns:
        message_values["is_blocked"] = False
    if "blocked_reason" in message_columns:
        message_values["blocked_reason"] = None
    if "created_at" in message_columns:
        message_values["created_at"] = created_at
    if "edited_at" in message_columns:
        message_values["edited_at"] = None
    if "deleted_at" in message_columns:
        message_values["deleted_at"] = None
    _insert_row(session, "chat_messages", message_values)
    session.execute(
        text("UPDATE chat_threads SET last_message_at = :last_message_at WHERE id::text = :thread_id"),
        {"thread_id": thread_id, "last_message_at": created_at},
    )
    if "last_read_message_id" in participant_columns:
        session.execute(
            text(
                """
                UPDATE chat_participants
                SET last_read_message_id = :message_id
                WHERE thread_id::text = :thread_id
                  AND user_id::text = :user_id
                """
            ),
            {"thread_id": thread_id, "user_id": sender_id, "message_id": message_id},
        )

    user_filter = _get_chat_user_filter(session, "cpu", "uu")
    user_participant = session.execute(
        text(
            """
            SELECT cpu.user_id::text AS user_id
            FROM chat_participants cpu
            LEFT JOIN users uu ON {user_join}
            WHERE cpu.thread_id::text = :thread_id
              AND {user_filter}
            LIMIT 1
            """.format(user_join=_text_eq("uu.id", "cpu.user_id"), user_filter=user_filter)
        ),
        {"thread_id": thread_id},
    ).mappings().first()
    if user_participant:
        _add_notification(
            session=session,
            user_id=str(user_participant["user_id"]),
```

```
            notif_type="support",
            body=clean_text,
            payload={"thread_id": thread_id, "message_id": message_id},
        )

    session.commit()
    return {
        "id": message_id,
        "thread_id": thread_id,
        "sender_id": sender_id,
        "sender_email": sender["email"],
        "sender_role": "support",
        "type": "text",
        "text": clean_text,
        "is_blocked": False,
        "blocked_reason": None,
        "created_at": created_at,
        "edited_at": None,
        "deleted_at": None,
    }


def list_restrictions(
    session: Session,
    search: Optional[str] = None,
    status: Optional[str] = "active",
) -> list[dict[str, Any]]:
    status_expr = _restriction_status_expr(session, "r")
    issued_by_exists = _has_column(session, "user_restrictions", "issued_by")
    issued_by_join = f"LEFT JOIN users iu ON {_text_eq('iu.id', 'r.issued_by')}" if issued_by_exists else ""
    type_expr = _text_expr("r.type") if _has_column(session, "user_restrictions", "type") else "NULL::text"
    issued_by_expr = _text_expr("r.issued_by") if issued_by_exists else "NULL::text"
    issued_by_email_expr = "iu.email" if issued_by_exists else "NULL::text"
    starts_at_expr = _restriction_timestamp_expr(session, "r")
            ends_at_expr   =   "r.ends_at"   if   _has_column(session,   "user_restrictions",   "ends_at")   else
"NULL::timestamptz"
        revoked_at_expr  =  "r.revoked_at"  if  _has_column(session,  "user_restrictions",  "revoked_at")  else
"NULL::timestamptz"
    sql = """
        SELECT r.id,
               r.user_id,
               uu.email AS user_email,
               {type_expr} AS type,
               {status_expr} AS status,
               {issued_by_expr} AS issued_by,
               {issued_by_email_expr} AS issued_by_email,
               {starts_at_expr} AS starts_at,
               {ends_at_expr} AS ends_at,
               {revoked_at_expr} AS revoked_at
        FROM user_restrictions r
        LEFT JOIN users uu ON {user_join}
        {issued_by_join}
        WHERE 1 = 1
    """.format(
        type_expr=type_expr,
        status_expr=status_expr,
        issued_by_expr=issued_by_expr,
        issued_by_email_expr=issued_by_email_expr,
        starts_at_expr=starts_at_expr,
        ends_at_expr=ends_at_expr,
        revoked_at_expr=revoked_at_expr,
        issued_by_join=issued_by_join,
        user_join=_text_eq("uu.id", "r.user_id"),
    )
    params: dict[str, Any] = {}
    if status:
        sql += f" AND {status_expr} = :status"
        params["status"] = status
```

```
    if search:
        sql += " AND COALESCE(uu.email, '') ILIKE :search"
        params["search"] = f"%{search.strip()}%"
    sql += f" ORDER BY {starts_at_expr} DESC NULLS LAST, r.id DESC LIMIT 300"
    rows = session.execute(text(sql), params).mappings().all()
    return [dict(row) for row in rows]


def list_moderation_actions(
    session: Session,
    action_type: Optional[str] = None,
    target_type: Optional[str] = None,
    moderator_search: Optional[str] = None,
) -> list[dict[str, Any]]:
    sql = """
        SELECT a.id,
               a.moderator_id,
               u.email AS moderator_email,
               a.action_type,
               a.target_type,
               a.target_id,
               a.reason,
               a.payload,
               a.created_at
        FROM moderation_actions a
        LEFT JOIN users u ON {user_join}
        WHERE 1 = 1
    """
    sql = sql.format(user_join=_text_eq("u.id", "a.moderator_id"))
    params: dict[str, Any] = {}
    if action_type:
        sql += " AND a.action_type = :action_type"
        params["action_type"] = action_type
    if target_type:
        sql += " AND a.target_type = :target_type"
        params["target_type"] = target_type

    if moderator_search:
        sql += " AND COALESCE(u.email, '') ILIKE :moderator_search"
        params["moderator_search"] = f"%{moderator_search.strip()}%"
    sql += " ORDER BY a.created_at DESC LIMIT 500"
    rows = session.execute(text(sql), params).mappings().all()
    return [dict(row) for row in rows]


def create_restriction(
    session: Session,
    user_id: str,
    restriction_type: str,
    moderator_id: str,
    ends_at: Optional[datetime] = None,
    comment: Optional[str] = None,
) -> dict[str, Any]:
    if restriction_type not in {"warning", "ban", "shadowban"}:
        raise ValueError("Unsupported restriction type")

    target_user = _get_user_row(session, user_id)
    if not target_user:
        raise ValueError("User not found")

    restriction_id = str(uuid.uuid4())
    now = _now()
    columns = _get_table_columns(session, "user_restrictions")
    values: Dict[str, Any] = {"id": restriction_id, "user_id": user_id}
    if "type" in columns:
        values["type"] = restriction_type
    status_value = _map_status_assignment(session, "user_restrictions", "status", "active")
    if "status" in columns and status_value is not None:
```

```
        values["status"] = status_value
    if "issued_by" in columns:
        values["issued_by"] = moderator_id
    if "starts_at" in columns:
        values["starts_at"] = now
    elif "created_at" in columns:
        values["created_at"] = now
    if "ends_at" in columns:
        values["ends_at"] = ends_at
    if "reason" in columns and comment:
        values["reason"] = comment
    _insert_row(session, "user_restrictions", values)
    _add_action(
        session=session,
        moderator_id=moderator_id,
        action_type="restriction_set",
        target_type="user",
        target_id=user_id,
        reason=comment,
        payload={"type": restriction_type, "ends_at": ends_at.isoformat() if ends_at else None},
    )
    _add_notification(
        session=session,
        user_id=user_id,
        notif_type="moderation",
        body=f"Для вашей учётной записи установлено ограничение: {restriction_type}",
        payload={"restriction_id": restriction_id, "type": restriction_type},
    )
    session.commit()
    rows = [item for item in list_restrictions(session, status=None) if str(item["id"]) == restriction_id]
    return rows[0] if rows else {
        "id": restriction_id,
        "user_id": user_id,
        "type": restriction_type,
        "status": "active",
        "issued_by": moderator_id,
        "issued_by_email": None,
        "starts_at": now,
        "ends_at": ends_at,
        "revoked_at": None,
    }


def revoke_restriction(
    session: Session,
    restriction_id: str,
    moderator_id: str,
    comment: Optional[str] = None,
) -> dict[str, Any]:
    type_expr = "type" if _has_column(session, "user_restrictions", "type") else "NULL::text AS type"
    restriction = session.execute(
        text(
            f"""
            SELECT id, user_id, {type_expr}
            FROM user_restrictions
            WHERE id = :restriction_id
            """
        ),
        {"restriction_id": restriction_id},
    ).mappings().first()
    if not restriction:
        raise ValueError("Restriction not found")

    columns = _get_table_columns(session, "user_restrictions")
    now = _now()
    update_clauses: list[str] = []
    params: dict[str, Any] = {"restriction_id": restriction_id}
    status_value = _map_status_assignment(session, "user_restrictions", "status", "revoked")
```

```
    if "status" in columns and status_value is not None:
        update_clauses.append("status = :status_value")
        params["status_value"] = status_value
    if "revoked_at" in columns:
        update_clauses.append("revoked_at = :revoked_at")
        params["revoked_at"] = now
    if "reason" in columns and comment:
        update_clauses.append("reason = :reason")
        params["reason"] = comment
    if update_clauses:
        session.execute(
                        text(f"UPDATE  user_restrictions  SET  {',  '.join(update_clauses)}  WHERE  id::text  =
:restriction_id"),
            params,
        )
    _add_action(
        session=session,
        moderator_id=moderator_id,
        action_type="restriction_revoke",
        target_type="user",
        target_id=str(restriction["user_id"]),
        reason=comment,
        payload={"restriction_id": str(restriction["id"]), "type": restriction.get("type")},
    )
    _add_notification(
        session=session,
        user_id=str(restriction["user_id"]),
        notif_type="moderation",
        body=f"Ограничение снято: {restriction.get('type') or 'restriction'}",
        payload={"restriction_id": str(restriction['id'])},
    )
    session.commit()
    rows = [item for item in list_restrictions(session, status=None) if str(item["id"]) == restriction_id]
    return rows[0] if rows else {
        "id": restriction_id,
        "user_id": str(restriction["user_id"]),
        "type": restriction.get("type"),
        "status": "revoked",
        "issued_by": None,
        "issued_by_email": None,
        "starts_at": None,
        "ends_at": None,
        "revoked_at": now,
    }



// ===== File 23/28: services/admin_panel/app/db.py =====
from __future__ import annotations

from collections.abc import Generator

from sqlalchemy import create_engine
from sqlalchemy.orm import Session, sessionmaker

from .settings import get_settings


settings = get_settings()


def _normalize_database_url(database_url: str) -> str:
    if database_url.startswith("postgresql+psycopg://"):
        return database_url
    if database_url.startswith("postgresql://"):
        return database_url.replace("postgresql://", "postgresql+psycopg://", 1)
    if database_url.startswith("postgres://"):
        return database_url.replace("postgres://", "postgresql+psycopg://", 1)
```

```
    return database_url

engine = create_engine(
    _normalize_database_url(settings.database_url),
    future=True,
    pool_pre_ping=True,
)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False, expire_on_commit=False)


def get_session() -> Generator[Session, None, None]:
    session = SessionLocal()
    try:
        yield session
    finally:
        session.close()



// ===== File 24/28: services/admin_panel/app/main.py =====
from __future__ import annotations

import os
from pathlib import Path

from fastapi import APIRouter, Depends, FastAPI, HTTPException, Request
from fastapi.responses import RedirectResponse
from fastapi.staticfiles import StaticFiles
from sqladmin import Admin
from starlette.middleware.sessions import SessionMiddleware
from starlette.templating import Jinja2Templates

from app.bootstrap import ensure_all_tables

from . import crud
from .auth import AdminAuth, require_admin_user, require_staff_user
from .db import SessionLocal, engine
from .schemas import (
    AnnouncementDecisionIn,
    ReportResolutionIn,
    RestrictionCreateIn,
    RestrictionRevokeIn,
    RoleUpdateIn,
    SupportReplyIn,
)
from .settings import get_settings
from .views_sqladmin import (
    AnnouncementsModerationView,
    ModerationActionsView,
    ReportsView,
    RestrictionsView,
    SupportThreadsView,
    UsersView,
)


settings = get_settings()
uploads_dir = Path(os.getenv("UPLOADS_DIR", "uploads"))


class StaffAdmin(Admin):
    async def index(self, request: Request) -> RedirectResponse:
        return RedirectResponse(url="/admin/users", status_code=302)


app = FastAPI(title=settings.title)
app.add_middleware(SessionMiddleware, secret_key=settings.session_secret)
app.mount("/static", StaticFiles(directory=str(settings.static_dir)), name="admin-static")
```

```
app.mount("/uploads", StaticFiles(directory=str(uploads_dir), check_dir=False), name="admin-uploads")
app.state.templates = Jinja2Templates(directory=str(settings.templates_dir))


@app.on_event("startup")
def startup() -> None:
    ensure_all_tables()


@app.get("/")
def root() -> RedirectResponse:
    return RedirectResponse(url="/admin/users", status_code=302)


@app.get("/health")
def health() -> dict[str, str]:
    return {"status": "ok"}


authentication_backend = AdminAuth(secret_key=settings.session_secret)
admin = StaffAdmin(
    app=app,
    engine=engine,
    authentication_backend=authentication_backend,
    base_url=settings.admin_base_url,
    title=settings.title,
    templates_dir=str(settings.templates_dir),
)
admin.add_view(UsersView)
admin.add_view(AnnouncementsModerationView)
admin.add_view(ReportsView)
admin.add_view(SupportThreadsView)
admin.add_view(RestrictionsView)
admin.add_view(ModerationActionsView)


api = APIRouter(prefix="/admin/api")


@api.get("/support/threads")
def api_support_threads(search: str | None = None, _: object = Depends(require_staff_user)):
    with SessionLocal() as session:
        return crud.list_support_threads(session, search=search)


@api.get("/support/threads/{thread_id}/messages")
def api_support_thread_messages(thread_id: str, _: object = Depends(require_staff_user)):
    with SessionLocal() as session:
        try:
            return crud.get_support_messages(session, thread_id)
        except ValueError as exc:
            raise HTTPException(status_code=404, detail=str(exc)) from exc


@api.post("/support/threads/{thread_id}/messages")
def api_support_thread_reply(
    thread_id: str,
    payload: SupportReplyIn,
    request: Request,
    _: object = Depends(require_staff_user),
):
    actor = require_staff_user(request)
    with SessionLocal() as session:
        try:
            return crud.post_support_message(session, thread_id, actor.id, payload.text)
        except PermissionError as exc:
            raise HTTPException(status_code=403, detail=str(exc)) from exc
        except ValueError as exc:
```

```
            raise HTTPException(status_code=400, detail=str(exc)) from exc


@api.post("/announcements/{ann_id}/decision")
def api_announcement_decision(
    ann_id: str,
    payload: AnnouncementDecisionIn,
    request: Request,
    _: object = Depends(require_staff_user),
):
    actor = require_staff_user(request)
    with SessionLocal() as session:
        try:
            announcement = crud.apply_announcement_decision(
                session=session,
                ann_id=ann_id,
                moderator_id=actor.id,
                decision=payload.decision,
                message=payload.message,
                reasons=[reason.model_dump() for reason in payload.reasons],
                suggestions=payload.suggestions,
            )
        except ValueError as exc:
            raise HTTPException(status_code=400, detail=str(exc)) from exc
    return {
        "id": announcement.id,
        "status": announcement.status,
        "deleted_at": announcement.deleted_at,
        "updated_at": announcement.updated_at,
    }


@api.post("/reports/{report_id}/resolve")
def api_resolve_report(
    report_id: str,
    payload: ReportResolutionIn,
    request: Request,
    _: object = Depends(require_staff_user),
):
    actor = require_staff_user(request)
    with SessionLocal() as session:
        try:
            report = crud.resolve_report(
                session=session,
                report_id=report_id,
                moderator_id=actor.id,
                resolution=payload.resolution,
                moderator_comment=payload.moderator_comment,
            )
        except ValueError as exc:
            raise HTTPException(status_code=400, detail=str(exc)) from exc
    return {
        "id": report["id"],
        "status": report["status"],
        "resolution": report["resolution"],
        "resolved_at": report["resolved_at"],
    }


@api.post("/restrictions")
def api_create_restriction(
    payload: RestrictionCreateIn,
    request: Request,
    _: object = Depends(require_staff_user),
):
    actor = require_staff_user(request)
    with SessionLocal() as session:
        try:
```

```
            restriction = crud.create_restriction(
                session=session,
                user_id=payload.user_id,
                restriction_type=payload.type,
                moderator_id=actor.id,
                ends_at=payload.ends_at,
                comment=payload.comment,
            )
        except ValueError as exc:
            raise HTTPException(status_code=400, detail=str(exc)) from exc
    return {
        "id": restriction["id"],
        "user_id": restriction["user_id"],
        "type": restriction["type"],
        "status": restriction["status"],

    }


@api.post("/restrictions/{restriction_id}/revoke")
def api_revoke_restriction(
    restriction_id: str,
    payload: RestrictionRevokeIn,
    request: Request,
    _: object = Depends(require_staff_user),
):
    actor = require_staff_user(request)
    with SessionLocal() as session:
        try:
            restriction = crud.revoke_restriction(
                session=session,
                restriction_id=restriction_id,
                moderator_id=actor.id,
                comment=payload.comment,
            )
        except ValueError as exc:
            raise HTTPException(status_code=400, detail=str(exc)) from exc
    return {
        "id": restriction["id"],
        "status": restriction["status"],
        "revoked_at": restriction["revoked_at"],
    }


@api.post("/users/{user_id}/role")
def api_update_user_role(
    user_id: str,
    payload: RoleUpdateIn,
    request: Request,
    _: object = Depends(require_admin_user),
):
    actor = require_admin_user(request)
    with SessionLocal() as session:
        try:
            user = crud.update_user_role(session, user_id, payload.role, actor.id, actor.role)
        except PermissionError as exc:
            raise HTTPException(status_code=403, detail=str(exc)) from exc
        except ValueError as exc:
            raise HTTPException(status_code=400, detail=str(exc)) from exc
    return user


app.include_router(api)



// ===== File 25/28: services/admin_panel/app/models_sqlalchemy.py =====
from __future__ import annotations
```

```python
from datetime import datetime
from typing import Any, Dict, Optional

from sqlalchemy import Boolean, DateTime, String, Text, text as sa_text
from sqlalchemy.dialects.postgresql import JSONB
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column


class Base(DeclarativeBase):
    pass


class User(Base):
    __tablename__ = "users"

    id: Mapped[str] = mapped_column(String, primary_key=True)
    email: Mapped[str] = mapped_column(String, nullable=False, unique=True)
    password_hash: Mapped[str] = mapped_column(String, nullable=False)
    role: Mapped[str] = mapped_column(String, nullable=False, default="user")
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)


class Announcement(Base):
    __tablename__ = "announcements"

    id: Mapped[str] = mapped_column(String, primary_key=True)
    user_id: Mapped[str] = mapped_column(String, nullable=False)
    category: Mapped[str] = mapped_column(String, nullable=False)
    title: Mapped[str] = mapped_column(String, nullable=False)
    status: Mapped[str] = mapped_column(String, nullable=False, default="active")
    data: Mapped[Dict[str, Any]] = mapped_column(
        JSONB,
        nullable=False,
        default=dict,
        server_default=sa_text("'{}'::jsonb"),
    )
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)
    updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)
    deleted_at: Mapped[Optional[datetime]] = mapped_column(DateTime(timezone=True), nullable=True)


class ChatThread(Base):
    __tablename__ = "chat_threads"

    id: Mapped[str] = mapped_column(String, primary_key=True)
    kind: Mapped[str] = mapped_column(String, nullable=False)
    task_id: Mapped[Optional[str]] = mapped_column(String, nullable=True)
    offer_id: Mapped[Optional[str]] = mapped_column(String, nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)
    last_message_at: Mapped[Optional[datetime]] = mapped_column(DateTime(timezone=True), nullable=True)


class ChatParticipant(Base):
    __tablename__ = "chat_participants"

    thread_id: Mapped[str] = mapped_column(String, primary_key=True)
    user_id: Mapped[str] = mapped_column(String, primary_key=True)
    role: Mapped[str] = mapped_column(String, nullable=False)
    joined_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)
    left_at: Mapped[Optional[datetime]] = mapped_column(DateTime(timezone=True), nullable=True)
    last_read_message_id: Mapped[Optional[str]] = mapped_column(String, nullable=True)


class ChatMessage(Base):
    __tablename__ = "chat_messages"

    id: Mapped[str] = mapped_column(String, primary_key=True)
```

```
    thread_id: Mapped[str] = mapped_column(String, nullable=False)
    sender_id: Mapped[str] = mapped_column(String, nullable=False)
    type: Mapped[str] = mapped_column(String, nullable=False)
    text: Mapped[str] = mapped_column(Text, nullable=False)
    is_blocked: Mapped[bool] = mapped_column(
        Boolean,
        nullable=False,
        default=False,
        server_default=sa_text("FALSE"),
    )
    blocked_reason: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)
    edited_at: Mapped[Optional[datetime]] = mapped_column(DateTime(timezone=True), nullable=True)
    deleted_at: Mapped[Optional[datetime]] = mapped_column(DateTime(timezone=True), nullable=True)


class Report(Base):
    __tablename__ = "reports"

    id: Mapped[str] = mapped_column(String, primary_key=True)
    reporter_id: Mapped[str] = mapped_column(String, nullable=False)
    target_type: Mapped[str] = mapped_column(String, nullable=False)
    target_id: Mapped[str] = mapped_column(String, nullable=False)
    reason_code: Mapped[str] = mapped_column(String, nullable=False)
    reason_text: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    status: Mapped[str] = mapped_column(String, nullable=False)
    resolution: Mapped[Optional[str]] = mapped_column(String, nullable=True)
    resolved_by: Mapped[Optional[str]] = mapped_column(String, nullable=True)
    moderator_comment: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)
    resolved_at: Mapped[Optional[datetime]] = mapped_column(DateTime(timezone=True), nullable=True)


class ModerationAction(Base):
    __tablename__ = "moderation_actions"

    id: Mapped[str] = mapped_column(String, primary_key=True)
    moderator_id: Mapped[str] = mapped_column(String, nullable=False)
    action_type: Mapped[str] = mapped_column(String, nullable=False)
    target_type: Mapped[str] = mapped_column(String, nullable=False)
    target_id: Mapped[str] = mapped_column(String, nullable=False)
    reason: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    payload: Mapped[Dict[str, Any]] = mapped_column(
        JSONB,
        nullable=False,
        default=dict,
        server_default=sa_text("'{}'::jsonb"),
    )
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)


class UserRestriction(Base):
    __tablename__ = "user_restrictions"

    id: Mapped[str] = mapped_column(String, primary_key=True)
    user_id: Mapped[str] = mapped_column(String, nullable=False)
    type: Mapped[str] = mapped_column(String, nullable=False)
    status: Mapped[str] = mapped_column(String, nullable=False)
    issued_by: Mapped[str] = mapped_column(String, nullable=False)
    starts_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)
    ends_at: Mapped[Optional[datetime]] = mapped_column(DateTime(timezone=True), nullable=True)
    revoked_at: Mapped[Optional[datetime]] = mapped_column(DateTime(timezone=True), nullable=True)


class Notification(Base):
    __tablename__ = "notifications"

    id: Mapped[str] = mapped_column(String, primary_key=True)
```

```
    user_id: Mapped[str] = mapped_column(String, nullable=False)
    type: Mapped[str] = mapped_column(String, nullable=False)
    body: Mapped[str] = mapped_column(Text, nullable=False)
    payload: Mapped[Dict[str, Any]] = mapped_column(
        JSONB,
        nullable=False,
        default=dict,
        server_default=sa_text("'{}'::jsonb"),
    )
    is_read: Mapped[bool] = mapped_column(
        Boolean,
        nullable=False,
        default=False,
        server_default=sa_text("FALSE"),
    )
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)
    read_at: Mapped[Optional[datetime]] = mapped_column(DateTime(timezone=True), nullable=True)



// ===== File 26/28: services/admin_panel/app/schemas.py =====
from __future__ import annotations

from datetime import datetime
from typing import Literal, Optional

from pydantic import BaseModel, EmailStr, Field


class AnnouncementDecisionReason(BaseModel):
    field: str = Field(..., min_length=1, max_length=64)
    code: str = Field(..., min_length=1, max_length=64)
    details: str = Field(..., min_length=1, max_length=2000)
    can_appeal: bool = True


class AnnouncementDecisionIn(BaseModel):
    decision: Literal["approve", "needs_fix", "reject", "archive", "delete"]
    message: str = Field(..., min_length=1, max_length=2000)
    reasons: list[AnnouncementDecisionReason] = Field(default_factory=list)
    suggestions: list[str] = Field(default_factory=list)


class SupportReplyIn(BaseModel):
    text: str = Field(..., min_length=1, max_length=5000)


class ReportResolutionIn(BaseModel):
    resolution: Literal["valid", "invalid"]
    moderator_comment: Optional[str] = Field(default=None, max_length=2000)


class RestrictionCreateIn(BaseModel):
    user_id: str = Field(..., min_length=1, max_length=128)
    type: Literal["warning", "ban", "shadowban"]
    ends_at: Optional[datetime] = None
    comment: Optional[str] = Field(default=None, max_length=2000)


class RestrictionRevokeIn(BaseModel):
    comment: Optional[str] = Field(default=None, max_length=2000)


class RoleUpdateIn(BaseModel):
    role: Literal["user", "support", "moderator", "admin"]


class StaffUserOut(BaseModel):
```

```
    id: str
    email: EmailStr
    role: str



// ===== File 27/28: services/admin_panel/app/settings.py =====
from __future__ import annotations

import os
from dataclasses import dataclass
from functools import lru_cache
from pathlib import Path

from dotenv import load_dotenv

load_dotenv()


APP_DIR = Path(__file__).resolve().parent
SERVICE_DIR = APP_DIR.parent


@dataclass(frozen=True)
class Settings:
    database_url: str
    jwt_secret: str
    jwt_alg: str
    session_secret: str
    admin_base_url: str
    title: str
    templates_dir: Path
    static_dir: Path


@lru_cache
def get_settings() -> Settings:
    database_url = os.getenv("DATABASE_URL", "").strip()
    if not database_url:
        raise RuntimeError("DATABASE_URL is not set")

    jwt_secret = os.getenv("JWT_SECRET", "DEV_JWT_SECRET_CHANGE_ME")
    jwt_alg = os.getenv("JWT_ALG", "HS256")
    session_secret = os.getenv("ADMIN_SESSION_SECRET", jwt_secret)

    return Settings(
        database_url=database_url,
        jwt_secret=jwt_secret,
        jwt_alg=jwt_alg,
        session_secret=session_secret,
        admin_base_url=os.getenv("ADMIN_BASE_URL", "/admin"),
        title=os.getenv("ADMIN_TITLE", "Vzaimno"),
        templates_dir=APP_DIR / "templates",
        static_dir=APP_DIR / "static",
    )



// ===== File 28/28: services/admin_panel/app/views_sqladmin.py =====
from __future__ import annotations

from datetime import datetime, timezone
from typing import Any, Optional

from fastapi import HTTPException, Request
from sqladmin import BaseView, expose
from starlette.responses import RedirectResponse
```

```
from . import crud
from .auth import require_admin_user, require_staff_user
from .db import SessionLocal

UI_LABELS = {
    "roles": {
        "user": "Пользователь",
        "support": "Поддержка",
        "moderator": "Модератор",
        "admin": "Администратор",
    },
    "announcement_statuses": {
        "pending_review": "На проверке",
        "needs_fix": "Нужно исправить",
        "rejected": "Отклонено",
        "active": "Активно",
        "archived": "В архиве",
        "draft": "Черновик",
    },
    "restriction_types": {
        "warning": "Предупреждение",
        "ban": "Блокировка",
        "shadowban": "Теневая блокировка",
    },
    "restriction_statuses": {
        "active": "Активно",
        "revoked": "Снято",
    },
    "report_statuses": {
        "open": "Открыта",
        "resolved": "Закрыта",
    },
    "report_resolutions": {
        "valid": "Обоснована",
        "invalid": "Не обоснована",
    },
    "target_types": {
        "announcement": "Объявление",
        "message": "Сообщение",
        "user": "Пользователь",
        "task": "Задание",
        "report": "Жалоба",
    },
    "sender_roles": {
        "user": "Пользователь",
        "support": "Поддержка",
        "moderator": "Модератор",
        "admin": "Администратор",
    },
    "actions": {
        "approve": "Одобрить",
        "needs_fix": "Отправить на доработку",
        "reject": "Отклонить",
        "archive": "Архивировать",
        "delete": "Удалить",
        "report_resolve": "Решение по жалобе",
        "restriction_set": "Назначено ограничение",
        "restriction_revoke": "Ограничение снято",
        "user_role_change": "Смена роли",
    },
}


def _format_dt(value: Any) -> str:
    if value in (None, ""):
        return "—"
    if isinstance(value, datetime):
        parsed = value
```

```
    else:
        raw = str(value).strip()
        if not raw:
            return "—"
        try:
            normalized = raw.replace("Z", "+00:00")
            parsed = datetime.fromisoformat(normalized)
        except ValueError:
            return raw
    if parsed.tzinfo is None:
        parsed = parsed.replace(tzinfo=timezone.utc)
    return parsed.astimezone().strftime("%d.%m.%Y %H:%M")


def _base_context(request: Request, **extra: Any) -> dict[str, Any]:
    staff_user = require_staff_user(request)
    context = {
        "request": request,
        "staff_user": staff_user,
        "is_admin": staff_user.role == "admin",
        "admin_base_url": "/admin",
        "labels": UI_LABELS,
        "format_dt": _format_dt,
    }
    context.update(extra)
    return context


async def _render(view: BaseView, request: Request, template_name: str, **extra: Any):
    templates = view.templates
    return await templates.TemplateResponse(request, template_name, _base_context(request, **extra))


def _redirect(url: str) -> RedirectResponse:
    return RedirectResponse(url=url, status_code=303)


def _parse_optional_dt(value: str) -> Optional[datetime]:
    raw = (value or "").strip()
    if not raw:
        return None
    parsed = datetime.fromisoformat(raw)
    if parsed.tzinfo is None:
        parsed = parsed.replace(tzinfo=timezone.utc)
    return parsed


class UsersView(BaseView):
    name = "Пользователи"
    identity = "users"
    icon = "fa-solid fa-users"

    @expose("/users", methods=["GET"], identity="users")
    async def a_index(self, request: Request):
        search = (request.query_params.get("search") or "").strip()
        with SessionLocal() as session:
            users = crud.list_users(session, search or None)
        return await _render(self, request, "users.html", users=users, search=search)

    @expose("/users/{user_id}", methods=["GET"], identity="users-detail")
    async def user_detail(self, request: Request):
        user_id = request.path_params["user_id"]
        with SessionLocal() as session:
            try:
                user = crud.get_user_detail(session, user_id)
            except ValueError as exc:
                raise HTTPException(status_code=404, detail=str(exc)) from exc
        return await _render(self, request, "user_detail.html", user=user)
```

```
    @expose("/users/{user_id}/role", methods=["POST"], identity="users-update-role")
    async def update_role(self, request: Request):
        actor = require_admin_user(request)
        user_id = request.path_params["user_id"]
        form = await request.form()
        role = str(form.get("role", "")).strip()
        with SessionLocal() as session:
            try:
                crud.update_user_role(session, user_id, role, actor.id, actor.role)
            except PermissionError as exc:
                raise HTTPException(status_code=403, detail=str(exc)) from exc
            except ValueError as exc:
                raise HTTPException(status_code=400, detail=str(exc)) from exc
        return _redirect(f"/admin/users/{user_id}")


class AnnouncementsModerationView(BaseView):
    name = "Модерация"
    identity = "announcements"
    icon = "fa-solid fa-shield-halved"

    @expose("/announcements", methods=["GET"], identity="announcements")
    async def a_index(self, request: Request):
        status_filter = (request.query_params.get("status") or "").strip() or None
        appeals_only = (request.query_params.get("appeals") or "").strip() == "1"
        search = (request.query_params.get("search") or "").strip()
        with SessionLocal() as session:
            items = crud.list_moderation_announcements(
                session=session,
                status_filter=status_filter,
                appeals_only=appeals_only,
                search=search or None,
            )
        return await _render(
            self,
            request,
            "moderation_list.html",
            announcements=items,
            status_filter=status_filter or "",
            appeals_only=appeals_only,
            search=search,
        )

    @expose("/announcements/{ann_id}", methods=["GET"], identity="announcements-detail")
    async def announcement_detail(self, request: Request):
        ann_id = request.path_params["ann_id"]
        with SessionLocal() as session:
            try:
                announcement = crud.get_announcement_detail(session, ann_id)
            except ValueError as exc:
                raise HTTPException(status_code=404, detail=str(exc)) from exc
        return await _render(self, request, "moderation_detail.html", announcement=announcement)


    @expose("/announcements/{ann_id}/decision", methods=["POST"], identity="announcements-decision")
    async def apply_decision(self, request: Request):
        actor = require_staff_user(request)
        ann_id = request.path_params["ann_id"]
        form = await request.form()
        decision = str(form.get("decision", "")).strip()
        message = str(form.get("message", "")).strip()
        reason_field = str(form.get("reason_field", "")).strip()
        reason_code = str(form.get("reason_code", "")).strip()
        reason_details = str(form.get("reason_details", "")).strip()
        can_appeal = str(form.get("can_appeal", "true")).lower() != "false"
        suggestions_raw = str(form.get("suggestions", "")).strip()
        reasons = []
```

```
        if reason_field and reason_code and reason_details:
            reasons.append(
                {
                    "field": reason_field,
                    "code": reason_code,
                    "details": reason_details,
                    "can_appeal": can_appeal,
                }
            )
        suggestions = [line.strip() for line in suggestions_raw.splitlines() if line.strip()]
        with SessionLocal() as session:
            try:
                crud.apply_announcement_decision(
                    session=session,
                    ann_id=ann_id,
                    moderator_id=actor.id,
                    decision=decision,
                    message=message,
                    reasons=reasons,
                    suggestions=suggestions,
                )
            except ValueError as exc:
                raise HTTPException(status_code=400, detail=str(exc)) from exc
        return _redirect(f"/admin/announcements/{ann_id}")


class ReportsView(BaseView):
    name = "Жалобы"
    identity = "reports"
    icon = "fa-solid fa-flag"

    @expose("/reports", methods=["GET"], identity="reports")
    async def a_index(self, request: Request):
        search = (request.query_params.get("search") or "").strip()
        status = (request.query_params.get("status") or "open").strip() or None
        with SessionLocal() as session:
            reports = crud.list_reports(session, search=search or None, status=status)
         return await _render(self, request, "reports_list.html", reports=reports, search=search, status=status
or "")

    @expose("/reports/{report_id}", methods=["GET"], identity="reports-detail")
    async def report_detail(self, request: Request):
        report_id = request.path_params["report_id"]
        with SessionLocal() as session:
            try:
                report = crud.get_report_detail(session, report_id)
            except ValueError as exc:
                raise HTTPException(status_code=404, detail=str(exc)) from exc
        return await _render(self, request, "report_detail.html", report=report)

    @expose("/reports/{report_id}/resolve", methods=["POST"], identity="reports-resolve")
    async def resolve(self, request: Request):
        actor = require_staff_user(request)
        report_id = request.path_params["report_id"]
        form = await request.form()
        resolution = str(form.get("resolution", "")).strip()
        moderator_comment = str(form.get("moderator_comment", "")).strip() or None
        with SessionLocal() as session:
            try:
                crud.resolve_report(session, report_id, actor.id, resolution, moderator_comment)
            except ValueError as exc:
                raise HTTPException(status_code=400, detail=str(exc)) from exc
        return _redirect(f"/admin/reports/{report_id}")


class SupportThreadsView(BaseView):
    name = "Поддержка"
    identity = "support"
```

```
    icon = "fa-solid fa-headset"

    @expose("/support", methods=["GET"], identity="support")
    async def a_index(self, request: Request):
        search = (request.query_params.get("search") or "").strip()
        with SessionLocal() as session:
            threads = crud.list_support_threads(session, search or None)
        return await _render(self, request, "support_threads.html", threads=threads, search=search)

    @expose("/support/{thread_id}", methods=["GET"], identity="support-thread")
    async def thread_view(self, request: Request):
        thread_id = request.path_params["thread_id"]
        with SessionLocal() as session:
            try:
                thread = crud.get_support_thread(session, thread_id)
                messages = crud.get_support_messages(session, thread_id)
            except ValueError as exc:
                raise HTTPException(status_code=404, detail=str(exc)) from exc
        return await _render(self, request, "support_thread.html", thread=thread, messages=messages)

    @expose("/support/{thread_id}/reply", methods=["POST"], identity="support-reply")
    async def reply(self, request: Request):
        actor = require_staff_user(request)
        thread_id = request.path_params["thread_id"]
        form = await request.form()
        text_value = str(form.get("text", "")).strip()
        with SessionLocal() as session:
            try:
                crud.post_support_message(session, thread_id, actor.id, text_value)
            except PermissionError as exc:
                raise HTTPException(status_code=403, detail=str(exc)) from exc
            except ValueError as exc:
                raise HTTPException(status_code=400, detail=str(exc)) from exc
        return _redirect(f"/admin/support/{thread_id}")


class RestrictionsView(BaseView):
    name = "Ограничения"
    identity = "restrictions"
    icon = "fa-solid fa-user-lock"

    @expose("/restrictions", methods=["GET"], identity="restrictions")
    async def a_index(self, request: Request):
        search = (request.query_params.get("search") or "").strip()
        status = (request.query_params.get("status") or "active").strip() or None
        with SessionLocal() as session:
            restrictions = crud.list_restrictions(session, search=search or None, status=status)
        return await _render(
            self,
            request,
            "restrictions.html",
            restrictions=restrictions,
            search=search,
            status=status or "",
        )

    @expose("/restrictions/create", methods=["POST"], identity="restrictions-create")
    async def create(self, request: Request):
        actor = require_staff_user(request)
        form = await request.form()
        user_id = str(form.get("user_id", "")).strip()
        restriction_type = str(form.get("type", "")).strip()
        comment = str(form.get("comment", "")).strip() or None
        ends_at = _parse_optional_dt(str(form.get("ends_at", "")).strip())
        with SessionLocal() as session:
            try:
                crud.create_restriction(session, user_id, restriction_type, actor.id, ends_at, comment)
            except ValueError as exc:
```

```
                raise HTTPException(status_code=400, detail=str(exc)) from exc
        return _redirect("/admin/restrictions")

    @expose("/restrictions/{restriction_id}/revoke", methods=["POST"], identity="restrictions-revoke")
    async def revoke(self, request: Request):
        actor = require_staff_user(request)
        restriction_id = request.path_params["restriction_id"]
        form = await request.form()
        comment = str(form.get("comment", "")).strip() or None
        with SessionLocal() as session:
            try:
                crud.revoke_restriction(session, restriction_id, actor.id, comment)
            except ValueError as exc:
                raise HTTPException(status_code=400, detail=str(exc)) from exc
        return _redirect("/admin/restrictions")


class ModerationActionsView(BaseView):
    name = "Аудит"
    identity = "audit"
    icon = "fa-solid fa-clock-rotate-left"

    @expose("/audit", methods=["GET"], identity="audit")
    async def a_index(self, request: Request):
        action_type = (request.query_params.get("action_type") or "").strip() or None
        target_type = (request.query_params.get("target_type") or "").strip() or None
        moderator = (request.query_params.get("moderator") or "").strip() or None
        with SessionLocal() as session:
            actions = crud.list_moderation_actions(
                session,
                action_type=action_type,
                target_type=target_type,
                moderator_search=moderator,
            )
        return await _render(
            self,
            request,
            "audit_log.html",
            actions=actions,
            action_type=action_type or "",
            target_type=target_type or "",
            moderator=moderator or "",
        )
```